الشرطة الوطنية الموريتانية
POLICE NATIONALE MAURITANIENNE

Note de 0,00 sur 1,00 Incorrect Question 1

: Contexte

En tant que développeur mobile pour la police nationale, vous participez à la conception d'une nouvelle fonctionnalité critique : le "Live Video Triage". Elle doit permettre à un agent sur le terrain de diffuser en direct la vidéo de son téléphone vers un centre de commandement : pour une évaluation immédiate de la situation. Le plan initial, discuté entre les équipes, est le suivant

- **Équipe Produit :** A défini la user story : "En tant qu'opérateur du centre de commandement, je veux voir un flux vidéo en direct depuis l'appareil d'un agent pour évaluer la situation."
- **Équipe Backend :** Propose que l'application mobile capture la vidéo, encode chaque image en JPEG, et envoie chaque image via une requête POST distincte à un endpoint RESTful (`POST /api/stream/{id}/frame`). Ils argumentent que c'est simple, rapide à développer et réutilise l'infrastructure REST existante.
- **Équipe UX/UI :** A fourni les maquettes pour un simple bouton "Démarrer/Arrêter la diffusion".

Conformément à votre fiche de poste qui stipule que vous devez "Analyser les besoins", "Participer à la conception technique et à l'architecture" et "Collaborer avec les équipes", vous devez évaluer cette proposition.

: Question

Parmi les actions suivantes, laquelle représente la démarche initiale la plus rigoureuse et la plus responsable que vous devriez entreprendre en tant que développeur mobile ?

a. Rejeter la proposition REST et exiger l'utilisation de WebSockets pour envoyer un flux vidéo encodé en H.264. Cela réduit la latence des requêtes HTTP multiples et est plus adapté au streaming.

Incorrect. Bien que techniquement supérieur à l'option REST, cette approche est sous-optimale et non collaborative ("exiger"). Elle ignore les protocoles standards et robustes comme WebRTC, qui sont spécifiquement conçus pour le streaming média et gèrent nativement des problématiques complexes (jitter, perte de paquets, synchronisation). Cela montre une connaissance technique partielle et un manque de collaboration.

b. Organiser immédiatement une réunion de "refinement" Agile avec toutes les équipes pour décomposer la user story en tâches techniques plus petites. C'est la procédure standard pour s'assurer que tout le monde est aligné avant de commencer le développement.

c. Proposer de remplacer l'endpoint REST par une "subscription" GraphQL. Cela permettrait une communication en temps réel plus moderne et s'alignerait avec les compétences listées dans la fiche de poste, tout en optimisant les échanges de données.

d. Accepter la proposition du backend pour livrer rapidement un prototype, en prévoyant d'optimiser les performances plus tard. Cette approche Agile permet de valider rapidement le besoin fonctionnel.

e. Aucune des réponses proposées.

: Analyse technique

La question évalue la capacité du candidat à identifier une faille architecturale fondamentale dans un projet et à réagir de manière constructive et experte, conformément aux responsabilités de la fiche de poste (analyse, conception architecturale, collaboration, veille technologique). La proposition initiale du backend (streaming vidéo via des JPEGs sur une API REST) est techniquement irréalisable pour une application en temps réel critique en raison de la latence, de la surcharge réseau et du manque de synchronisation.

: Justification

Aucune des options proposées ne constitue une réponse complète et adéquate. Un expert doit non seulement identifier les problèmes dans les solutions A, B, C et D, mais aussi savoir quelle est la bonne approche, qui n'est pas listée.

: La démarche correcte consisterait à

1. **Identifier et communiquer clairement** les raisons pour lesquelles l'approche REST/JPEG est vouée à l'échec (performance, coût, latence).
2. **Provoquer une discussion architecturale collaborative** avec les équipes backend et produit.
3. **Proposer une veille technologique** sur les protocoles standards de l'industrie pour le streaming vidéo en temps réel, comme **WebRTC** (idéal pour la faible latence) ou **RTSP/SRT**, en expliquant leurs avantages (gestion de la gigue, correction des pertes de paquets, synchronisation) et leurs contraintes (complexité, infrastructure serveur nécessaire comme un SFU/MCU).

"Puisque cette démarche n'est décrite dans aucune des options, la seule réponse correcte est "Aucune des réponses proposées.

La réponse correcte est : Aucune des réponses proposées.

Note de 0,50 sur 1,00 Partiellement correct Question 2

: Contexte

La police nationale mauritanienne doit développer une nouvelle application critique pour ses agents sur le terrain. Les exigences fondamentales sont : 1) des performances extrêmes pour l'affichage de cartes tactiques et les communications en temps réel, 2) une intégration profonde avec le matériel du terminal (ex: Secure Element/Keystore pour l'authentification, API avancées de la caméra), 3) un niveau de sécurité maximal contre la rétro-ingénierie et le 'tampering', et 4) une rapidité de développement pour déployer un prototype à un groupe pilote. L'équipe de développement actuelle possède des compétences en Kotlin et Swift, ainsi que des connaissances en JavaScript.

En tant qu'architecte de la solution, vous devez évaluer les options technologiques : Développement Natif (Kotlin/Swift), Flutter, et React Native.

: Question

Parmi les affirmations suivantes, lesquelles représentent avec précision les compromis (trade-offs) critiques et la recommandation architecturale la plus logique pour ce contexte spécifique ?

a. Flutter et React Native, étant tous deux cross-platform, partagent la même architecture basée sur un pont (bridge) JavaScript pour communiquer avec les modules natifs, ce qui limite leurs performances pour les tâches graphiques intensives.

b. Aucune des réponses proposées.

c. React Native est le choix optimal car il permet de capitaliser sur les compétences JavaScript de l'équipe, réduisant ainsi le temps de développement. Ses performances, bien qu'inférieures au natif, sont suffisantes pour la plupart des cas d'usage, et l'accès au matériel peut se faire via des bibliothèques tierces fiables.

d. Flutter pourrait être envisagé pour un prototypage rapide grâce à sa base de code unique et son "hot reload". Cependant, pour la version de production, des défis subsisteraient pour l'intégration de modules matériels très spécifiques non couverts par des plugins existants, nécessitant le développement de "platform channels" complexes et potentiellement coûteux en maintenance.

e. Le développement natif (Kotlin/Swift) est la recommandation la plus prudente. Il offre des performances graphiques et de calcul inégalées, un accès direct et complet aux API matérielles (Secure Enclave/Keystore, API de caméra avancées), et le plus haut niveau de sécurité via des protections spécifiques à la plateforme, ce qui est non négociable pour les fonctionnalités tactiques.

Correct. Cette option identifie correctement que pour une application critique, les garanties de performance, de sécurité et d'accès matériel offertes par le développement natif priment sur la vitesse de développement potentielle des frameworks cross-platform.

: Analyse technique

Le choix d'une technologie mobile pour une application critique doit être principalement guidé par les exigences non fonctionnelles les plus contraignantes, qui sont ici la performance, la sécurité et l'accès au matériel de bas niveau.

: Justification

- **Développement Natif (Kotlin/Swift) :** C'est l'approche la plus sûre et la plus performante. Elle garantit un accès direct et sans restriction à toutes les API de la plateforme, y compris les fonctionnalités de sécurité matérielle (Keystore/Secure Enclave), ce qui est indispensable. Les

performances sont optimales car il n'y a aucune couche d'abstraction entre le code de l'application et le système d'exploitation. C'est la référence pour les applications critiques.

- **Flutter :** C'est une excellente option pour le prototypage rapide et pour des applications avec des interfaces utilisateur complexes grâce à son moteur de rendu Skia. Ses performances sont très proches du natif pour la plupart des tâches. Cependant, pour une intégration matérielle très spécifique et non couverte par des plugins existants, il faut développer des "platform channels". Ces ponts peuvent introduire une complexité, une latence et une charge de maintenance qui peuvent être risquées pour une application critique. C'est donc une analyse correcte de le considérer pour le prototypage tout en soulignant les défis pour la production.
- **React Native :** Son architecture basée sur un pont JavaScript est un goulot d'étranglement connu pour les applications exigeantes en performance. De plus, dépendre de bibliothèques tierces pour des fonctionnalités de sécurité critiques est une mauvaise pratique dans ce contexte. L'argument de la réutilisation des compétences JS ne l'emporte pas sur les exigences de sécurité et de performance.
- **Erreur de raisonnement (Distracteur) :** L'affirmation selon laquelle Flutter et React Native partagent la même architecture de pont JavaScript est fausse. Flutter compile en code natif et utilise son propre moteur de rendu, tandis que React Native interprète le JavaScript et communique avec les composants natifs via un pont. C'est une distinction fondamentale qu'un expert doit connaître.

Les réponses correctes sont : Le développement natif (Kotlin/Swift) est la recommandation la plus prudente. Il offre des performances graphiques et de calcul inégalées, un accès direct et complet aux API matérielles (Secure Enclave/Keystore, API de caméra avancées), et le plus haut niveau de sécurité via des protections spécifiques à la plateforme, ce qui est non négociable pour les fonctionnalités tactiques., Flutter pourrait être envisagé pour un prototypage rapide grâce à sa base de code unique et son "hot reload". Cependant, pour la version de production, des défis subsisteraient pour l'intégration de modules matériels très spécifiques non couverts par des plugins existants, nécessitant le développement de "platform channels" complexes et potentiellement coûteux en maintenance.

Note de 0,00 sur 1,00 Incorrect **Question 3**

: Contexte

En tant qu'expert développeur mobile pour la police nationale, vous êtes chargé d'une application critique développée en React Native. Les agents sur le terrain signalent que l'écran affichant l'historique des incidents d'un individu se fige pendant plusieurs secondes, rendant l'application inutilisable. Ce problème ne se produit que sur les anciens modèles de terminaux Android et n'est pas reproductible en environnement de développement. Conformément à votre mission de "Collaborer avec les équipes" (activité 8), vous avez confirmé avec l'équipe backend que leur API GraphQL répond en moins de 200ms. Une première analyse de performance, au titre de votre mission "Optimiser les performances" (activité 6), utilisant le profiler standard de React Native, n'a révélé ni de re-renders excessifs des composants, ni de blocage notable du thread JavaScript pendant les freezes.

: Question

Face à ce diagnostic initial non concluant, quelle est la stratégie d'investigation la plus pertinente et rigoureuse à adopter ensuite ?

- [ ] a. Proposer une réécriture immédiate de l'écran en natif (Kotlin) via un module natif, car c'est la seule garantie d'atteindre les performances requises sur les appareils anciens.
- [ ] b. Insister auprès de l'équipe backend pour qu'elle mette en place une pagination sur l'endpoint GraphQL, car une charge utile, même reçue rapidement, peut être trop lourde à traiter pour un appareil ancien.
- [x] c. Appliquer systématiquement `React.memo` à tous les composants de la liste et envelopper toutes les fonctions de rappel dans `useCallback` pour éliminer toute possibilité de re-render inutile.

  Incorrect. Le scénario stipule explicitement que le profiler n'a PAS montré de re-renders excessifs. Appliquer ces optimisations à l'aveugle sans preuve est une mauvaise pratique qui peut complexifier le code inutilement.
- [ ] d. Utiliser une bibliothèque de virtualisation de liste tierce plus performante que la `FlatList` de base et augmenter la taille de la mémoire allouée à l'application dans le `AndroidManifest.xml`.
- [ ] e. Aucune des réponses proposées.

: Analyse technique

Le scénario est conçu pour piéger les candidats qui appliquent des solutions standards sans réfléchir. La phrase clé est "une première analyse de performance [...] n'a révélé ni de re-renders excessifs [...] ni de blocage notable du thread JavaScript". Cela invalide les hypothèses les plus courantes de problèmes de performance sur React Native. Un expert doit alors conclure que le problème se situe à un niveau plus profond, probablement natif, et que la prochaine étape doit être un diagnostic à ce niveau, et non l'application de solutions prématurées.

: Justification

Puisque les causes communes (re-renders, JS thread) ont été écartées, un expert doit passer à un niveau d'analyse plus profond. La prochaine étape logique serait d'utiliser les outils de profilage natifs d'Android Studio (CPU Profiler, Memory Profiler) pour inspecter le comportement de l'application au niveau du système (thread UI natif, allocation mémoire, appels système). Cela permettrait de détecter des problèmes comme une fuite de mémoire dans un module natif, une contention sur le thread UI natif causée par la `RecyclerView` sous-jacente à la `FlatList`, ou un bug spécifique à la version de l'OS des appareils concernés. Aucune des options A, B, C ou D ne propose cette démarche de diagnostic essentielle, rendant "Aucune des .réponses proposées" la seule option correcte

.La réponse correcte est : Aucune des réponses proposées

Note de -0,25 sur 1,00 Incorrect Question 4

: Contexte

En tant que développeur mobile expert pour la police nationale, vous avez conçu l'architecture de sécurité d'une application Android native critique. Un agent signale la perte de son appareil de service dans une zone sans aucune couverture réseau. L'appareil est donc : actuellement hors-ligne. Les mesures de sécurité que vous aviez implémentées sont les suivantes

- .L'appareil est verrouillé par un code PIN standard de 6 chiffres imposé par la politique de sécurité
- Les données sensibles de l'application (dossiers, informations sur les informateurs) sont stockées localement dans une base de .données SQLite chiffrée avec SQLCipher (AES-256)
- La clé de chiffrement de la base de données SQLCipher est générée et stockée dans l'Android Keystore, configurée pour n'être .accessible qu'après l'authentification de l'utilisateur (c'est-à-dire, après le déverrouillage de l'appareil avec le code PIN)
- Une solution de gestion des appareils mobiles (MDM) est en place, mais la commande d'effacement à distance (remote wipe) est en .attente, car l'appareil est injoignable

: Question

Conformément à votre mission de "Participer à la conception technique et à l'architecture" (activité 3), vous devez évaluer les risques immédiats. Parmi les affirmations suivantes, laquelle décrit avec le plus d'exactitude et de lucidité la posture de sécurité réelle et la ? vulnérabilité la plus critique dans ce scénario précis

a. .Aucune de ces réponses

b. Les données sont raisonnablement sécurisées. L'attaquant doit d'abord contourner le code PIN de 6 chiffres, ce qui est non trivial. Ensuite, il doit extraire la clé du Keystore, qui est protégé par le matériel. La combinaison de ces deux obstacles rend l'accès aux données très .difficile dans un temps limité

c. La vulnérabilité la plus immédiate est la possibilité pour l'attaquant de décompiler l'APK pour analyser le code de l'application. Cela pourrait révéler la logique de chiffrement, les noms des endpoints API et d'autres informations qui pourraient être utilisées pour attaquer le système .backend ou d'autres appareils

d. Le risque le plus critique est qu'un attaquant parvienne à obtenir les privilèges root (ou un jailbreak) sur l'appareil. Cela lui permettrait d'utiliser des outils avancés pour analyser la mémoire et potentiellement extraire la clé du Keystore, même si celui-ci est .soutenu par le matériel

Incorrect. Bien que le root soit une menace sérieuse, ce n'est pas la seule voie d'attaque et elle n'identifie pas la faiblesse fondamentale. La dépendance de la sécurité de la clé à un simple code PIN est une faille architecturale plus directe et plus facile à exploiter que de trouver un exploit de root pour un système d'exploitation potentiellement .à jour

e. La principale vulnérabilité est l'incapacité du MDM à contacter l'appareil. La sécurité des données au repos est robuste, mais elle devient caduque si l'appareil se reconnecte à un réseau non sécurisé avant que l'effacement ne soit effectué, permettant une attaque de type .Man-in-the-Middle

: Analyse technique

La bonne réponse est "Aucune de ces réponses". Chacune des propositions contient une part de vérité mais échoue à identifier la faille architecturale .la plus critique ou minimise le risque de manière incorrecte

: Justification

La vulnérabilité fondamentale et la plus critique dans cette architecture est le **couplage d'un mécanisme de chiffrement fort (SQLCipher avec AES-256) à une clé dont la protection ultime dépend d'un secret faible (un code PIN de 6 chiffres)**.

Un attaquant ayant un accès physique à l'appareil n'a pas besoin de s'attaquer directement au chiffrement AES-256. Il concentrera ses efforts sur le maillon le plus faible : le code PIN. Les techniques pour contourner le verrouillage d'écran Android sont nombreuses et bien documentées (attaques par force brute hors ligne sur des appareils plus anciens, exploitation de vulnérabilités logicielles, techniques de "chip-off" pour lire la mémoire flash directement, etc.).

Une fois le verrouillage d'écran contourné, l'attaquant peut utiliser des outils d'instrumentation (comme Frida) pour se "hooker" à l'application au moment où elle demande la clé au Keystore. Comme l'authentification de l'utilisateur (le PIN) a été satisfaite, le Keystore fournira la clé de déchiffrement à l'application, et donc à l'attaquant.

Les autres options sont incorrectes car :

- **A** est trop optimiste et sous-estime gravement la facilité avec laquelle un code PIN peut être contourné avec un accès physique.
- **B** est passive et ignore le fait qu'un attaquant compétent n'autorisera jamais l'appareil à se reconnecter à un réseau.
- **C** identifie une menace (l'extraction de clé via root) mais ce n'est pas la seule ni la plus simple. Le contournement du lockscreen est souvent plus direct.
- **D** dé-priorise la menace la plus imminente (la fuite de données sensibles immédiate) au profit d'une menace future.

Une architecture véritablement robuste aurait nécessité un facteur d'authentification supplémentaire, indépendant du verrouillage de l'appareil, pour déverrouiller la clé de la base de données (par exemple, une passphrase complexe demandée au lancement de l'application).

La réponse correcte est : Aucune de ces réponses.

Note de -0,33 sur 1,00 Incorrect Question 5

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes responsable de l'architecture d'une application critique existante développée en React Native. Une nouvelle exigence hautement prioritaire est demandée : l'intégration d'une fonctionnalité de diffusion vidéo en direct ("live streaming") depuis la caméra de l'appareil vers le centre de commandement. Cette fonctionnalité est destinée aux situations d'urgence et doit garantir une très faible latence, une haute fiabilité et une consommation de batterie minimale, même sur des appareils plus anciens. L'équipe a des compétences mixtes en React Native, Kotlin et Swift. Votre mission, au titre de vos activités de "conception technique et à l'architecture" (3) et d'"optimisation des performances" (6), est de choisir la meilleure approche d'implémentation.

: Question

Parmi les stratégies d'architecture suivantes, laquelle représente l'approche la plus rigoureuse et la plus professionnelle pour répondre à cette exigence, en équilibrant performance, maintenabilité et rapidité de développement ?

a. Développer l'intégralité de la fonctionnalité en JavaScript en utilisant des bibliothèques React Native open-source populaires (ex: `react-native-camera`, `react-native-webrtc`) pour maximiser la réutilisation du code et accélérer la livraison.

Incorrect. Bien que cette approche soit rapide, elle présente un risque de performance élevé. Le passage constant des trames vidéo brutes à travers le bridge React Native est un goulot d'étranglement connu. Pour une application critique, se fier uniquement à des bibliothèques communautaires sans garantir le contrôle des performances n'est pas une approche rigoureuse.

b. Créer un module natif dédié (en Kotlin/Swift) pour gérer la capture, l'encodage matériel (hardware-accelerated) et la diffusion du flux vidéo, en exposant une interface de contrôle simple (ex: start/stop) au code JavaScript via le bridge React Native.

c. Proposer une réécriture complète de l'application en natif (Kotlin et Swift), en arguant que React Native est fondamentalement incapable de garantir le niveau de performance et de fiabilité requis pour une fonctionnalité aussi critique.

d. Intégrer la fonctionnalité via une `WebView` qui charge une application web optimisée utilisant WebRTC. Cette approche permettrait de mutualiser le développement avec une éventuelle interface web pour le centre de commandement.

e. Aucune des réponses proposées.

: Analyse technique

Cette question évalue la capacité du candidat à prendre une décision architecturale critique dans un contexte de technologie hybride (React Native) avec des exigences de haute performance. Un expert doit comprendre les limites du bridge React Native pour les flux de données à haute fréquence (comme la vidéo) et savoir quand il est nécessaire de déléguer des tâches au code natif pour garantir la performance et la fiabilité, tout en profitant de .la rapidité de développement de React Native pour l'UI

: Justification

La solution correcte (créer un module natif) est la quintessence de l'approche "le bon outil pour le bon travail". Elle utilise le code natif (Kotlin/Swift) pour ce qu'il fait de mieux : les opérations de bas niveau, gourmandes en ressources et nécessitant un accès direct au matériel (capture et encodage vidéo). Elle réserve la couche JavaScript/React Native à la gestion de l'interface utilisateur et de la logique métier, ce qui est son point fort. Les autres options représentent des compromis inacceptables pour une application critique : la solution pure JS risque la performance, la WebView est une anti- .pattern pour ce cas d'usage, et la réécriture complète est une réaction disproportionnée et irréaliste face à la contrainte de temps

La réponse correcte est : Créer un module natif dédié (en Kotlin/Swift) pour gérer la capture, l'encodage matériel (hardware-accelerated) et la .diffusion du flux vidéo, en exposant une interface de contrôle simple (ex: start/stop) au code JavaScript via le bridge React Native

Note de 0,50 sur 1,00 **Partiellement correct** **Question 6**

: Contexte

En tant que développeur mobile expert pour la police nationale, vous devez concevoir une nouvelle fonctionnalité hautement prioritaire : un "bouton d'alerte". Lorsqu'un agent appuie sur ce bouton, une alerte critique (contenant son identifiant, sa position GPS et un horodatage) doit être envoyée au centre de commandement avec la plus haute garantie de livraison possible, même en cas de connectivité réseau intermittente ou nulle. L'équipe travaille en Scrum. L'équipe backend, actuellement en pleine migration d'une API REST vers GraphQL, propose une solution rapide : un unique endpoint `POST /api/v1/alert`. Ils soutiennent que cette approche est simple et rapide à mettre en .œuvre. Le Product Owner insiste pour que la fonctionnalité soit livrée dans le prochain sprint

: Question

Conformément à vos missions ("Analyser les besoins", "Participer à la conception technique et à l'architecture", "Collaborer avec les ? équipes"), quelle(s) action(s) initiale(s) parmi les suivantes représente(nt) la démarche la plus rigoureuse et professionnelle

- [ ] .a Refuser d'implémenter la fonctionnalité tant que l'équipe backend n'a pas fourni une API plus adaptée au temps-réel, comme une .connexion WebSocket ou un broker MQTT, car c'est la seule solution techniquement parfaite
- [ ] .b Esquisser et proposer une architecture client robuste qui utilise un patron de conception de type 'Outbox'. L'action de l'utilisateur ajoute l'alerte à une file d'attente persistante (base de données locale), garantissant une réponse UI immédiate. Un service d'arrière-plan (ex: .WorkManager sur Android) gère ensuite l'envoi fiable de la file d'attente avec une stratégie de relance (ex: backoff exponentiel)
- [ ] .c .Aucune des réponses proposées
- [x] .d Proposer l'organisation d'un atelier de conception rapide (design workshop) avec les équipes backend et produit pour définir formellement les exigences non-fonctionnelles (NFRs) de la fonctionnalité, notamment la latence maximale acceptable, la politique de nouvelles .tentatives, et la garantie de livraison attendue (ex: 'at-least-once delivery')

  Correct. Avant de coder, un expert doit s'assurer que les exigences critiques, surtout non-fonctionnelles, sont comprises et validées par toutes les parties prenantes. Cela correspond directement aux missions .d'analyse des besoins et de collaboration
- [ ] .e Accepter la proposition du backend et implémenter immédiatement l'appel `POST /api/v1/alert` directement dans la logique du bouton, en .ajoutant une simple boucle de 3 tentatives en cas d'échec pour répondre à la demande du Product Owner

: Analyse technique

Ce scénario évalue la capacité d'un développeur expert à aller au-delà de la simple exécution technique pour adopter une posture d'architecte et de .conseiller technique, en particulier pour une fonctionnalité critique où l'échec n'est pas une option

: Justification

- **L'atelier de conception (A)** est une étape fondamentale de la conception logicielle professionnelle. Pour une fonctionnalité vitale, se précipiter dans le code sans avoir défini et validé les exigences non-fonctionnelles (fiabilité, latence, comportement en cas d'échec) est une faute ."professionnelle. Cette action démontre une compréhension des activités "Analyser les besoins" et "Collaborer avec les équipes
- **L'architecture "Outbox" (B)** est la réponse technique standard et robuste au problème de la livraison garantie de messages dans des environnements peu fiables. Elle découple l'action de l'utilisateur de l'envoi réseau, assurant la persistance des données critiques et la résilience du système. Proposer cette architecture démontre une expertise technique et une anticipation des problèmes, conformément à la mission ""Participer à la conception technique et à l'architecture".
- L'implémentation directe avec une boucle de retry (C) est une solution fragile qui ne survit pas à un redémarrage et peut bloquer l'UI.
- Le refus de travailler sans une technologie "parfaite" (D) ignore les contraintes du monde réel et la collaboration, ce qui n'est pas le rôle d'un expert pragmatique.

Les réponses correctes sont : Proposer l'organisation d'un atelier de conception rapide (design workshop) avec les équipes backend et produit pour définir formellement les exigences non-fonctionnelles (NFRs) de la fonctionnalité, notamment la latence maximale acceptable, la politique de nouvelles tentatives, et la garantie de livraison attendue (ex: 'at-least-once delivery')., Esquisser et proposer une architecture client robuste qui utilise un patron de conception de type 'Outbox'. L'action de l'utilisateur ajoute l'alerte à une file d'attente persistante (base de données locale), garantissant une réponse UI immédiate. Un service d'arrière-plan (ex: WorkManager sur Android) gère ensuite l'envoi fiable de la file d'attente avec une stratégie de relance (ex: backoff exponentiel).

Note de 0,00 sur 1,00 **Incorrect** **Question 7**

: Contexte

En tant que développeur mobile expert pour la police nationale, vous travaillez sur une application native (Kotlin/Swift) de carte tactique. Cette application reçoit un flux continu de données via une connexion WebSocket à haute fréquence (~30 messages/seconde). Chaque message est un événement discret (ex: `PositionUpdate`, `NewAlert`, `StatusChange`). L'architecture actuelle agrège tous ces événements dans un unique objet d'état global (par exemple, un `StateFlow<TacticalSituation>`). À chaque nouvel événement, une nouvelle instance de `TacticalSituation` est émise, ce qui déclenche une recomposition complète de l'interface utilisateur de la carte, y compris tous les marqueurs d'officiers, les zones d'alerte et les panneaux d'information. Lors d'opérations à haute intensité, l'interface utilisateur devient saccadée ("janky") et ne répond plus. Le profilage de l'application confirme que le goulot d'étranglement n'est ni le réseau, ni le parsing JSON, mais le temps excessif passé sur le thread principal à redessiner l'UI.

: Question

Conformément à vos missions de "Participer à la conception technique et à l'architecture" (activité 3) et "Optimiser les performances" (activité 6), quelles sont les stratégies les plus efficaces et architecturalement saines pour résoudre ce problème de performance tout en garantissant la réactivité en temps réel et l'intégrité des données ?

a. Refactoriser le `StateFlow<TacticalSituation>` monolithique en plusieurs flux d'état plus petits et indépendants (ex: `StateFlow<List<OfficerPosition>>`, `StateFlow<List<Alert>>`). Les composants UI ne s'abonneront qu'aux flux de données dont ils ont besoin, limitant ainsi les recompositions aux seules parties pertinentes de l'interface.

Correct. Cette approche, connue sous le nom de granularisation de l'état, est fondamentale pour les applications réactives. Elle garantit que les mises à jour de l'état sont localisées, empêchant ainsi des re-rendus en cascade inutiles et coûteux. C'est la solution la plus architecturale au problème.

b. Déplacer la logique de mise à jour de l'objet `TacticalSituation` vers un pool de threads d'arrière-plan dédié (`Dispatchers.Default`) pour éviter de bloquer le thread principal.

Incorrect. Le scénario précise que le goulot d'étranglement n'est pas le calcul ou le parsing, mais le temps de rendu sur le thread principal. Bien que le travail en arrière-plan soit une bonne pratique, cela ne résout pas le problème fondamental qui est que trop de composants UI sont redessinés inutilement sur le thread principal.

c. Appliquer un opérateur de `debounce` ou `throttle` sur le flux WebSocket entrant avec une fenêtre de 250ms pour réduire la charge de traitement.

Incorrect. Bien que cela réduise le nombre de mises à jour, le `debounce` ou `throttle` introduit une latence artificielle et peut écarter des événements critiques intermédiaires. Dans un contexte tactique, un retard de 250ms ou la perte d'une mise à jour de statut est inacceptable. Cette approche traite le symptôme (trop de mises à jour) et non la cause (gestion inefficace des mises à jour).

d. Aucune de ces réponses.

e. Mettre en œuvre un mécanisme de "diffing" au niveau de la couche de présentation (UI). Par exemple, en utilisant `DiffUtil` avec `RecyclerView.Adapter` sur Android, ou des identifiants stables (`key`) dans les listes Jetpack Compose / SwiftUI, pour s'assurer que seuls les éléments de l'interface qui ont réellement changé sont redessinés, même si la liste entière est soumise.

f. Réécrire la couche de rendu de la carte en C++ natif via le NDK (Android) ou Metal (iOS) pour obtenir des performances de dessin maximales.

: Analyse technique

Cette question teste la capacité à diagnostiquer un problème de performance complexe dans une application mobile temps-réel. La cause profonde n'est pas évidente et nécessite une compréhension approfondie des principes de l'UI réactive et de l'architecture de gestion d'état. Les réponses correctes (A et B) représentent une solution architecturale robuste à deux volets : minimiser la portée des mises à jour d'état (A) et s'assurer que la couche UI gère efficacement ces mises à jour (B). Les distracteurs représentent des approches courantes mais erronées : C introduit une latence inacceptable, D interprète mal le goulot d'étranglement, et E est un cas classique de sur-ingénierie qui ne résout pas le défaut architectural.

Les réponses correctes sont : Refactoriser le `StateFlow<TacticalSituation>` monolithique en plusieurs flux d'état plus petits et indépendants (ex: `StateFlow<List<OfficerPosition>>`, `StateFlow<List<Alert>>`). Les composants UI ne s'abonneront qu'aux flux de données dont ils ont besoin, limitant ainsi les recompositions aux seules parties pertinentes de l'interface., Mettre en œuvre un mécanisme de "diffing" au niveau de la couche de présentation (UI). Par exemple, en utilisant `DiffUtil` avec `RecyclerView.Adapter` sur Android, ou des identifiants stables (`key`) dans les listes Jetpack Compose / SwiftUI, pour s'assurer que seuls les éléments de l'interface qui ont réellement changé sont redessinés, même si la liste entière est soumise.

Note de -0,25 sur 1,00 Incorrect Question 8

: Contexte

En tant que développeur mobile principal pour la police nationale, vous êtes chargé de définir la stratégie technique pour une nouvelle application critique. Conformément à la fiche de poste, vous devez analyser les besoins, participer à la conception de l'architecture et effectuer une veille technologique.

: Besoins et contraintes

- **Exigences critiques :** Performances extrêmes pour l'affichage de cartes tactiques en temps réel, intégration profonde avec les API natives de sécurité (Secure Enclave/Keystore), et fiabilité maximale.
- **Contraintes projet :** L'équipe est de taille réduite et doit livrer un prototype fonctionnel sur Android et iOS dans un délai très court pour validation par un groupe pilote.
- **Compétences de l'équipe :** Maîtrise de Kotlin et Swift, bonnes connaissances en JavaScript.

: Question

En analysant ces éléments, quelle stratégie de développement parmi les suivantes représente la décision architecturale la plus rigoureuse et la moins risquée pour démarrer le projet ?

a. Adopter React Native. Cette approche permet de capitaliser sur les compétences JavaScript de l'équipe, de partager un maximum de code et donc de produire très rapidement un prototype pour les deux plateformes, quitte à optimiser les performances plus tard.

Incorrect. Cette stratégie ignore les exigences critiques de performance et de sécurité qui ne peuvent être "optimisées plus tard". Pour une application de police, la fiabilité dès le premier jour est non négociable.

b. Aucune des réponses proposées.

c. Choisir Flutter. Il offre un excellent compromis en compilant en code natif, garantissant des performances identiques au natif pour l'interface utilisateur et les cartes, tout en utilisant une seule base de code pour accélérer le développement.

d. Développer deux applications natives distinctes (Kotlin et Swift). C'est la seule approche qui garantit d'atteindre le niveau de performance et de sécurité requis. Il faut accepter que le prototype prendra deux fois plus de temps à développer.

e. Commencer par développer l'application nativement sur une seule plateforme (ex: Android). Bien que cela permette de valider la technologie, cette approche retarde inévitablement le projet global car aucun travail n'est commencé pour l'autre plateforme.

: Analyse technique

La bonne réponse est "Aucune des réponses proposées". Ce scénario est conçu pour évaluer la capacité du candidat à identifier les risques architecturaux et à ne pas accepter de solution "magique" face à des contraintes contradictoires. Un expert doit reconnaître que chaque option .proposée sacrifie un aspect critique du projet

: Justification

- **Option A (React Native) :** Sacrifie les performances et la sécurité, qui sont des exigences non négociables pour une application de ce type. Le pont JavaScript peut devenir un goulot d'étranglement pour les cartes temps réel, et l'accès aux API de sécurité natives peut être complexe et .moins robuste
- **Option B (Flutter) :** Bien que performant, affirmer que ses performances sont "identiques" au natif dans tous les cas est une simplification excessive, surtout pour des intégrations matérielles profondes. De plus, cela ignore la contrainte de "délai très court" en introduisant une .nouvelle technologie (Dart) que l'équipe ne maîtrise pas, ce qui implique un temps de montée en compétence
- **Option C (Natif) :** C'est la meilleure option technique pour la performance et la sécurité, mais elle ignore complètement la contrainte de livrer .rapidement un prototype sur les deux plateformes avec une équipe réduite. Le coût et le délai seraient doublés
- **Option D (Natif sur une plateforme) :** C'est la stratégie la plus proche d'une bonne réponse, mais l'affirmation qu'elle "retarde inévitablement le projet global" est une conclusion hâtive. Pour un prototype, valider les fonctionnalités les plus risquées (performance des cartes, sécurité) sur une seule plateforme est souvent la stratégie la plus rapide et la plus efficace pour réduire les risques avant de s'engager sur la seconde .plateforme ou une solution cross-platform. La formulation de l'option la rend donc discutable

Un candidat expert aurait dû identifier ces failles et conclure qu'aucune de ces stratégies n'est présentée de manière totalement satisfaisante. Une approche plus mature consisterait à proposer une analyse de risque et une stratégie de prototypage plus fine (ex: un PoC natif sur une plateforme pour .les fonctions critiques, puis une réévaluation)

.La réponse correcte est : Aucune des réponses proposées

Note de 0,00 sur 1,00 Incorrect Question 9

: Contexte

Vous êtes recruté en tant que développeur mobile expert. Le processus actuel de build et de déploiement des applications natives Kotlin/Swift est entièrement manuel, réalisé sur les machines des développeurs. Ceci engendre des builds inconsistants, des régressions fréquentes et des cycles de livraison lents. Votre première mission, en accord avec vos responsabilités de "conception technique et à l'architecture" (activité 3), "réaliser des tests" (activité 5) et "effectuer une veille technologique" (activité 9), est de proposer une .architecture de pipeline CI/CD complète et robuste. Les applications manipulent des données hautement sensibles

: Question

En considérant le contexte et les exigences de la fiche de poste, quelle(s) proposition(s) parmi les suivantes représente(nt) une architecture de pipeline CI/CD rigoureuse, sécurisée et professionnelle ? (Plusieurs réponses peuvent être correctes)

- [ ] .a Une pipeline multi-étapes où : 1) À chaque push sur une branche de fonctionnalité, une pipeline "rapide" exécute le linting et les tests unitaires. 2) À la création d'une pull request, une pipeline plus complète ajoute les scans de sécurité (SAST, scan de dépendances) et construit un artefact non signé. 3) Lors de la fusion dans une branche de `release`, une pipeline finale signe l'artefact, exécute des tests .end-to-end automatisés sur une ferme de terminaux (ex: Firebase Test Lab), et le déploie pour validation finale avant production

- [ ] .b .Aucune des réponses proposées

- [x] .c Une pipeline déclenchée sur chaque push vers la branche `main` qui : 1) Compile le code. 2) Exécute les tests unitaires. 3) Construit l'APK/IPA en utilisant des clés de signature stockées en clair comme variables d'environnement dans les paramètres de l'outil de CI/CD. 4) Déploie le build directement sur un portail de téléchargement interne accessible à tous les agents de .police

  Incorrect. Cette approche est extrêmement risquée. Pousser directement sur `main` est une mauvaise pratique, stocker des secrets en clair est une faille de sécurité critique, et l'absence d'une étape de validation par une équipe QA dédiée avant un .déploiement large est inacceptable

- [ ] .d Une pipeline nocturne (nightly) qui s'exécute à minuit sur la branche `develop`. Elle consiste à : 1) Construire l'application. 2) Exécuter tous les tests (unitaires et UI) en une seule longue étape. 3) En cas de succès, l'artefact signé est automatiquement déposé sur un lecteur .réseau partagé pour que l'équipe QA le télécharge et le teste le lendemain

e. Une pipeline déclenchée sur chaque pull request vers la branche `develop`, incluant les étapes obligatoires suivantes : 1) Scan des dépendances tierces pour les vulnérabilités connues (ex: avec Snyk ou OWASP Dependency-Check). 2) Analyse statique du code (Linting) et Test de Sécurité Statique des Applications (SAST). 3) Compilation et exécution des tests unitaires et d'intégration avec validation de la couverture de code. 4) Construction d'un artefact signé en utilisant des secrets gérés dans un coffre-fort dédié (ex: HashiCorp Vault, AWS/GCP Secret Manager). 5) Déploiement automatisé de l'artefact à un groupe de testeurs dédiés via une plateforme comme Firebase App Distribution.

: Analyse technique

L'objectif est de concevoir une pipeline qui automatise la validation de la qualité, de la performance et de la sécurité à chaque changement de code, tout en utilisant les meilleures pratiques. Les pipelines A et B illustrent des approches modernes et robustes. La pipeline A est une approche "tout-en-un" sur la PR, très sécurisée. La pipeline B est une approche optimisée qui fournit un feedback rapide aux développeurs tout en réservant les tâches lourdes aux étapes d'intégration critiques, ce qui est également une stratégie experte. Les pipelines C et D contiennent des anti-patrons majeurs (mauvaise gestion des branches et des secrets, cycles de feedback lents, processus manuels cachés).

Les réponses correctes sont : Une pipeline déclenchée sur chaque pull request vers la branche `develop`, incluant les étapes obligatoires suivantes : 1) Scan des dépendances tierces pour les vulnérabilités connues (ex: avec Snyk ou OWASP Dependency-Check). 2) Analyse statique du code (Linting) et Test de Sécurité Statique des Applications (SAST). 3) Compilation et exécution des tests unitaires et d'intégration avec validation de la couverture de code. 4) Construction d'un artefact signé en utilisant des secrets gérés dans un coffre-fort dédié (ex: HashiCorp Vault, AWS/GCP Secret Manager). 5) Déploiement automatisé de l'artefact à un groupe de testeurs dédiés via une plateforme comme Firebase App Distribution., Une pipeline multi-étapes où : 1) À chaque push sur une branche de fonctionnalité, une pipeline "rapide" exécute le linting et les tests unitaires. 2) À la création d'une pull request, une pipeline plus complète ajoute les scans de sécurité (SAST, scan de dépendances) et construit un artefact non signé. 3) Lors de la fusion dans une branche de `release`, une pipeline finale signe l'artefact, exécute des tests end-to-end automatisés sur une ferme de terminaux (ex: Firebase Test Lab), et le déploie pour validation finale avant production.

Note de -0,25 sur 1,00 Incorrect Question 10

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes chargé d'optimiser une application Android native (Kotlin) existante. Les agents sur le terrain signalent que l'interface utilisateur (UI) de l'écran de la carte tactique devient fréquemment non réactive (provoquant des "janks" ou des "freezes") pendant plusieurs centaines de millisecondes lors de la réception de mises à jour. L'analyse révèle que l'application reçoit via une API REST une charge utile JSON volumineuse toutes les quelques secondes. Une collaboration avec l'équipe backend (conformément à l'activité 8 de votre fiche de poste) confirme que leurs temps de réponse sont excellents (inférieurs à 100 ms) et qu'ils ne peuvent pas modifier la structure de l'API à court terme. Votre mission, au titre de l'activité 6 ("Corriger les bugs et optimiser les performances"), est de résoudre ce problème de performance critique côté client.

: Question

Parmi les stratégies suivantes, laquelle représente la meilleure approche initiale pour diagnostiquer et résoudre ce problème de manière robuste et professionnelle ?

a. Insister auprès de l'équipe backend pour qu'elle modifie l'API afin de fournir des mises à jour différentielles (uniquement les changements) au lieu de la charge utile complète. C'est la seule façon de résoudre le problème à la racine.

b. Mettre en place une stratégie de "backoff" exponentiel pour les appels API et utiliser un client HTTP plus performant comme OkHttp avec une politique de cache agressive pour réduire la charge réseau.

Incorrect. Cette réponse diagnostique mal le problème. Le contexte indique clairement que les temps de réponse de l'API sont excellents (<100ms). Le goulot d'étranglement n'est donc pas le réseau, mais le traitement (parsing) de la réponse une fois qu'elle est reçue par le client. Ces optimisations réseau, bien que bonnes en général, ne résoudront pas le problème de "freeze" de l'UI.

c. Déplacer le parsing JSON hors du thread principal en utilisant un `AsyncTask`. Dans la méthode `onPostExecute`, mettre à jour l'adaptateur du `RecyclerView` avec la nouvelle liste complète et appeler `notifyDataSetChanged()`.

d. Aucune des réponses proposées.

e. Optimiser le `RecyclerView` lui-même en s'assurant que les ViewHolders sont correctement recyclés, en utilisant `setHasFixedSize(true)` et en implémentant une pagination pour ne pas afficher tous les points de données en même temps.

: Analyse technique

Le problème décrit est un cas classique de blocage du thread principal (Main Thread) sur Android. Le "freeze" de l'UI se produit parce qu'une opération longue et gourmande en CPU (le parsing de la volumineuse charge utile JSON) est effectuée sur le même thread que celui responsable du .dessin de l'interface utilisateur. Tant que cette opération n'est pas terminée, l'UI ne peut pas être mise à jour, ce qui la rend non réactive

: Justification

Aucune des options proposées ne constitue une solution complète et experte. Une solution idéale et moderne impliquerait une combinaison de : plusieurs techniques avancées

1. **Exécution en arrière-plan :** Utiliser des Coroutines Kotlin (avec `Dispatchers.Default` pour le parsing intensif en CPU) pour déplacer l'ensemble du processus de parsing JSON hors du thread principal. C'est une approche plus moderne et plus sûre que les `AsyncTask` .dépréciés
2. **Mises à jour efficaces de l'UI :** Une fois les données parsées et transformées en une liste d'objets, il est crucial de ne pas simplement appeler `notifyDataSetChanged()` sur l'adaptateur du `RecyclerView`. La bonne pratique consiste à utiliser la classe `DiffUtil` pour calculer les .différences entre l'ancienne et la nouvelle liste. Ce calcul doit également être effectué sur un thread d'arrière-plan
3. **Application des changements :** Le résultat de `DiffUtil` (`DiffResult`) est ensuite utilisé pour mettre à jour l'adaptateur sur le thread principal (`withContext(Dispatchers.Main)`). Cela garantit que le `RecyclerView` n'effectue que les animations et les mises à jour minimales nécessaires ."(ajouts, suppressions, modifications), ce qui est extrêmement performant et évite le "jank

Les options proposées sont des distracteurs car elles sont soit incomplètes (B), soit elles diagnostiquent mal le problème (A, D), soit elles ignorent les .contraintes du projet (C). Un expert doit identifier la combinaison correcte de threading moderne et d'algorithmes de mise à jour d'UI efficaces

.La réponse correcte est : Aucune des réponses proposées

Note de -0,25 sur 1,00 Incorrect Question 11

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes responsable de la pipeline CI/CD d'une application Android native (Kotlin) critique. Récemment, la pipeline a commencé à échouer de manière intermittente, mais uniquement sur la branche `release`, jamais .sur les branches de fonctionnalités

L'échec se produit systématiquement lors de l'étape `instrumented_tests`, qui exécute une suite de tests d'interface utilisateur sur une ferme d'appareils Firebase Test Lab. Le message d'erreur est un générique "`Instrumentation run failed due to 'Process crashed'`", et les journaux de crash (crash logs) sont inconsistants, pointant parfois vers des erreurs de mémoire, parfois vers des conditions de .concurrence (race conditions)

La dernière modification majeure fusionnée dans `release` était une mise à niveau d'une bibliothèque tierce de chargement d'images et une refactorisation mineure d'un service en arrière-plan qui utilise cette bibliothèque. Le développeur responsable de ce changement est en .congé

: Question

Conformément à vos missions de "Participer à la conception technique et à l'architecture" (activité 3), "Réaliser des tests" (5) et "Corriger les bugs et optimiser les performances" (6), quelle serait la démarche de dépannage la plus rigoureuse et systématique pour identifier la ? cause racine de cette défaillance

.a Se concentrer sur une investigation locale. Cloner la branche `release`, écrire des tests unitaires spécifiques pour la nouvelle logique du service en arrière-plan et utiliser le profileur de mémoire d'Android Studio pour détecter d'éventuelles fuites liées à la .nouvelle bibliothèque d'images

Bien que l'investigation locale soit une bonne pratique, se concentrer uniquement sur les tests unitaires est une erreur. Le bug se produit lors des tests instrumentés, suggérant un problème contextuel (cycle de vie, interaction UI) que les .tests unitaires ne captureront probablement pas

.b Revertir immédiatement le dernier commit fusionné sur la branche `release` à l'aide de `git revert`. Relancer ensuite la pipeline pour .débloquer les livraisons, et planifier l'analyse du bug pour un sprint ultérieur

.c Prioriser la reproductibilité du bug dans l'environnement CI/CD. Modifier la configuration de la pipeline pour activer l'enregistrement vidéo des tests, augmenter la verbosité des logs, et forcer l'exécution des tests sur un modèle d'appareil spécifique à faible mémoire dans .Firebase Test Lab

d. Aucune des réponses proposées.

e. Appliquer l'activité 8 ("Collaborer avec les équipes"). Organiser immédiatement une réunion avec les responsables produit et QA pour réévaluer la priorité du bug. Parallèlement, documenter le problème et l'envoyer à l'équipe pour une analyse collective dès le retour du développeur concerné.

Analyse technique :

La bonne réponse est "Aucune des réponses proposées". Ce scénario est conçu pour évaluer la capacité du candidat à adopter une approche de dépannage holistique et méthodique, plutôt que de se précipiter sur une solution partielle. Chaque option proposée représente une facette d'une bonne stratégie, mais aucune n'est complète en soi.

Justification :

- L'option A (revert) est une solution de contournement, pas une solution de fond. Elle sacrifie la compréhension du problème pour une disponibilité immédiate de la pipeline, ce qui est une mauvaise pratique pour un bug critique et intermittent qui risque de réapparaître.
- L'option B (tests unitaires locaux) est une bonne intention, mais mal ciblée. Le problème se manifeste lors des tests instrumentés (UI), qui s'exécutent dans un contexte d'application complet. Les tests unitaires, bien qu'utiles, ne pourront probablement pas reproduire un bug lié au cycle de vie, à la mémoire de l'appareil ou à des interactions complexes.
- L'option C (focus sur la CI) est pertinente pour améliorer la reproductibilité, mais elle ignore la boucle de feedback beaucoup plus rapide du débogage local. Un expert devrait d'abord tenter de reproduire le problème sur sa machine avant de lancer de multiples exécutions de pipeline, qui sont lentes et coûteuses.
- L'option D (escalade) est prématurée. Avant d'impliquer d'autres équipes, un développeur expert doit épuiser les pistes d'investigation technique relevant de sa responsabilité. C'est une forme de déresponsabilisation qui ne démontre pas une expertise technique.

Une approche experte et systématique combinerait plusieurs de ces étapes dans le bon ordre :

1. **Analyse approfondie :** Examiner en détail tous les journaux et artefacts disponibles de Firebase Test Lab (journaux Logcat, traces de pile, vidéos si disponibles) pour trouver un schéma commun.
2. **Reproduction locale :** Tenter de reproduire le bug en exécutant la même suite de tests instrumentés localement, d'abord sur un émulateur, puis sur un appareil physique (idéalement un modèle à faibles ressources).
3. **Amélioration de la CI :** Si la reproduction locale échoue, modifier la pipeline (comme dans l'option C) pour forcer l'exécution sur un type d'appareil spécifique où l'échec est plus fréquent et activer un maximum de diagnostics.
4. **Isolation et Correction :** Une fois le bug reproduit de manière fiable, utiliser les outils de profilage (mémoire, CPU) pour identifier la cause racine (ex: une fuite de mémoire dans la nouvelle bibliothèque sur les appareils à faible RAM) et développer un correctif.

Aucune des options ne décrivant cette démarche complète, la seule réponse correcte est de les rejeter toutes.

La réponse correcte est : Aucune des réponses proposées.

Note de 0,00 sur 1,00 Incorrect Question 12

Contexte :

En tant que développeur mobile expert pour la police nationale, vous êtes responsable de la maintenance d'une application Android native (Java/Kotlin) critique. Une fonctionnalité clé, le suivi de la localisation en temps réel des agents, est implémentée via un `Service` classique, démarré avec `startService()`. Les agents équipés de terminaux récents (Android 10 et plus) rapportent que le suivi cesse de fonctionner de manière fiable après que l'application a été mise en arrière-plan pendant un certain temps, ou lorsque l'appareil est inactif. Sur les appareils plus anciens (Android 7), le problème n'apparaît pas. L'analyse du manifeste de l'application révèle que la `targetSdkVersion` est actuellement fixée à 25. Votre mission, au titre de vos activités "Corriger les bugs et optimiser les performances" (6) et "Assurer la maintenance et les mises à jour" (7), est de diagnostiquer la cause profonde et de proposer une solution architecturale robuste.

Question :

Parmi les stratégies de correction suivantes, laquelle représente la solution la plus complète, moderne et fiable pour garantir un suivi de localisation continu et robuste sur toutes les versions d'Android, en tenant compte du caractère critique de l'application ?

a. La cause est la restriction des services en arrière-plan depuis Android 8. La solution est de transformer le service en un service de premier plan (Foreground Service) en affichant une notification persistante. Cela garantit que le système ne le tuera pas.

Incorrect. Bien que l'utilisation d'un Foreground Service soit nécessaire, cette solution est incomplète. Elle omet la nécessité de demander la permission `ACCESS_BACKGROUND_LOCATION`, qui est obligatoire depuis Android 10 pour accéder à la localisation en arrière-plan.

b. Aucune des réponses proposées.

c. L'utilisation d'un `Service` est obsolète. La solution moderne consiste à le remplacer par un `PeriodicWorkRequest` de `WorkManager`, planifié pour s'exécuter toutes les minutes. `WorkManager` est conçu pour garantir l'exécution des tâches en arrière-plan de manière fiable et optimisée pour la batterie.

d. La solution consiste à mettre à jour la `targetSdkVersion`, à implémenter un `Foreground Service` et à demander la permission `ACCESS_BACKGROUND_LOCATION`. Pour optimiser la batterie, le service doit utiliser l'API `FusedLocationProviderClient` avec la priorité `PRIORITY_BALANCED_POWER_ACCURACY`.

e. Le problème est causé par le mode Doze et la gestion agressive de la batterie. La solution consiste à acquérir un `WakeLock` partiel dans le service pour maintenir le CPU actif et à demander aux utilisateurs de désactiver manuellement les optimisations de batterie pour l'application.

> **Analyse technique :**
>
> La bonne réponse est "Aucune des réponses proposées" car chaque option présente une faille critique ou une mauvaise pratique inacceptable pour une application de cette nature. Un candidat expert doit être capable d'identifier les lacunes dans chaque proposition, même celles qui semblent correctes en surface.
>
> **Justification :**
>
> - **Option A :** L'utilisation d'un `WakeLock` permanent est une anti-pattern qui draine massivement la batterie. De plus, se reposer sur une action manuelle de l'utilisateur ("désactiver les optimisations de batterie") est totalement inacceptable pour une application critique dont la fiabilité doit être garantie par conception.
> - **Option B :** Bien que la transformation en `Foreground Service` soit une étape nécessaire, cette réponse est dangereusement incomplète. Depuis Android 10 (API 29), l'accès à la localisation en arrière-plan nécessite l'obtention de la permission `ACCESS_BACKGROUND_LOCATION`, qui a un processus d'approbation spécifique par l'utilisateur. Omettre ce point crucial rend la solution non fonctionnelle sur les appareils récents.
> - **Option C :** C'est une mauvaise application de la technologie. `WorkManager` est conçu pour des tâches différables et non-exactes (par exemple, synchroniser des photos une fois par heure). Il n'offre aucune garantie d'exécution en temps réel, ce qui est l'exigence fondamentale pour une fonctionnalité de suivi de sécurité.
> - **Option D :** Cette option est la plus plausible mais contient une faille subtile et critique. Pour une application de sécurité où la position exacte et à jour d'un agent peut être une question de vie ou de mort, utiliser `PRIORITY_BALANCED_POWER_ACCURACY` est un compromis inacceptable. La seule priorité viable est `PRIORITY_HIGH_ACCURACY`. Un expert doit savoir adapter les choix techniques aux exigences métier critiques, même si cela a un coût (ici, la batterie).
>
> La solution experte complète impliquerait la mise à jour de la `targetSdkVersion`, l'implémentation d'un `Foreground Service` avec une notification claire, la demande des permissions `ACCESS_FINE_LOCATION` et `ACCESS_BACKGROUND_LOCATION`, et l'utilisation du `FusedLocationProviderClient` avec `PRIORITY_HIGH_ACCURACY`, tout en gérant intelligemment le cycle de vie du service pour qu'il ne s'exécute que lorsque cela est strictement nécessaire.

La réponse correcte est : Aucune des réponses proposées.

Note de 0,50 sur 1,00 Partiellement correct Question 13

Contexte :

Vous êtes développeur mobile pour la police nationale. Une application Android native (Kotlin), utilisée par les agents en patrouille, devient de plus en plus lente après plusieurs heures d'utilisation et finit par planter avec une erreur `OutOfMemoryError`. Le problème est particulièrement notable après la consultation de nombreux dossiers contenant des images. L'analyse du code révèle le patron de conception suivant pour l'accès aux données :

```
// Simplified Singleton pattern for data access
class CaseFileRepository(private val context: Context) {
companion object {
@Volatile
private var INSTANCE: CaseFileRepository? = null

fun getInstance(context: Context): CaseFileRepository {
return INSTANCE ?: synchronized(this) {
val instance = CaseFileRepository(context)
INSTANCE = instance
instance
}
}
}
// ... methods to fetch data from a local DB or network
}

// Usage within a Fragment
class CaseDetailsFragment : Fragment() {
private lateinit var repository: CaseFileRepository

override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
super.onViewCreated(view, savedInstanceState)

// An Activity context is passed here
repository = CaseFileRepository.getInstance(requireActivity())

// ... code to load case details and images using the repository
}
}
```

: Question

En vous basant sur le code fourni et le contexte décrit, quelles sont les analyses et actions correctives les plus pertinentes pour résoudre de manière robuste le problème de fuite mémoire ? (Plusieurs réponses peuvent être correctes)

- [ ] a. Le singleton `CaseFileRepository` est initialisé avec un contexte d'`Activity`, ce qui crée une fuite mémoire car le singleton a une durée de vie plus longue que l'`Activity`. La correction principale consiste à passer `context.applicationContext` à la méthode `getInstance`.

- [ ] b. Le bloc `synchronized` dans la méthode `getInstance` est une source de contention des threads qui dégrade les performances jusqu'au crash. Il faut le remplacer par une initialisation via un framework d'injection de dépendances (ex: Hilt) pour garantir la performance et la sécurité.

- [ ] c. Le problème principal est le chargement inefficace des images. La solution consiste à intégrer une bibliothèque tierce comme Glide ou Picasso pour gérer le décodage, le redimensionnement et la mise en cache des images, sans modifier l'architecture du `Repository`.

- [x] d. Le `CaseDetailsFragment` doit s'assurer d'annuler l'enregistrement de tout listener ou callback auprès du `repository` dans sa méthode de cycle de vie `onDestroyView()` pour éviter que le fragment lui-même ne fuite entre les navigations.

  Correct. C'est une mesure de sécurité complémentaire essentielle. Même si le repository utilise le contexte d'application, il peut toujours causer une fuite du Fragment s'il détient une référence directe à celui-ci via un listener non annulé.

- [ ] e. Aucune de ces réponses.

: Analyse technique

Le problème principal est une fuite de mémoire classique sur Android causée par la rétention d'un contexte d'`Activity` par un objet à longue durée de vie (le singleton `CaseFileRepository`).

: Justification

- **Réponse A (Correcte) :** Le singleton `INSTANCE` est statique et vivra tant que le processus de l'application est en vie. En lui passant `requireActivity()`, on lui donne une référence à l'`Activity`. Même lorsque l'utilisateur quitte cette `Activity` (et son `Fragment`), le singleton

conserve cette référence, empêchant le Garbage Collector de libérer la mémoire occupée par l'`Activity`, ses vues, et toutes les ressources associées. L'utilisation de `context.applicationContext` résout ce problème car le contexte d'application a la même durée de vie que l'application elle-même, il n'y a donc pas de fuite.

- **Réponse B (Correcte) :** C'est une bonne pratique de défense en profondeur. Si le `Fragment` s'abonne à des mises à jour du `Repository` (par exemple, via des listeners, des callbacks ou des flux de données comme LiveData/Flow), il doit impérativement annuler cet abonnement dans `onDestroyView()`. Sinon, le `Repository` (qui est un singleton) pourrait conserver une référence au `Fragment`, l'empêchant d'être collecté même après que l'utilisateur a navigué ailleurs, créant une autre fuite de mémoire.
- **Réponse C (Incorrecte) :** Bien que la gestion inefficace des images soit une cause fréquente d'`OutOfMemoryError`, le code fourni pointe vers une erreur architecturale plus fondamentale. Un expert doit d'abord identifier la fuite de contexte, qui est une certitude à la lecture du code. Résoudre le problème des images sans corriger la fuite de contexte ne ferait que retarder l'inévitable crash.
- **Réponse D (Incorrecte) :** Le bloc `synchronized` est une implémentation standard (bien que légèrement datée) du double-checked locking pour la sécurité des threads. Il peut introduire une légère contention lors de la toute première initialisation, mais il n'est pas la cause d'une fuite mémoire ou d'un crash systématique. Suggérer un framework de DI est une bonne pratique générale, mais cela ne diagnostique pas le problème spécifique présenté.

Les réponses correctes sont : Le singleton `CaseFileRepository` est initialisé avec un contexte d'`Activity`, ce qui crée une fuite mémoire car le singleton a une durée de vie plus longue que l'`Activity`. La correction principale consiste à passer `context.applicationContext` à la méthode `getInstance`., Le `CaseDetailsFragment` doit s'assurer d'annuler l'enregistrement de tout listener ou callback auprès du `repository` dans sa méthode de cycle de vie `onDestroyView()` pour éviter que le fragment lui-même ne fuite entre les navigations.

Note de 0,00 sur 1,00 Incorrect **Question 14**

Contexte :

En tant que développeur mobile expert pour la police nationale, vous êtes chargé d'optimiser une application Android native (Kotlin) de patrouille. Les agents rapportent que l'écran de scan de plaques d'immatriculation, qui utilise la caméra en continu, devient extrêment lent après une utilisation prolongée (plus de 50 scans) et finit par crasher avec une erreur `OutOfMemoryError`. L'analyse de performance avec l'Android Studio Profiler montre une augmentation constante de la mémoire dans la catégorie "Native", tandis que la mémoire "Java" reste stable. L'architecture de traitement d'image est la suivante : un `ImageAnalysis` de CameraX fournit des `ImageProxy`. Pour chaque image, un appel est fait via JNI à une librairie C++ legacy qui effectue la reconnaissance optique de caractères (OCR).

Question :

Conformément à votre mission de "Corriger les bugs et optimiser les performances" (activité 6), quelle stratégie de correction initiale est la plus rigoureuse et la plus susceptible de résoudre la cause racine du problème ?

a. La librairie C++ est obsolète et probablement la source de la fuite. Il faut la réécrire entièrement en Kotlin en utilisant une bibliothèque d'OCR moderne comme ML Kit pour garantir une gestion de la mémoire contrôlée par la JVM.

b. Le garbage collector de Java n'est pas assez agressif pour ce cas d'usage. Il faut insérer des appels manuels à `System.gc()` après le traitement de chaque lot de 10 images pour forcer la libération de la mémoire Java.

c. Aucune des réponses proposées.

d. Le traitement d'image est trop lourd pour le thread principal. Il faut encapsuler l'appel JNI dans une coroutine lancée sur `Dispatchers.IO` pour décharger le thread principal et améliorer la réactivité de l'UI.

e. L'application ne libère pas les `ImageProxy` assez rapidement, ce qui sature le pool de mémoire de CameraX. La solution est d'envelopper l'appel JNI et le traitement subséquent dans un bloc `try...finally` pour garantir que `image.close()` est appelé systématiquement pour chaque frame.

C'est une excellente pratique, mais elle ne résout que les fuites de mémoire côté Java. Le profilage indique une fuite "Native". Si la librairie C++ copie les données de l'image dans sa propre mémoire sans la libérer, `image.close()` n'aura aucun effet sur cette fuite native.

Analyse technique :

Le diagnostic clé est que la mémoire "Native" augmente alors que la mémoire "Java" est stable. Cela pointe de manière quasi certaine vers une fuite de mémoire dans le code natif (C++) ou dans la couche de communication JNI, et non dans le code managé (Kotlin/Java). Une `OutOfMemoryError` peut être déclenchée lorsque le tas natif consomme toute la mémoire disponible pour le processus, même si le tas Java est sain.

: Justification

- **A** est une solution extrême et prématurée. La réécriture est coûteuse et le problème pourrait se situer dans la manière dont la librairie est appelée (la "glue" JNI) plutôt que dans la librairie elle-même.
- **B** est une bonne pratique pour la réactivité de l'UI, mais ne résout en rien la fuite de mémoire. Déplacer le travail sur un autre thread ne fera que masquer le problème plus longtemps, mais le crash surviendra toujours.
- **C** est une excellente pratique pour la gestion des `ImageProxy` et prévient les fuites de mémoire *côté Java/CameraX*. Cependant, si la librairie C++ copie les données du buffer de l'image dans sa propre mémoire allouée (avec `malloc` ou `new`) sans jamais la libérer (avec `free` ou `delete`), appeler `image.close()` ne libérera pas cette mémoire native copiée. C'est un distracteur très plausible car c'est la première chose à vérifier, mais le profilage indique un problème natif plus profond.
- **D** est une mauvaise pratique. Appeler `System.gc()` manuellement est déconseillé, imprévisible et inefficace. De plus, cela n'a aucun effet sur la mémoire allouée sur le tas natif, qui est la source du problème.

Par conséquent, aucune des solutions proposées ne s'attaque directement et avec certitude à la cause racine identifiée par le profilage (fuite de mémoire native). La démarche correcte serait d'utiliser des outils de débogage de mémoire native (comme `HWASan`, `MTE` ou Valgrind sur des émulateurs compatibles) pour localiser précisément la fuite dans le code C++ ou JNI.

La réponse correcte est : Aucune des réponses proposées.

Note de -0,25 sur 1,00 **Incorrect** **Question 15**

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes chargé d'une application critique développée en React Native. Les agents sur le terrain signalent que l'écran de la carte tactique, qui affiche les positions en temps réel via une souscription GraphQL, devient extrêmement lent et saccadé ("janky") après environ 30 minutes d'utilisation continue, rendant l'application inutilisable. Le problème est particulièrement prononcé sur les terminaux Android d'ancienne génération.

Votre analyse initiale, au titre de vos missions "Optimiser les performances" (activité 6) et "Collaborer avec les équipes" (activité 8), révèle les points suivants :

- Le profileur de performance de React Native (côté JavaScript) ne montre aucune surcharge du thread JS, ni de re-rendus excessifs des composants React.
- L'équipe backend confirme que le serveur GraphQL envoie les mises à jour de manière efficace et que la latence réseau est faible.
- Le problème semble donc provenir de la couche native (Android) de l'application, potentiellement lié à la manière dont la bibliothèque de cartographie tierce gère les mises à jour fréquentes des marqueurs.

: Question

Face à ce diagnostic initial, quelle stratégie d'investigation parmi les suivantes représente l'approche la plus rigoureuse et la plus professionnelle pour identifier la cause racine du problème ?

a. Refactoriser la communication entre JavaScript et la vue native de la carte pour envoyer les mises à jour de position en lots (batching) plutôt qu'individuellement. L'hypothèse est que la surcharge du pont (bridge) React Native est la cause du ralentissement du thread UI.

b. Créer un "fork" de la bibliothèque de cartographie, identifier le code natif Android responsable de la mise à jour des marqueurs, et y ajouter manuellement des appels pour forcer le nettoyage des ressources graphiques (ex: `Bitmap.recycle()`). Cela permet une correction rapide et ciblée du problème suspecté de fuite de mémoire.

c. Aucune des réponses proposées.

d. Appliquer des techniques de mémoïsation (`React.memo`, `useCallback`) sur tous les composants liés à la carte et à ses marqueurs. L'objectif est de réduire le nombre de re-rendus du Virtual DOM côté JavaScript, ce qui allègera la charge de travail transmise au thread UI natif.

Incorrect. Le contexte précise que le profileur JS n'a pas révélé de re-rendus excessifs. Cette approche se concentre sur le mauvais symptôme et ignore l'indice pointant vers un problème natif.

e. Mettre en place une stratégie de "throttling" ou de "debouncing" côté client pour limiter la fréquence des mises à jour de l'état provenant de la souscription GraphQL. En réduisant le nombre de mises à jour par seconde, on diminue mécaniquement la charge sur le moteur de rendu.

: Analyse technique

Le scénario est conçu pour tester la capacité d'un développeur expert à dépasser les outils de débogage standards d'un framework cross-platform (React Native) et à investiguer des problèmes de performance au niveau natif. Les indices clés sont que le problème est spécifique aux appareils Android anciens et que le thread JS n'est pas le goulot d'étranglement. Cela pointe fortement vers un problème de gestion de la mémoire ou des .ressources graphiques dans la couche native

: Justification

.Aucune des options proposées ne constitue une démarche de diagnostic complète et rigoureuse

- Les options A et B se concentrent sur des optimisations au niveau JavaScript, alors que le contexte indique clairement que le problème n'est pas .là. Un expert doit savoir reconnaître quand il faut changer de niveau d'analyse
- L'option C propose une hypothèse plausible (le pont React Native) mais suggère de refactoriser avant d'avoir une preuve concrète. La bonne .démarche est de profiler d'abord pour confirmer l'hypothèse
- L'option D saute directement à une solution (forker la librairie) qui est une très mauvaise pratique de maintenance, sans avoir validé la cause .exacte du problème ni cherché de solutions alternatives (mise à jour, configuration, issue déjà connue)

: La démarche experte correcte, non listée, serait

1. Utiliser les outils de profilage natifs d'Android Studio (Memory Profiler, CPU Profiler, Perfetto) sur un build de production de l'application pour .analyser l'utilisation de la mémoire native (en particulier graphique/GPU) et l'activité du thread UI natif pendant les saccades
2. Isoler le problème en créant un projet de test minimal qui utilise uniquement la bibliothèque de carte et reproduit le scénario de mise à jour .fréquente des marqueurs
3. Consulter le repository public de la bibliothèque (ex: GitHub Issues) pour voir si ce problème de performance ou de fuite de mémoire est déjà .connu et si une solution de contournement ou une version corrigée existe
4. Basé sur les résultats, soit mettre à jour la bibliothèque, soit appliquer une configuration recommandée, soit soumettre un rapport de bug .détaillé aux mainteneurs

.La réponse correcte est : Aucune des réponses proposées

Noté sur 1,00 Non répondue Question 16

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes chargé d'optimiser une application Android legacy écrite en Java. L'application affiche une liste de rapports d'incident via un `ListView` et un `BaseAdapter` personnalisé. Les utilisateurs signalent de graves problèmes de performance (saccades/lags) lors du défilement de listes longues. Votre analyse (Activité 6 : "Optimiser les performances") révèle que la méthode `getView()` de l'adaptateur effectue, pour chaque élément de la liste, un appel réseau bloquant vers un service tiers afin de récupérer une image de carte statique de la localisation de l'incident. L'équipe backend, avec qui vous devez "Collaborer" (Activité .8), affirme ne pas pouvoir modifier son API principale (`/api/reports`) pour y inclure l'URL de l'image de la carte à court terme

: Question

Conformément à vos missions "Participer à la conception technique et à l'architecture" (Activité 3) et "Rédiger un code propre, maintenable" (Activité 4), quelle(s) stratégie(s) parmi les suivantes représente(nt) l'approche la plus robuste et professionnelle pour ? résoudre définitivement ce problème de performance

- a. .Aucune des réponses proposées
- b. Conserver le `ListView` mais modifier la méthode `getView()` pour utiliser une `AsyncTask` pour chaque appel réseau, afin de télécharger .l'image en arrière-plan
- c. Mettre en place une couche de cache persistante (ex: base de données Room) spécifiquement pour les images des cartes, en utilisant les coordonnées géographiques comme clé. L'adaptateur vérifierait d'abord la présence de l'image dans le cache avant de lancer un .téléchargement asynchrone
- d. Migrer de `ListView` à `RecyclerView` et intégrer une bibliothèque de chargement d'images (ex: Glide, Picasso). Dans la méthode `onBindViewHolder` du nouvel adaptateur, utiliser la bibliothèque pour charger de manière asynchrone l'URL de l'image de la carte dans .l'`ImageView`

e. Mettre en place une pagination côté client en modifiant l'adaptateur pour ne charger initialement que les 20 premiers rapports, avec un bouton "Charger plus" pour récupérer les suivants.

: Analyse technique

Le problème fondamental est l'exécution d'opérations d'entrées/sorties (I/O) réseau, qui sont longues et imprévisibles, sur le thread principal (UI thread) lors du rendu de l'interface utilisateur (dans la méthode `getView()`). Cela bloque le thread principal, l'empêchant de dessiner les images suivantes à temps, ce qui provoque les saccades ("jank"). Une solution robuste doit impérativement déplacer le travail réseau hors du thread principal et optimiser le rendu de la liste.

: Justification

- **(A)** **Migration vers RecyclerView et utilisation d'une bibliothèque de chargement d'images :** C'est la solution standard et la meilleure pratique sur Android. `RecyclerView` est beaucoup plus performant que `ListView` car il impose le pattern ViewHolder et gère intelligemment le recyclage des vues. Les bibliothèques comme Glide ou Picasso sont conçues pour gérer de manière optimale le chargement asynchrone d'images, incluant la gestion des threads, le cache en mémoire et sur disque, et la gestion du cycle de vie pour éviter les fuites de mémoire.
- **(B)** **Mise en place d'une couche de cache persistante :** C'est une excellente amélioration architecturale. En stockant les images téléchargées dans une base de données (comme Room), on réduit considérablement le nombre d'appels réseau futurs pour les mêmes localisations. Cela améliore non seulement les performances de défilement lors de visites ultérieures de la liste, mais rend également l'application plus résiliente aux problèmes de réseau. Cette approche est complémentaire à la solution (A).
- **(C)** **Utilisation d'AsyncTask :** C'est une approche obsolète et dangereuse. Bien qu'elle déplace le travail hors du thread principal, `AsyncTask` est déprécié et connu pour causer des fuites de mémoire et des problèmes de cycle de vie. De plus, sans une gestion manuelle complexe, il peut conduire à l'affichage d'images incorrectes dans les vues recyclées.
- **(D)** **Pagination côté client :** Cette technique optimise la quantité de données chargées initialement, mais ne résout pas le problème de fond. Le défilement des 20 éléments chargés sera toujours saccadé car la cause racine (appel réseau bloquant dans `getView()`) n'est pas traitée.

Les réponses correctes sont : Migrer de `ListView` à `RecyclerView` et intégrer une bibliothèque de chargement d'images (ex: Glide, Picasso). Dans la méthode `onBindViewHolder` du nouvel adaptateur, utiliser la bibliothèque pour charger de manière asynchrone l'URL de l'image de la carte dans l'`ImageView`., Mettre en place une couche de cache persistante (ex: base de données Room) spécifiquement pour les images des cartes, en utilisant les coordonnées géographiques comme clé. L'adaptateur vérifierait d'abord la présence de l'image dans le cache avant de lancer un téléchargement asynchrone.

Noté sur 1,00 Non répondue Question 17

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes chargé de résoudre un problème critique sur une application Android native (Kotlin). Les agents rapportent que la fonctionnalité de téléversement de preuves (photos, vidéos) échoue de manière intermittente avec une erreur générique "Erreur réseau". Après investigation, vous déterminez que le problème ne se produit que lorsque les appareils sont connectés à certains réseaux Wi-Fi publics (ex: palais de justice, hôpitaux) mais ne se produit jamais sur les réseaux 4G/5G ou sur le Wi-Fi du bureau. L'équipe backend, avec qui vous devez "Collaborer" (activité 8), a analysé ses logs et confirme qu'elle ne reçoit absolument aucune requête entrante provenant des appareils concernés au moment des échecs. L'application utilise une pile réseau moderne et standard (Retrofit/OkHttp). Votre mission est de "Corriger les bugs et optimiser les performances" (activité 6) en identifiant la cause racine.

: Question

Parmi les stratégies de diagnostic initiales suivantes, laquelle représente l'approche la plus rigoureuse et systématique pour identifier la cause racine du problème ?

a. Il s'agit d'un problème classique de résolution DNS sur les réseaux publics. La solution consiste à implémenter un client DNS-over-HTTPS (DoH) dans la configuration d'OkHttp pour contourner le résolveur DNS du réseau local et garantir que le domaine de l'API est toujours correctement résolu.

b. L'équipe backend se trompe. Le problème vient certainement d'un pare-feu ou d'un équilibreur de charge mal configuré qui bloque les requêtes provenant des plages d'adresses IP des Wi-Fi publics. Il faut immédiatement escalader le problème à l'équipe infrastructure en leur fournissant les adresses IP des réseaux défaillants.

c. Le problème est probablement une condition de concurrence (race condition) dans le code de téléversement qui ne se manifeste que sous la latence plus élevée des réseaux Wi-Fi. La stratégie doit être de refactoriser immédiatement la logique en utilisant des Coroutines Kotlin avec une concurrence structurée pour garantir la robustesse.

d. Aucune des réponses proposées.

e. Le problème est probablement un bug dans une version ancienne de la bibliothèque OkHttp ou Retrofit. La première étape la plus efficace consiste à mettre à jour toutes les dépendances réseau vers leurs dernières versions stables, à nettoyer le projet et à retester sur le terrain.

: Analyse technique

Le scénario décrit un problème de connectivité qui est spécifique à l'environnement réseau (certains Wi-Fi publics) et où les requêtes n'atteignent jamais le serveur. Les options A, B, C et D proposent toutes des solutions ou des pistes basées sur des hypothèses non vérifiées.

A. Suggère une race condition, mais cela n'explique pas pourquoi le problème est limité à des réseaux spécifiques.
B. Accuse le backend sans preuve. La première étape d'un bon diagnostic est de valider ce qui se passe côté client.
C. Mettre à jour les bibliothèques est une tentative à l'aveugle qui ne repose sur aucune analyse.
D. Suggère un problème DNS et une solution (DoH), mais sans avoir d'abord vérifié si la résolution DNS est bien la cause.

La démarche d'un expert consiste à collecter des données avant de formuler une hypothèse. La seule façon de savoir ce qui se passe réellement au niveau du réseau sur l'appareil est d'intercepter et d'analyser le trafic. Un outil de proxy (comme Charles, Fiddler, ou mitmproxy) installé pour inspecter le trafic de l'application révélerait très probablement la cause racine : une redirection HTTP vers un portail captif (Captive Portal). Ces portails, courants sur les Wi-Fi publics, interceptent les requêtes HTTPS initiales pour forcer l'utilisateur à accepter des conditions d'utilisation. La pile réseau de l'application, s'attendant à une réponse TLS de l'API, reçoit une réponse inattendue (souvent une redirection HTTP), ce qui provoque une erreur de handshake TLS ou une autre erreur réseau générique, expliquant ainsi l'échec avant même que la requête n'atteigne le serveur de destination.

: Justification

Aucune des options proposées ne suggère la première étape logique et indispensable du dépannage réseau avancé : l'inspection du trafic réel sortant de l'appareil. Toutes les autres options sont des suppositions qui pourraient conduire à une perte de temps et de ressources. Par conséquent, la seule réponse correcte est "Aucune des réponses proposées", car la bonne démarche n'est pas listée.

La réponse correcte est : Aucune des réponses proposées.

Noté sur 1,00 Non répondue Question 18

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes chargé de fiabiliser la pipeline d'Intégration Continue et de Déploiement Continu (CI/CD) d'une application Android native (Kotlin) critique. Actuellement, la pipeline est instable et lente, ce qui a érodé la confiance de l'équipe qui privilégie de plus en plus les builds manuels. La pipeline se compose des étapes suivantes : `lint`, `unit_tests`, `build_apk`, et `security_scan`. Vous constatez les problèmes suivants :

- L'étape `unit_tests` échoue de manière intermittente à cause de tests "flaky" (non déterministes) liés à des opérations asynchrones mal gérées.
- L'étape `build_apk` prend plus de 20 minutes, ralentissant considérablement les cycles de feedback.
- L'étape `security_scan`, utilisant un outil SAST, bloque la pipeline en signalant de nombreux faux positifs, ce qui nécessite une validation manuelle fastidieuse.

: Question

Conformément à vos missions ("Participer à la conception technique et à l'architecture", "Réaliser des tests", "Optimiser les performances"), quelles actions initiales parmi les suivantes constituent la stratégie la plus rigoureuse et professionnelle pour stabiliser et optimiser cette pipeline ?

a. Aucune de ces réponses.

b. Prioriser la correction des tests "flaky" en refactorisant le code pour utiliser l'injection de dépendances et des dispatchers de test dédiés (ex: `TestCoroutineDispatcher`) afin de contrôler l'exécution des opérations asynchrones. En parallèle, lancer une analyse du build avec l'option `--scan` de Gradle pour identifier les tâches les plus longues et activer le cache de build Gradle pour accélérer les builds successifs.

c. Configurer l'outil SAST pour établir une "baseline" des vulnérabilités existantes et ne faire échouer la pipeline que si de nouvelles vulnérabilités de haute sévérité sont introduites. Organiser une session de formation avec l'équipe pour présenter le plan d'amélioration, expliquer les causes des problèmes et restaurer la confiance dans le processus automatisé.

d. Pour restaurer immédiatement la confiance, désactiver temporairement les étapes `unit_tests` et `security_scan`. Augmenter les timeouts et configurer 3 tentatives automatiques pour l'ensemble de la pipeline afin de masquer les erreurs intermittentes et d'obtenir un build "vert" plus souvent.

e. Proposer une migration immédiate de la pipeline vers un service CI/CD plus moderne (ex: GitLab CI, GitHub Actions), car l'outil actuel est probablement la cause principale de la lenteur et de l'instabilité. Mettre en pause les corrections sur la pipeline existante pour se concentrer sur la preuve de concept de la nouvelle solution.

: Analyse technique

Cette question évalue la capacité d'un développeur expert à diagnostiquer et à résoudre des problèmes complexes dans un environnement CI/CD, en appliquant une approche systémique plutôt que de se contenter de solutions de contournement. Un expert doit équilibrer la vitesse, la qualité et la sécurité, tout en gérant les aspects humains (confiance de l'équipe).

: Justification

- **Réponse A (Correcte) :** S'attaquer aux tests "flaky" est la priorité absolue car une pipeline non déterministe est inutile. L'utilisation de l'injection de dépendances et de dispatchers de test pour les coroutines (comme `TestCoroutineDispatcher` ou `UnconfinedTestDispatcher`) est la solution standard et robuste pour maîtriser l'asynchronisme en test. L'analyse des builds via `--scan` et l'activation du cache Gradle sont les premières étapes fondamentales pour optimiser les temps de compilation avant d'envisager des solutions plus coûteuses.
- **Réponse B (Correcte) :** Une approche pragmatique de la sécurité est cruciale. Bloquer la pipeline pour des problèmes existants est contre-productif. La création d'une "baseline" de sécurité permet de se concentrer sur les nouvelles vulnérabilités sans arrêter le développement. L'aspect humain est également essentiel ; restaurer la confiance de l'équipe par la communication et la formation est aussi important que les correctifs techniques pour garantir l'adoption de la pipeline.
- **Réponse C (Incorrecte) :** C'est une solution radicale et prématurée. Changer d'outil CI/CD sans avoir analysé et tenté de résoudre les problèmes de fond (tests, configuration de build) est une erreur d'architecture classique. Le problème réside probablement dans la configuration du projet et non dans l'outil lui-même.
- **Réponse D (Incorrecte) :** C'est la pire approche. Désactiver les tests et la sécurité sacrifie la qualité et la sécurité, ce qui est inacceptable pour une application critique. Augmenter les timeouts et les tentatives ne fait que masquer les problèmes de "flakiness" sans les résoudre, rendant la pipeline encore moins fiable à long terme.

Les réponses correctes sont : Prioriser la correction des tests "flaky" en refactorisant le code pour utiliser l'injection de dépendances et des dispatchers de test dédiés (ex: `TestCoroutineDispatcher`) afin de contrôler l'exécution des opérations asynchrones. En parallèle, lancer une analyse du build avec l'option `--scan` de Gradle pour identifier les tâches les plus longues et activer le cache de build Gradle pour accélérer les builds successifs., Configurer l'outil SAST pour établir une "baseline" des vulnérabilités existantes et ne faire échouer la pipeline que si de nouvelles vulnérabilités de haute sévérité sont introduites. Organiser une session de formation avec l'équipe pour présenter le plan d'amélioration, expliquer les causes des problèmes et restaurer la confiance dans le processus automatisé.

Noté sur 1,00 Non répondue Question 19

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes chargé de mettre en place une pipeline CI/CD (activités 3, 5, 9) pour une nouvelle application native (Kotlin/Swift) critique. L'application doit être déployée sur plusieurs environnements distincts : `dev` (développement), `staging` (qualification), et `prod` (production). Chaque environnement requiert une configuration spécifique, incluant des points d'accès API (endpoints), des clés d'API, et d'autres secrets. Une exigence fondamentale est que les secrets ne doivent jamais être stockés en clair dans le dépôt Git et que la gestion de ces configurations doit être entièrement automatisée, sécurisée et auditable.

: Question

Parmi les stratégies suivantes, laquelle représente l'approche la plus robuste, sécurisée et maintenable pour gérer ces configurations et secrets spécifiques à chaque environnement au sein de la pipeline CI/CD ?

a. Créer un dépôt Git privé distinct contenant les fichiers de configuration pour chaque environnement. Chiffrer les fichiers contenant des secrets avec un outil comme `git-crypt`. La pipeline CI/CD est configurée avec une clé de déchiffrement pour cloner ce dépôt et accéder aux secrets au moment du build.

b. Aucune des réponses proposées.

c. Stocker toutes les configurations et secrets en tant que variables d'environnement sécurisées directement dans les paramètres du système CI/CD (ex: GitHub Actions Secrets, GitLab CI/CD Variables). Un script dans la pipeline lit ces variables au moment du build et génère dynamiquement les fichiers de configuration nécessaires.

d. Utiliser des branches Git distinctes pour chaque environnement (ex: `dev`, `staging`, `master`). Chaque branche contient les fichiers de configuration et les clés d'API spécifiques à son environnement. La pipeline CI/CD est configurée pour construire à partir de la branche appropriée.

e. Implémenter des "Build Flavors" (sur Android) et des "Schemes" (sur iOS). Chaque saveur/schéma lit sa configuration depuis un fichier de propriétés (ex: `dev.properties`, `prod.properties`) versionné dans Git. Les secrets sont placés dans un fichier non versionné (ex: `.secrets.properties`) qui doit être créé manuellement sur chaque machine de développeur et sur le serveur CI/CD.

: Analyse technique

Cette question évalue la capacité à concevoir une architecture CI/CD sécurisée et évolutive, une compétence clé pour un développeur expert (activités 3, 4, 9). Elle teste la connaissance des bonnes pratiques et des anti-patterns en matière de gestion des secrets, un sujet critique pour une application de police. Chacune des options proposées représente une approche commune, mais comporte des failles de sécurité ou de maintenabilité significatives qui les rendent sous-optimales pour un contexte critique.

: Justification

Aucune des solutions proposées n'est idéale. La meilleure pratique moderne consiste à utiliser un gestionnaire de secrets externe dédié (comme HashiCorp Vault, AWS Secrets Manager, ou Google Secret Manager). Dans cette approche, la pipeline CI/CD s'authentifie auprès du service de secrets (souvent via un rôle ou une identité à courte durée de vie) pour récupérer les secrets nécessaires juste au moment de la construction (just-in-time). Cela offre une gestion centralisée, des pistes d'audit robustes, une rotation facile des secrets et un contrôle d'accès granulaire, surpassant en sécurité et en scalabilité toutes les options listées.

- **Option A (Branches Git) :** C'est un anti-pattern. Elle complique la gestion du code (merge hell) et ne résout pas le problème du stockage des secrets dans Git.
- **Option B (Fichiers locaux) :** Crée une charge de configuration manuelle sur les agents CI/CD, n'est pas scalable et pose un risque de sécurité si les agents ne sont pas correctement isolés et nettoyés.
- **Option C (Variables d'environnement CI/CD) :** C'est une approche courante mais qui devient rapidement ingérable, difficile à auditer et à maintenir synchronisée à grande échelle. La gestion des secrets via une interface graphique est un anti-pattern.
- **Option D (Dépôt Git chiffré) :** C'est une bonne approche, mais elle déplace le problème vers la gestion sécurisée de la clé de déchiffrement, qui se retrouve souvent elle-même en tant que variable d'environnement (voir Option C), créant un point de défaillance unique.

La réponse correcte est : Aucune des réponses proposées.

Noté sur 1,00 Non répondue Question 20

: Contexte

En tant que développeur mobile expert pour la police nationale, vous travaillez sur une application iOS critique possédant une base de code ancienne et volumineuse en Objective-C. Votre équipe opère en Scrum avec des sprints de deux semaines. À mi-sprint, une vulnérabilité critique (score CVSS 9.1) est découverte dans un composant central, non documenté et non testé : un singleton Objective-C nommé `AuthManager`, responsable de la gestion des jetons d'authentification. Cette vulnérabilité permet le détournement de session (token hijacking) dans certaines conditions réseau. Le responsable de la sécurité exige une correction immédiate et définitive. Le Product Owner s'inquiète de l'impact sur les fonctionnalités promises pour la fin du sprint. L'équipe backend est disponible pour collaborer, mais tout changement d'API de leur côté nécessiterait un processus de validation distinct.

: Question

Conformément à vos missions de "Corriger les bugs" (activité 6), "Participer à la conception technique et à l'architecture" (3) et "Collaborer avec les équipes" (8), quelle approche parmi les suivantes représente la démarche la plus rigoureuse et professionnelle pour gérer cette situation, en équilibrant les impératifs de sécurité immédiats, les engagements du projet et la santé du code à long terme ?

a. Arrêter immédiatement tout le travail du sprint. Se consacrer à la réécriture complète du `AuthManager` en Swift en utilisant une approche TDD (Test-Driven Development). Informer le Product Owner que l'objectif du sprint est abandonné et que les nouvelles fonctionnalités seront retardées.

b. Aucune des réponses proposées.

c. Demander à l'équipe backend d'implémenter immédiatement des mesures de mitigation côté serveur (ex: validation plus stricte de l'adresse IP ou de l'agent utilisateur des jetons) pour réduire la surface d'attaque. Pendant ce temps, continuer le développement des fonctionnalités du sprint et planifier la refonte côté mobile pour le sprint suivant.

d. Organiser une réunion d'urgence avec le Product Owner, un représentant de la sécurité et le responsable technique du backend. Présenter une analyse détaillée de l'impact de la vulnérabilité et proposer deux options : un hotfix temporaire avec une refonte planifiée, ou une refonte complète immédiate qui interrompt le sprint. Laisser le Product Owner prendre la décision finale en fonction du risque métier.

e. Proposer un "hotfix" rapide dans le code Objective-C existant pour neutraliser la vulnérabilité, même si la solution est imparfaite. Créer un ticket de dette technique pour une refonte propre dans un sprint futur. Cela permet à l'équipe de poursuivre les fonctionnalités du sprint en cours avec une interruption minimale.

: Analyse technique

Cette question évalue la capacité du candidat à gérer une crise technique complexe dans un environnement Agile, en faisant appel non seulement à ses compétences techniques mais aussi à son jugement professionnel et sa capacité à collaborer. La bonne réponse est "Aucune des réponses proposées" car chaque option présente une faille significative dans la démarche, représentant des archétypes de mauvaises décisions ou de décisions incomplètes en situation de crise.

: Justification

: Une approche d'expert consisterait à combiner les points forts de plusieurs options tout en évitant leurs faiblesses. La démarche idéale serait

1. Convoquer immédiatement la réunion d'urgence (comme dans l'option C) avec toutes les parties prenantes.
2. Présenter une analyse d'impact claire et formuler une recommandation technique forte : recommander l'arrêt du sprint pour traiter la vulnérabilité, en expliquant pourquoi un "quick fix" (option B) est inacceptable pour une faille de sécurité critique dans un code non testé, et pourquoi les mesures palliatives côté backend (option D) sont insuffisantes.
3. Collaborer avec le Product Owner pour redéfinir le but du sprint autour de la résolution de cette vulnérabilité, en transformant la crise en travail planifié.
4. Proposer une stratégie de correction technique robuste qui n'est pas nécessairement une réécriture complète immédiate (comme dans l'option A). Une approche plus professionnelle serait d'isoler le composant legacy, d'écrire des tests de caractérisation pour comprendre son comportement, puis de le remplacer de manière incrémentale et sécurisée par un nouveau module Swift, en utilisant potentiellement un patron de conception comme Façade ou Adapter pour minimiser les risques de régression.

Aucune des options proposées ne décrit cette démarche complète et nuancée, qui démontre à la fois la rigueur technique, la compréhension des processus Agile et la maturité professionnelle.

La réponse correcte est : Aucune des réponses proposées.

Noté sur 1,00 Non répondue Question 21

: Contexte

En tant que développeur mobile expert pour la police nationale, vous travaillez sur une application Android native (Kotlin) utilisée par les agents pour soumettre des rapports d'incident. L'équipe opère en Scrum avec des sprints de 2 semaines. Un bug critique est découvert en production : dans les zones à faible connectivité, si le réseau est perdu pendant l'envoi d'un rapport, celui-ci est parfois définitivement perdu sans que l'agent en soit clairement notifié. L'analyse révèle que l'application effectue un appel réseau direct depuis la couche UI, sans mécanisme de persistance locale ou de nouvelle tentative. Votre mission, conformément à votre fiche de poste, est de "Corriger les bugs" (activité 6), "Participer à la conception technique et à l'architecture" (3) et "Collaborer avec les équipes" (8).

: Question

Face à cette situation critique au milieu d'un sprint, quelle(s) approche(s) parmi les suivantes démontre(nt) le plus haut niveau de maturité professionnelle et stratégique, en équilibrant les impératifs opérationnels, la qualité technique et les principes Agile ?

a. S'engager à livrer la solution architecturale complète et robuste (avec persistance, file d'attente et service en arrière-plan) avant la fin du sprint en cours, en travaillant en dehors des heures de bureau si nécessaire. L'objectif est de résoudre le problème de fond le plus rapidement possible sans perturber le planning des sprints.

b. Escalader immédiatement la criticité du bug au Product Owner et au Scrum Master. Argumenter que sa gravité compromet l'objectif du sprint et justifie une "annulation de sprint" (Sprint Cancellation). L'objectif serait de démarrer immédiatement un nouveau sprint, plus court, entièrement dédié à l'implémentation de la solution architecturale correcte et complète pour garantir la fiabilité des données.

c. Aucune des réponses proposées.

d. Proposer au Product Owner un plan en deux phases : 1) Développer immédiatement un "hotfix" minimaliste qui persiste le rapport localement (ex: dans SharedPreferences ou un fichier) uniquement si l'envoi initial échoue, avec un indicateur visuel clair pour l'agent. 2) Créer et prioriser des tâches techniques détaillées pour le prochain sprint afin de refactoriser la couche réseau avec une architecture robuste et résiliente (ex: Repository Pattern, base de données Room, WorkManager pour la synchronisation en arrière-plan).

e. Informer le Product Owner qu'un "hotfix" introduirait une dette technique inacceptable et refuser de l'implémenter. Insister sur le fait que la seule approche professionnelle est de planifier correctement la refonte architecturale pour le prochain sprint et de continuer le travail actuellement planifié pour ne pas compromettre les engagements du sprint en cours.

: Analyse technique

Cette question évalue la capacité d'un développeur expert à naviguer dans une situation complexe qui mêle un problème technique critique, des contraintes de processus (Scrum) et des impératifs opérationnels (sécurité des données). Il ne s'agit pas seulement de trouver une solution technique, mais de proposer une stratégie de résolution mature.

: Justification

- **Réponse A :** Cette approche est excellente car elle est pragmatique. Elle traite l'urgence immédiate avec un "hotfix" qui sécurise les données (la priorité absolue) tout en reconnaissant et en planifiant le remboursement de la dette technique. La communication claire avec le Product Owner démontre une collaboration et une prise de responsabilité conformes au rôle d'un expert.
- **Réponse B :** C'est une réponse d'expert qui démontre une compréhension approfondie des mécanismes de Scrum. L'annulation d'un sprint est une mesure drastique mais justifiée lorsque son objectif n'est plus pertinent ou qu'un événement externe de haute priorité (comme ce bug critique) survient. Proposer cette option montre que le candidat comprend la gravité de la situation et connaît les outils de gestion de projet pour y faire face correctement.
- **Réponse C (Incorrecte) :** C'est l'anti-pattern du "développeur héros". Bien que l'intention soit bonne, se précipiter pour implémenter une solution complexe sous pression augmente considérablement le risque d'introduire de nouveaux bugs, ne respecte pas le processus Agile (estimation, planification) et crée une culture de travail insoutenable.
- **Réponse D (Incorrecte) :** C'est l'anti-pattern de "l'architecte puriste". En refusant toute solution à court terme, le candidat ignore le risque opérationnel critique (perte de données de rapport d'incident), ce qui est inacceptable dans le contexte de la police. La qualité du code est importante, mais pas au détriment de la mission principale.

Les réponses correctes sont : Proposer au Product Owner un plan en deux phases : 1) Développer immédiatement un "hotfix" minimaliste qui persiste le rapport localement (ex: dans SharedPreferences ou un fichier) uniquement si l'envoi initial échoue, avec un indicateur visuel clair pour l'agent. 2) Créer et prioriser des tâches techniques détaillées pour le prochain sprint afin de refactoriser la couche réseau avec une architecture robuste et résiliente (ex: Repository Pattern, base de données Room, WorkManager pour la synchronisation en arrière-plan)., Escalader immédiatement la criticité du bug au Product Owner et au Scrum Master. Argumenter que sa gravité compromet l'objectif du sprint et justifie une "annulation de sprint" (Sprint Cancellation). L'objectif serait de démarrer immédiatement un nouveau sprint, plus court, entièrement dédié à l'implémentation de la solution architecturale correcte et complète pour garantir la fiabilité des données.

Noté sur 1,00 Non répondue Question 22

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes responsable de la pipeline CI/CD (activités 3, 5, 9) des applications natives (Kotlin/Swift). Un nouvel outil d'analyse de sécurité (SAST/DAST) intégré à la pipeline bloque la livraison d'une nouvelle version en signalant une vulnérabilité critique de type "Remote Code Execution" (RCE) avec un score CVSS de 9.8. La vulnérabilité se situe dans une bibliothèque open-source tierce, très populaire et profondément intégrée, utilisée pour le rendu d'images. L'exploitation est possible via le traitement d'un fichier image malveillant (ex: un PNG ou un WebP spécialement conçu). L'application utilise cette bibliothèque pour afficher des photos de preuves et des images de profil provenant d'une API REST interne. L'équipe de sécurité exige une correction immédiate, tandis que l'équipe produit s'inquiète de l'impact sur les délais du sprint en cours.

: Question

Conformément à vos missions ("Corriger les bugs et optimiser les performances" - activité 6, "Participer à la conception technique et à l'architecture" - 3, et "Collaborer avec les équipes" - 8), quelle(s) action(s) parmi les suivantes représente(nt) la démarche la plus rigoureuse et professionnelle pour gérer cette situation critique ?

a. Mettre immédiatement à jour la bibliothèque vers la dernière version corrigée, relancer la pipeline, et si elle passe, fusionner la modification et notifier les équipes.

b. Désactiver temporairement l'étape de scan de sécurité pour débloquer la pipeline et livrer les fonctionnalités du sprint. Créer un ticket dans le backlog pour traiter la vulnérabilité dans un sprint futur, en arguant que le risque est faible car les images proviennent d'une source "interne "de confiance.

c. Aucune des réponses proposées.

d. Créer une branche dédiée. Y ajouter un test d'intégration qui reproduit la vulnérabilité en utilisant un fichier image PoC (Proof of Concept), et vérifier qu'il échoue. Mettre ensuite à jour la bibliothèque et s'assurer que ce même test passe désormais. Enfin, lancer une campagne de tests de non-régression complète avant de demander la fusion.

e. En parallèle des actions de correction sur le client, collaborer immédiatement avec l'équipe backend pour mettre en place une sanitisation des images côté serveur (par exemple, en re-compressant systématiquement toutes les images uploadées). Ceci ajoute une couche de défense en profondeur et protège l'ensemble du parc d'appareils, y compris ceux qui n'ont pas encore été mis à jour.

: Analyse technique

Cette question évalue la capacité d'un développeur expert à réagir à une alerte de sécurité critique de manière structurée, en équilibrant l'urgence, la rigueur technique et la vision architecturale à long terme. Il ne s'agit pas seulement de "patcher" le problème, mais de le faire de manière vérifiable, sûre et stratégique.

: Justification

- **Réponse B :** C'est la réponse tactique la plus rigoureuse. Elle suit une approche de développement piloté par les tests (TDD) pour la sécurité : 1) Prouver l'existence de la vulnérabilité dans le contexte de l'application (créer un test qui échoue), 2) Appliquer le correctif (mise à jour de la bibliothèque), 3) Prouver que le correctif fonctionne (le test passe), et 4) S'assurer que le correctif n'a pas introduit de régressions. C'est la marque d'un ingénieur senior qui ne se contente pas d'appliquer un patch à l'aveugle.
- **Réponse D :** C'est la réponse stratégique et architecturale par excellence. Elle démontre une compréhension de la "défense en profondeur". Plutôt que de compter uniquement sur la sécurité du client (qui peut toujours être faillible), cette approche ajoute une couche de protection au niveau du serveur. La sanitisation côté serveur (en reconvertissant toutes les images dans un format sûr) neutralise cette classe de vulnérabilité à la source, protégeant tous les clients, y compris les anciennes versions de l'application qui n'auraient pas encore été mises à jour. C'est une excellente démonstration de collaboration inter-équipes (activité 8) et de conception architecturale (activité 3).
- **Réponse A (Incorrecte) :** C'est une approche précipitée et risquée. Mettre à jour une bibliothèque majeure sans évaluer son impact peut introduire des bugs subtils ou des régressions fonctionnelles. De plus, cela ne vérifie pas que la mise à jour corrige réellement la vulnérabilité dans le contexte spécifique de l'application.
- **Réponse C (Incorrecte) :** C'est la pire approche. Ignorer une vulnérabilité RCE critique est une faute professionnelle grave, surtout dans un contexte policier. L'argument de la "source de confiance" est un sophisme de sécurité bien connu ; les systèmes internes peuvent être compromis et devenir des vecteurs d'attaque.

Les réponses correctes sont : Créer une branche dédiée. Y ajouter un test d'intégration qui reproduit la vulnérabilité en utilisant un fichier image PoC (Proof of Concept), et vérifier qu'il échoue. Mettre ensuite à jour la bibliothèque et s'assurer que ce même test passe désormais. Enfin, lancer une campagne de tests de non-régression complète avant de demander la fusion., En parallèle des actions de correction sur le client, collaborer immédiatement avec l'équipe backend pour mettre en place une sanitisation des images côté serveur (par exemple, en re-compressant systématiquement toutes les images uploadées). Ceci ajoute une couche de défense en profondeur et protège l'ensemble du parc d'appareils, y compris ceux qui n'ont pas encore été mis à jour.

Question 23 Non répondue Noté sur 1,00

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes responsable de la maintenance d'une application Android native (Kotlin) critique. La pipeline CI/CD, récemment mise à jour avec un nouvel outil d'analyse de sécurité statique (SAST), bloque désormais les builds de la branche principale en signalant une vulnérabilité de haute sévérité : "Improper Neutralization of Special Elements used in an HTTP Header ('HTTP Header Injection')". L'outil pointe vers un intercepteur OkHttp personnalisé, `HeaderInterceptor`, qui ajoute un en-

tête `X-Officer-Info` à toutes les requêtes sortantes. La valeur de cet en-tête est construite en utilisant le nom complet de l'agent, qui peut potentiellement contenir des séquences de caractères malveillants (ex: `\r\n`). L'équipe de sécurité exige une correction immédiate, arguant que même si le backend est protégé, la politique est de ne tolérer aucune vulnérabilité de ce type côté client.

: Voici le code incriminé

```
// Dans la configuration du client OkHttp
class HeaderInterceptor(private val userSession: UserSession) : Interceptor {
override fun intercept(chain: Interceptor.Chain): Response {
val originalRequest = chain.request()
// La valeur de getOfficerName() peut être "John Doe\r\nInjected-Header: a-value"
val officerName = userSession.getOfficerName()

val newRequest = originalRequest.newBuilder()
.header("X-Officer-Info", "name=${officerName}")
.build()

return chain.proceed(newRequest)
}
}
```

: Question

Conformément à vos missions de "Rédiger un code propre, maintenable" (activité 4), "Participer à la conception technique et à l'architecture" (3) et "Corriger les bugs" (6), quelle(s) action(s) parmi les suivantes représente(nt) l'approche la plus robuste et professionnelle pour corriger définitivement cette vulnérabilité ?

- a. Implémenter une fonction de validation stricte pour la chaîne `officerName` qui applique une liste blanche (whitelist) de caractères autorisés (ex: alphanumériques, espaces, tirets) et une limite de longueur. Rejeter la requête ou utiliser une valeur par défaut sécurisée si la validation échoue.
- b. Assainir (sanitize) la chaîne `officerName` dans l'intercepteur en supprimant systématiquement tous les caractères de retour à la ligne (`\n`) et de retour chariot (`\r`) avant de construire l'en-tête.
- c. Aucune de ces réponses.
- d. Refactoriser l'intercepteur pour qu'il envoie un identifiant stable et non modifiable de l'agent (ex: `officerId`) au lieu de son nom. Collaborer avec l'équipe backend (activité 8) pour qu'elle résolve cet identifiant côté serveur, déplaçant ainsi la gestion des données non fiables dans un environnement contrôlé.
- e. Encoder la valeur de `officerName` en Base64 avant de l'insérer dans l'en-tête. Cela masquera les caractères spéciaux et empêchera l'injection.

> **: Analyse technique**
>
> La vulnérabilité de type "HTTP Header Injection" ou "CRLF Injection" se produit lorsque des données contrôlées par l'utilisateur sont insérées dans un en-tête HTTP sans validation ou assainissement approprié. Les caractères `\r\n` (CRLF) peuvent être utilisés pour terminer l'en-tête actuel et en injecter de nouveaux, permettant à un attaquant de contourner les sécurités (ex: filtres WAF), d'empoisonner le cache, ou de mener des attaques de type "session fixation".
>
> **: Justification**
>
> - L'option B est la meilleure solution architecturale. Elle déplace la responsabilité de la gestion des données potentiellement non fiables (le nom de l'utilisateur) vers le backend, qui est un environnement contrôlé. En ne transmettant qu'un identifiant stable et non modifiable, on élimine la surface d'attaque côté client.
> - L'option C est une solution de défense en profondeur très robuste côté client. L'utilisation d'une liste blanche (whitelist) de caractères autorisés est une pratique de sécurité standard et bien plus efficace que le "blacklisting" (liste noire) de caractères dangereux connus. Elle empêche non seulement l'injection CRLF mais aussi d'autres formes d'injection potentielles.
> - L'option A est une solution faible. Elle corrige le symptôme (injection CRLF) mais pas la cause (confiance aveugle dans les données utilisateur). Un attaquant pourrait trouver d'autres séquences de caractères pour exploiter la vulnérabilité sur certains serveurs.
> - L'option D est une erreur de conception courante. L'encodage (comme Base64) n'est pas une mesure de sécurité. Il s'agit d'une transformation de données réversible. Le backend décodera la chaîne, et les caractères malveillants `\r\n` seront toujours présents, rendant l'attaque possible.

Les réponses correctes sont : Refactoriser l'intercepteur pour qu'il envoie un identifiant stable et non modifiable de l'agent (ex: `officerId`) au lieu de son nom. Collaborer avec l'équipe backend (activité 8) pour qu'elle résolve cet identifiant côté serveur, déplaçant ainsi la gestion des données non fiables dans un environnement contrôlé., Implémenter une fonction de validation stricte pour la chaîne `officerName` qui applique une liste blanche (whitelist) de caractères autorisés (ex: alphanumériques, espaces, tirets) et une limite de longueur. Rejeter la requête ou utiliser une valeur .par défaut sécurisée si la validation échoue

Noté sur 1,00 Non répondue Question 24

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes chargé d'ajouter une nouvelle fonctionnalité critique à une application iOS existante. L'application possède une base de code volumineuse et ancienne en Objective-C avec une architecture monolithique. La nouvelle fonctionnalité est un scanner de documents sécurisé qui doit être écrit en Swift. Elle doit capturer des images, effectuer une reconnaissance optique de caractères (OCR) sur l'appareil, chiffrer les résultats à l'aide du Secure Enclave, puis les téléverser. L'intégration doit être sécurisée, maintenable et testable, conformément à vos activités de "conception technique et à .l'architecture" (3), "rédiger un code propre et maintenable" (4) et "réaliser des tests" (5)

: Question

? Parmi les stratégies d'intégration suivantes, laquelle représente l'approche la plus rigoureuse et professionnelle

- .a Créer un singleton Swift `SecureScannerManager` exposé à Objective-C via un bridging header. Ce singleton gérera l'accès à la caméra, le .chiffrement et les appels réseau. Les `ViewControllers` Objective-C existants appelleront directement `[SecureScannerManager shared]`
- .b .Aucune des réponses proposées
- .c Proposer une refonte complète des `ViewControllers` Objective-C concernés et de leur logique associée en Swift, en utilisant une .architecture moderne comme MVVM, avant d'implémenter la nouvelle fonctionnalité de scan
- .d Définir un protocole Swift `SecureScanner` et son implémentation. Pour passer la clé de chiffrement du Secure Enclave à la couche réseau .Objective-C existante, convertir la clé en objet `Data` et la passer en tant que `NSData` à travers le pont de communication
- .e Intégrer la fonctionnalité en utilisant un module Flutter autonome, présenté via un `FlutterViewController`. Cela permet d'utiliser un .écosystème moderne pour la nouvelle fonctionnalité tout en minimisant les modifications dans le code Objective-C existant

: Analyse technique

Cette question évalue la capacité d'un développeur expert à faire des compromis architecturaux pragmatiques lors de la modernisation d'une base de code legacy. Elle teste la connaissance des patrons de conception (Singleton, Adapter), des principes de couplage faible, de la testabilité, et surtout, une compréhension approfondie des implications de sécurité lors de l'interaction entre Swift et Objective-C, notamment la gestion des données sensibles comme les clés de chiffrement. Un expert doit être capable de rejeter des solutions apparemment simples ou idéales qui cachent des .défauts critiques

: Justification

: Aucune des options proposées n'est adéquate

- .L'option A réplique un anti-pattern (Singleton "God Object") qui nuit à la testabilité et à la maintenabilité
- .L'option B, bien qu'idéale techniquement, ignore les contraintes de temps du projet ("big bang refactoring")
- .L'option C est un cas d'over-engineering, introduisant une complexité et une dépendance inutiles (Flutter)
- L'option D introduit une vulnérabilité de sécurité critique en exposant une clé de chiffrement brute à travers le pont Objective-C, où elle pourrait .être interceptée en mémoire

La solution experte, non listée, serait d'utiliser le **patron de conception Adapter (ou Façade)**. On créerait un module Swift entièrement nouveau, testable et isolé. Une seule classe Swift, marquée `@objc`, servirait d'adaptateur. Cette classe serait la seule interface visible par le code Objective-C. Elle orchestrerait les composants Swift internes (gestionnaire de caméra, moteur OCR, service de chiffrement) et ne retournerait au code Objective-C que le résultat final (par exemple, un blob de données chiffrées), sans jamais exposer les clés ou la logique interne. Cette approche est à la fois .pragmatique, sécurisée et architecturale saine

.La réponse correcte est : Aucune des réponses proposées

Noté sur 1,00 Non répondue Question 25

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes chargé de la revue de code sur une application iOS critique, majoritairement en Objective-C. Un développeur junior de votre équipe soumet une Pull Request (PR) pour une nouvelle fonctionnalité : la : mise en cache locale du jeton d'authentification de l'agent. Le code Swift ajouté est le suivant

```
// File: CredentialManager.swift
import Foundation

class CredentialManager {
static let shared = CredentialManager()
private let userTokenKey = "com.police.userAuthToken"

private init() {}

func save(token: String) {
UserDefaults.standard.set(token, forKey: userTokenKey)
}

func getToken() -> String? {
return UserDefaults.standard.string(forKey: userTokenKey)
}

func clearToken() {
UserDefaults.standard.removeObject(forKey: userTokenKey)
}
}

// File: CredentialManagerTests.swift
import XCTest
@testable import PoliceApp

class CredentialManagerTests: XCTestCase {
func testSaveAndGetToken() {
// Given
let manager = CredentialManager.shared
let testToken = "dummy-test-token-12345"

// When
manager.save(token: testToken)
let retrievedToken = manager.getToken()

// Then
XCTAssertEqual(retrievedToken, testToken, "The retrieved token should match the saved token.")

// Cleanup
manager.clearToken()
}
}
```

: Question

Conformément à vos missions de "Rédiger un code propre, maintenable" (activité 4), "Réaliser des tests" (5) et "Collaborer avec les équipes" (8), quelle(s) action(s) parmi les suivantes constitue(nt) la revue de code la plus rigoureuse, constructive et techniquement saine ?

- [ ] a. Commenter la PR en soulignant les problèmes de sécurité liés à `UserDefaults` et le manque d'isolation du test. Proposer une refonte architecturale où le `CredentialManager` n'est plus un singleton, mais une instance qui reçoit une dépendance abstraite (un protocole `SecureStorage`) pour le stockage. Cela permettrait d'injecter une implémentation `KeychainStorage` en production et une implémentation `InMemoryStorage` (mock) pour les tests unitaires, rendant le code testable et découplé.

b. Aucune des réponses proposées.

c. Demander une modification pour remplacer `UserDefaults` par le `Keychain` pour des raisons de sécurité. Une fois la modification effectuée, approuver la PR. Le test unitaire, bien qu'imparfait, valide le comportement de base.

d. Approuver la PR et la fusionner. Le code fonctionne comme attendu selon le test, et il est important de ne pas bloquer le travail du développeur junior. Les améliorations de sécurité pourront être traitées dans un ticket de dette technique ultérieur.

e. Rejeter la PR avec des commentaires détaillés expliquant pourquoi `UserDefaults` est inapproprié pour stocker des secrets et suggérer l'utilisation du `Keychain`. Expliquer que le test unitaire actuel a des effets de bord (il écrit dans les vrais `UserDefaults`) et n'est pas isolé. Proposer une session de pair-programming avec le junior pour implémenter la solution avec le Keychain et écrire des tests robustes en utilisant l'injection de dépendances et des mocks.

Analyse technique :

Le code soumis présente trois défauts majeurs qui doivent être identifiés par un développeur expert :

1. **Vulnérabilité de sécurité :** `UserDefaults` stocke les données dans un fichier .plist non chiffré dans le sandbox de l'application. Sur un appareil jailbreaké, ces données sont triviales à extraire. Pour des informations sensibles comme un jeton d'authentification, l'utilisation du `Keychain` est impérative.
2. **Problème de testabilité et d'architecture :** L'utilisation du pattern Singleton (`CredentialManager.shared`) crée un couplage fort à travers l'application et rend les tests unitaires difficiles. Le test actuel n'est pas un vrai test unitaire car il a un effet de bord : il interagit avec le système de fichiers de l'appareil (`UserDefaults.standard`), n'est pas isolé et ne peut pas être exécuté en parallèle de manière fiable.
3. **Manque de professionnalisme dans la collaboration :** Approuver un code défectueux ou exiger un correctif sans explication n'est pas une approche constructive, surtout avec un développeur junior.

Justification :

Les options B et D sont correctes car elles identifient toutes les deux les problèmes de sécurité et de testabilité. L'option B propose une solution pragmatique qui inclut le mentorat (pair-programming), ce qui est une excellente pratique de collaboration. L'option D propose une solution architecturale plus avancée (injection de dépendances via un protocole), qui est la meilleure pratique pour garantir la maintenabilité et la testabilité à long terme. Un expert reconnaîtrait la validité et la complémentarité de ces deux approches. L'option C est un distracteur plausible car elle résout le problème de sécurité le plus évident, mais ignore complètement les aspects de qualité du code et de testabilité, ce qui n'est pas suffisant pour un poste d'expert. L'option A est une mauvaise pratique flagrante.

Les réponses correctes sont : Rejeter la PR avec des commentaires détaillés expliquant pourquoi `UserDefaults` est inapproprié pour stocker des secrets et suggérer l'utilisation du `Keychain`. Expliquer que le test unitaire actuel a des effets de bord (il écrit dans les vrais `UserDefaults`) et n'est pas isolé. Proposer une session de pair-programming avec le junior pour implémenter la solution avec le Keychain et écrire des tests robustes en utilisant l'injection de dépendances et des mocks., Commenter la PR en soulignant les problèmes de sécurité liés à `UserDefaults` et le manque d'isolation du test. Proposer une refonte architecturale où le `CredentialManager` n'est plus un singleton, mais une instance qui reçoit une dépendance abstraite (un protocole `SecureStorage`) pour le stockage. Cela permettrait d'injecter une implémentation `KeychainStorage` en production et une implémentation `InMemoryStorage` (mock) pour les tests unitaires, rendant le code testable et découplé.

Noté sur 1,00 Non répondue Question 26

Contexte :

En tant que développeur mobile principal pour la police nationale, vous concevez l'architecture d'une nouvelle application native critique (Kotlin/Swift) pour les opérations tactiques en temps réel. Vos missions incluent "Analyser les besoins" (activité 1), "Participer à la conception technique et à l'architecture" (3), et "Collaborer avec les équipes backend et produit" (8).
La situation du backend est la suivante : une migration stratégique d'une API REST monolithique vers une nouvelle architecture microservices basée sur GraphQL est en cours.
- L'API REST est stable et complète, mais nécessite de multiples appels et sur-charge ("over-fetching") les données pour l'écran principal de la nouvelle application.
- L'API GraphQL est la plateforme d'avenir et offre des requêtes plus performantes pour les données qu'elle expose déjà. Cependant, des fonctionnalités critiques, comme les abonnements pour les événements en temps réel, ne seront pas disponibles avant 6 mois.
La contrainte du projet est de livrer un MVP (Minimum Viable Product) fonctionnel dans 3 mois.

Question :

Compte tenu de ce contexte, quelle(s) stratégie(s) architecturale(s) pour la couche d'accès aux données de l'application mobile est (sont) la (les) plus rigoureuse(s) et stratégiquement la (les) plus saine(s), en équilibrant les délais de livraison, la maintenabilité à long terme et la performance ?

- a. Construire l'intégralité du MVP en utilisant uniquement l'API REST existante pour garantir le respect du délai de 3 mois. Planifier une refonte complète de la couche réseau pour une V2 de l'application, une fois que l'API GraphQL sera entièrement finalisée.
- b. Proposer la mise en place d'une couche "Backend for Frontend" (BFF). Ce nouveau microservice, dédié au client mobile, agirait comme un agrégateur, interrogeant à la fois l'ancienne API REST et la nouvelle API GraphQL pour exposer une API unifiée et optimisée pour les besoins de l'application.
- c. Mettre en place une couche d'accès aux données hybride en utilisant le pattern "Repository". Chaque Repository encapsulerait la logique pour interroger soit l'API REST, soit l'API GraphQL, en fonction de la disponibilité et de l'efficacité de la source pour une donnée spécifique.
- d. Aucune des réponses proposées.
- e. Adopter immédiatement une stratégie "tout GraphQL". Pour les fonctionnalités manquantes, insister auprès de l'équipe backend pour qu'elle accélère sa feuille de route ou, à défaut, mettre en place des solutions de contournement côté client, même si elles sont inefficaces.

: Analyse technique

Cette question évalue la capacité d'un architecte mobile à naviguer dans un contexte technique et organisationnel complexe. Il ne s'agit pas seulement de connaître les technologies (REST, GraphQL), mais de savoir comment les orchestrer pour répondre à des contraintes contradictoires : livraison rapide (MVP), performance, et évolution à long terme.

: Justification

Les solutions correctes (Repository Hybride et BFF) démontrent une pensée architecturale mature. Elles proposent des couches d'abstraction (le Repository ou le BFF) pour isoler la complexité et permettre une migration progressive et contrôlée. Elles permettent de livrer de la valeur rapidement tout en construisant une base saine pour l'avenir.
Les approches "tout REST" ou "tout GraphQL" sont trop dogmatiques et ignorent une partie des contraintes. L'approche "tout REST" accumule de la dette technique, tandis que l'approche "tout GraphQL" introduit un risque de dépendance et de retard inacceptable pour un MVP.

Les réponses correctes sont : Mettre en place une couche d'accès aux données hybride en utilisant le pattern "Repository". Chaque Repository encapsulerait la logique pour interroger soit l'API REST, soit l'API GraphQL, en fonction de la disponibilité et de l'efficacité de la source pour une donnée spécifique., Proposer la mise en place d'une couche "Backend for Frontend" (BFF). Ce nouveau microservice, dédié au client mobile, agirait comme un agrégateur, interrogeant à la fois l'ancienne API REST et la nouvelle API GraphQL pour exposer une API unifiée et optimisée pour les besoins de l'application.

Note de 1,00 sur 1,00 Correct **Question 27**

Contexte : Vous intégrez la police nationale en tant que développeur mobile expérimenté. L'organisation maintient actuellement deux applications natives distinctes : une pour Android (en Java historique, architecture monolithique, faible couverture de tests) et une pour iOS (en Objective-C historique, avec des problèmes similaires). Les deux applications souffrent de délais de livraison de fonctionnalités importants, de régressions fréquentes et de goulots d'étranglement en termes de performance lors de la manipulation de données temps réel provenant d'API REST. La direction souhaite accélérer le développement et améliorer la qualité. Conformément à votre fiche de poste ("Participer à la conception technique et à l'architecture"), on vous demande d'évaluer plusieurs stratégies de refonte.

Question : Parmi les propositions suivantes, laquelle représente la stratégie la plus complète et la plus rigoureuse sur le plan technique pour résoudre les problèmes de fond, en tenant compte de l'ensemble des contraintes (maintenance, performance, tests, collaboration, veille technologique) ?

- a. Aucune des réponses proposées.

Correct. Aucune des stratégies proposées n'est complète et ne minimise suffisamment les risques inhérents à la modernisation d'une application critique. Chacune présente un défaut majeur, qu'il s'agisse du risque, de la vision à long terme ou du périmètre incomplet.

.b Proposer une réécriture complète et immédiate des deux applications en utilisant un framework multiplateforme comme React Native, en capitalisant sur les compétences JavaScript de l'équipe. L'objectif est de maximiser le partage de code et d'unifier les efforts de .développement pour livrer les fonctionnalités plus rapidement

.c Lancer un projet de réécriture complète en natif moderne (Kotlin/Jetpack Compose et Swift/SwiftUI) pour les deux plateformes. Pour accélérer, le développement se fera en parallèle des applications existantes, qui seront maintenues en mode minimal jusqu'à ce que les .nouvelles applications soient prêtes pour un remplacement complet

.d Suggérer une refonte progressive en commençant par migrer le code existant vers les langages modernes (Java vers Kotlin, Objective-C vers Swift) et en introduisant une architecture modulaire. Parallèlement, mettre en place des pipelines CI/CD pour chaque application afin .d'améliorer la qualité et d'automatiser les tests

.e Préconiser l'adoption de Kotlin Multiplatform Mobile (KMM) pour développer une bibliothèque de logique métier (logique, réseau, persistance) partagée. Cette bibliothèque serait ensuite intégrée dans les applications natives existantes, qui conserveraient leur interface .utilisateur actuelle, permettant une migration progressive et sécurisée

> **Analyse technique :** Les options proposées présentent toutes des défauts stratégiques majeurs pour un projet critique : le risque élevé du "big bang" .(A, D), le manque de vision à long terme (B), ou une vision incomplète (C)
>
> : **Justification :** Une stratégie d'expert, alignée avec la fiche de poste, combinerait plusieurs approches de manière incrémentale
>
> 1. **Stabiliser** : Augmenter immédiatement la couverture de tests et mettre en place la CI/CD sur les applications existantes pour sécuriser les .futures modifications
> 2. **Unifier la logique** : Introduire une bibliothèque partagée avec Kotlin Multiplatform Mobile (KMM) pour la logique métier et l'accès aux données, .afin de réduire la duplication et d'accélérer le développement des nouvelles fonctionnalités
> 3. **Intégrer progressivement** : Remplacer morceau par morceau la logique métier des applications existantes par des appels à la nouvelle .bibliothèque KMM
> 4. **Moderniser l'UI par étapes** : Une fois la logique métier stabilisée dans KMM, moderniser l'interface utilisateur écran par écran en utilisant les .frameworks déclaratifs modernes (Jetpack Compose, SwiftUI) pour améliorer les performances et la maintenabilité
>
> .Cette approche itérative et à faible risque maximise la livraison de valeur continue tout en modernisant l'ensemble de la pile technique

.La réponse correcte est : Aucune des réponses proposées

Noté sur 1,00 **Non répondue** **Question 28**

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes chargé de la maintenance d'une application Android native (Kotlin) critique. L'application utilise une architecture Model-View-Presenter (MVP) legacy. Les `Presenter`s détiennent souvent des références directes à des `Context`s Android pour accéder aux services et ressources du système, et il n'y a aucun framework d'injection de .dépendances. L'application souffre de fuites de mémoire et est très difficile à tester unitairement

Une nouvelle exigence à haute priorité est demandée : implémenter un service en arrière-plan qui synchronise périodiquement des fichiers de données volumineux (preuves de dossiers) même lorsque l'application n'est pas au premier plan. Ce service doit être robuste aux redémarrages de l'appareil et aux changements de réseau. La logique de synchronisation est complexe et nécessite l'accès aux mêmes .dépôts (repositories) et classes utilitaires (qui dépendent elles-mêmes de `Context`) utilisés par les `Presenter`s

Votre mission, conformément à vos activités de "Participer à la conception technique et à l'architecture" (3), "Rédiger un code propre, .maintenable" (4) et "Réaliser des tests" (5), est de proposer la stratégie d'implémentation la plus robuste et professionnelle

: Question

Compte tenu des contraintes de l'architecture MVP existante et des exigences du nouveau service en arrière-plan, laquelle des stratégies ? suivantes est la plus saine sur le plan technique et la plus maintenable

.a Refactoriser d'abord les dépendances pour utiliser l'injection par constructeur, puis introduire un conteneur d'injection de dépendances manuel initialisé dans la classe `Application`. Ensuite, implémenter la logique de synchronisation dans un `JobIntentService` qui récupère .ses dépendances depuis ce conteneur

b. Intégrer Hilt pour l'injection de dépendances et refactoriser les classes nécessaires. Implémenter la tâche avec `WorkManager` et une `PeriodicWorkRequest`. Pour garantir que la synchronisation ne soit pas retardée, configurer la requête comme une tâche accélérée (`Expedited`) via `setExpedited()`.

c. Créer un `IntentService` pour la synchronisation. À l'intérieur du service, instancier manuellement les dépendances (dépôts, etc.) en leur passant le `Context` du service. Utiliser `AlarmManager` pour planifier son exécution périodique.

d. Aucune de ces réponses.

e. Utiliser `WorkManager` avec un `CoroutineWorker`. Pour fournir le contexte aux dépendances, créer un singleton statique temporaire dans la classe `Application` qui exposera l'`applicationContext`, permettant ainsi au Worker d'initialiser les classes nécessaires.

: Analyse technique

Le scénario présente un défi courant : ajouter une fonctionnalité moderne et robuste (tâche en arrière-plan) à une application legacy avec une dette architecturale (MVP avec fuites de Context, absence d'injection de dépendances). Une implémentation simple propagerait les mauvaises pratiques, tandis qu'une réécriture complète n'est souvent pas réalisable dans les délais impartis. La solution idéale nécessite une approche de refactoring incrémental combinée à l'utilisation correcte des API Android modernes.

: Justification

Toutes les options proposées contiennent au moins une faille technique majeure, ce qui en fait des solutions sous-optimales ou incorrectes pour un expert :

- L'option A utilise `IntentService`, qui est une API obsolète (deprecated) et ne résout pas les problèmes de testabilité.
- L'option B, bien qu'utilisant le bon outil (`WorkManager`), introduit un anti-pattern majeur (singleton statique pour le contexte) qui nuit gravement à la testabilité et à la maintenabilité.
- L'option C identifie correctement le besoin de refactoring pour l'injection de dépendances, mais propose d'utiliser `JobIntentService`, une autre API obsolète.
- L'option D est la plus proche de la bonne réponse mais fait une erreur d'expert : elle utilise les requêtes `Expedited` de manière inappropriée. Celles-ci sont conçues pour des tâches immédiates et importantes pour l'utilisateur, et non pour une synchronisation périodique de fichiers volumineux, qui devrait être différable.

La stratégie correcte, non listée, consisterait à introduire un framework d'injection de dépendances (comme Hilt), à refactoriser les dépendances, puis à implémenter la fonctionnalité avec une requête `PeriodicWorkRequest` standard de `WorkManager` avec des contraintes appropriées. Par conséquent, "Aucune de ces réponses" est la seule option correcte pour un candidat expert qui identifie les défauts dans chaque proposition.

La réponse correcte est : Aucune de ces réponses.

Noté sur 1,00 **Non répondue** **Question 29**

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes responsable d'une application Android native existante, écrite en Java historique. L'application présente une dette technique significative : une architecture monolithique (une seule `God Activity` gérant de multiples `Fragments`), un couplage fort entre la logique métier et l'interface utilisateur, et une couverture de tests unitaires inférieure à 5%. Une nouvelle exigence hautement prioritaire est demandée : l'intégration d'un module de génération de formulaires dynamiques. Le backend fournira un schéma JSON décrivant la structure du formulaire, et l'application devra le rendre nativement. L'objectif est de permettre la création de nouveaux types de rapports d'incident sans redéployer l'application. Le délai pour livrer un prototype fonctionnel est très court.

: Question

Conformément à vos missions ("Participer à la conception technique et à l'architecture", "Rédiger un code propre, maintenable" et "Réaliser des tests"), quelle(s) approche(s) parmi les suivantes représente(nt) la stratégie la plus pragmatique et techniquement la plus saine pour intégrer cette fonctionnalité tout en gérant la dette technique existante ?

a. Appliquer le pattern "Strangler Fig" en créant une nouvelle `Activity` dédiée exclusivement à la génération de formulaires. Cette `Activity` sera développée en Kotlin en suivant les meilleures pratiques actuelles (architecture MVVM, injection de dépendances) et bénéficiera d'une couverture de tests unitaires complète. L'ancienne `God Activity` se contentera de lancer cette nouvelle `Activity` de manière isolée.

b. Proposer une approche en deux temps : 1) Isoler la nouvelle fonctionnalité dans une `Activity` moderne et autonome pour limiter l'impact sur le code existant. 2) Pour sécuriser l'intégration, développer une suite de tests d'intégration (avec Espresso) qui valide le passage de données entre l'ancienne `God Activity` et la nouvelle. Planifier le refactoring du reste de l'application dans le backlog.

c. Refuser de développer la fonctionnalité tant qu'une refonte complète de l'application vers une architecture moderne (ex: MVVM avec un `ViewModel` par écran) n'est pas planifiée et approuvée. Argumenter que l'ajout de complexité à la base de code actuelle est trop risqué.

d. Aucune des réponses proposées.

e. Implémenter la logique de rendu du formulaire directement dans un nouveau `Fragment` géré par la `God Activity` existante. Pour accélérer le développement, utiliser une librairie tierce de rendu de formulaires JSON et reporter la création de tests et le refactoring à une version ultérieure pour respecter le délai.

f. Suggérer de réécrire la fonctionnalité en utilisant une technologie cross-platform comme React Native et de l'intégrer via un `Fragment` qui héberge une `WebView` ou une vue React Native. Mettre en avant le gain de temps potentiel pour le développement futur de la version iOS.

: Analyse technique

La question évalue la capacité d'un développeur expert à prendre une décision architecturale stratégique dans un contexte de forte dette technique et de contraintes de temps. Il ne s'agit pas seulement de connaître les bonnes pratiques, mais de savoir comment les appliquer pragmatiquement. Les options A et B représentent des extrêmes : le raccourci dangereux et l'idéalisme bloquant. L'option E introduit une complexité inutile et des risques de performance pour ce cas d'usage. Les options C et D représentent des approches professionnelles et matures, typiques du pattern "Strangler Fig", qui permettent de livrer de la valeur tout en améliorant progressivement la qualité de la base de code. L'option D est légèrement supérieure car elle adresse explicitement le risque d'intégration par des tests automatisés, une préoccupation majeure dans un projet avec une faible couverture de tests.

: Justification

Les réponses C et D sont correctes car elles proposent d'isoler la nouvelle fonctionnalité complexe du code legacy, ce qui limite les risques de régression. Elles introduisent des pratiques modernes (Kotlin, MVVM, tests) de manière incrémentale, ce qui est une stratégie de gestion de la dette technique reconnue. Elles répondent aux exigences de la fiche de poste en équilibrant la livraison de fonctionnalités, la qualité du code, l'architecture et les tests.

Les réponses correctes sont : Appliquer le pattern "Strangler Fig" en créant une nouvelle `Activity` dédiée exclusivement à la génération de formulaires. Cette `Activity` sera développée en Kotlin en suivant les meilleures pratiques actuelles (architecture MVVM, injection de dépendances) et bénéficiera d'une couverture de tests unitaires complète. L'ancienne `God Activity` se contentera de lancer cette nouvelle `Activity` de manière isolée., Proposer une approche en deux temps : 1) Isoler la nouvelle fonctionnalité dans une `Activity` moderne et autonome pour limiter l'impact sur le code existant. 2) Pour sécuriser l'intégration, développer une suite de tests d'intégration (avec Espresso) qui valide le passage de données entre l'ancienne `God Activity` et la nouvelle. Planifier le refactoring du reste de l'application dans le backlog.

Noté sur 1,00 Non répondue Question 30

: Contexte

En tant que développeur mobile expert pour la police nationale, vous héritez d'une application native critique (Android/Java et iOS/Objective-C) présentant une dette technique significative. Une nouvelle exigence hautement prioritaire est demandée : permettre aux agents de joindre à leurs rapports des clips vidéo courts (max 15s), chiffrés, et ce même depuis des zones à faible connectivité. L'équipe backend, avec qui vous devez "Collaborer" (activité 8), a entamé une migration vers une architecture microservices avec GraphQL, mais celle-ci ne sera pas disponible avant 9 mois. L'API REST actuelle, que vous devez maintenir (activité 7), possède un endpoint d'upload de fichiers simple, avec une limite de 10 Mo et sans gestion de la reprise sur erreur.

: Question

Conformément à vos missions de "Participer à la conception technique et à l'architecture" (activité 3) et "Optimiser les performances" (activité 6), quelle(s) stratégie(s) parmi les suivantes représente(nt) l'approche la plus pragmatique et techniquement robuste pour livrer cette fonctionnalité, tout en gérant la dette existante et en préparant l'avenir ?

a. Recommander de refuser l'implémentation sur le système actuel pour ne pas aggraver la dette technique, et de bloquer la fonctionnalité jusqu'à ce que la nouvelle architecture backend GraphQL soit entièrement disponible dans 9 mois.

b. Proposer à l'équipe backend une modification minimale de l'API REST existante (ex: augmentation du timeout et de la limite de taille) et implémenter côté client une logique robuste de compression vidéo native (`MediaCodec`/`AVFoundation`) avant l'envoi, couplée à un mécanisme de tentatives (retry) avec backoff exponentiel pour gérer les échecs réseau.

c. Développer un mécanisme complexe purement côté client qui segmente la vidéo en plusieurs "chunks" de moins de 10 Mo et les envoie en série via de multiples appels au endpoint d'upload existant, sans solliciter de modification du backend.

d. Aucune des réponses proposées.

e. Initier une refonte immédiate du module de rapport en utilisant un framework cross-platform (Flutter/React Native) pour accélérer le développement de la nouvelle fonctionnalité vidéo sur les deux plateformes simultanément.

f. En collaboration avec l'équipe backend, proposer la création rapide d'un microservice dédié et isolé pour l'upload vidéo (ex: via une fonction Cloud type Firebase/AWS Lambda). Ce service, conçu pour les fichiers volumineux, serait appelé par l'application mobile et se chargerait ensuite d'intégrer la référence de la vidéo dans le système legacy, découplant ainsi la nouvelle fonctionnalité.

: Analyse technique

Cette question évalue la capacité du candidat à prendre des décisions architecturales pragmatiques dans un contexte de dette technique et de contraintes organisationnelles, une compétence clé issue des activités 3, 6, 7 et 8 de la fiche de poste. Il doit équilibrer la nécessité de livrer une fonctionnalité critique, d'optimiser les performances, de collaborer avec une équipe backend limitée et de ne pas compromettre la stratégie à long terme.

: Justification

- **Réponse A (Correcte)** : Cette option est excellente car elle est pragmatique et collaborative. Elle minimise le travail côté backend, respectant leurs contraintes, tout en résolvant le problème de manière robuste côté client (compression, gestion des erreurs réseau). C'est une solution à court terme efficace qui démontre une compréhension approfondie des optimisations mobiles.
- **Réponse B (Correcte)** : Cette option démontre une vision architecturale avancée. Le pattern "Strangler Fig" (ici sous forme de microservice) permet d'introduire de nouvelles fonctionnalités de manière moderne et isolée sans toucher au monolithe legacy. Cela facilite la maintenance, l'évolutivité et prépare la transition vers la future architecture microservices. C'est une solution stratégique à moyen terme.
- **Réponse C (Incorrecte)** : C'est un distracteur qui représente une position dogmatique mais irréaliste. Bloquer une fonctionnalité critique pour des raisons de pureté architecturale va à l'encontre des objectifs de l'organisation et de la mission de service.
- **Réponse D (Incorrecte)** : Ce distracteur est une "fausse bonne idée" technique. Bien que possible, elle ignore complètement l'impact sur le backend. L'API REST legacy n'est probablement pas conçue pour gérer un tel flux de requêtes séquentielles, ce qui pourrait entraîner des problèmes de performance, de concurrence et de fiabilité, déplaçant le problème sans le résoudre.
- **Réponse E (Incorrecte)** : C'est un distracteur qui propose une solution disproportionnée. Introduire un framework cross-platform pour un seul module dans une application native mature est complexe, risqué (intégration, performances, maintenance) et ne résout pas le problème fondamental qui est l'upload de fichiers volumineux vers une API inadaptée.

Les réponses correctes sont : Proposer à l'équipe backend une modification minimale de l'API REST existante (ex: augmentation du timeout et de la limite de taille) et implémenter côté client une logique robuste de compression vidéo native (`MediaCodec`/`AVFoundation`) avant l'envoi, couplée à un mécanisme de tentatives (retry) avec backoff exponentiel pour gérer les échecs réseau., En collaboration avec l'équipe backend, proposer la création rapide d'un microservice dédié et isolé pour l'upload vidéo (ex: via une fonction Cloud type Firebase/AWS Lambda). Ce service, conçu pour les fichiers volumineux, serait appelé par l'application mobile et se chargerait ensuite d'intégrer la référence de la vidéo dans le système legacy, découplant ainsi la nouvelle fonctionnalité.

Question 31 Non répondue Noté sur 1,00

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes chargé de l'évolution d'une application Android native critique, écrite en Java historique. L'architecture actuelle est monolithique : la couche réseau est implémentée avec des appels directs `HttpURLConnection` imbriqués dans des `AsyncTask`s, fortement couplés aux `Activity`s, rendant le code difficile à maintenir et à tester (activités 4 et 5 de la fiche de poste). L'équipe backend, avec qui vous devez "Collaborer" (activité 8), vient de livrer un nouvel endpoint GraphQL pour une fonctionnalité prioritaire de "dossier interconnecté", qui doit agréger de nombreuses informations relationnelles (suspects, véhicules, incidents liés). L'équipe produit exige un prototype fonctionnel pour le prochain sprint (2 semaines).

: Question

Conformément à vos missions de "Participer à la conception technique et à l'architecture" (activité 3) et "Rédiger un code propre, maintenable" (activité 4), quelle stratégie d'implémentation parmi les suivantes représente l'approche la plus rigoureuse et pragmatique, équilibrant la vélocité de développement et la santé architecturale à long terme ?

a. Aucune de ces réponses.

b. Isoler la nouvelle fonctionnalité dans un module Gradle distinct. Dans ce module, implémenter une couche réseau moderne dédiée à GraphQL (ex: Apollo Client) et une architecture propre (MVVM avec Repository et UseCases). Cette approche, inspirée du "Strangler Fig Pattern", permet de livrer la fonctionnalité rapidement avec des technologies modernes sans perturber le code legacy, tout en établissant un standard pour les futures migrations.

c. Créer un `NetworkManager` singleton qui encapsule à la fois les anciens appels `HttpURLConnection` et un nouveau client GraphQL. La nouvelle `Activity` utilisera ce singleton pour effectuer son appel, masquant ainsi la complexité sous-jacente.

d. Créer une nouvelle classe `GraphQLAsyncTask` qui exécute la requête GraphQL et parse le JSON manuellement, puis appeler cette classe directement depuis la nouvelle `Activity`. Cette approche réutilise le modèle existant pour une livraison rapide.

e. Refuser de commencer la nouvelle fonctionnalité et exiger une refonte complète de la couche réseau de l'application avec Retrofit, une architecture MVVM et une couverture de tests de 80% avant tout nouveau développement.

: Analyse technique

Cette question évalue la capacité d'un développeur expert à prendre des décisions architecturales stratégiques dans un contexte de dette technique et de contraintes de temps. La bonne réponse (le Strangler Fig Pattern) démontre une compréhension de l'équilibre entre les besoins immédiats du produit et la santé à long terme de la base de code. Les distracteurs représentent des erreurs communes : la réplication de mauvais patterns pour aller vite (A), l'idéalisme technique déconnecté des réalités (B), et la création de solutions de contournement qui aggravent le problème (D).

La réponse correcte est : Isoler la nouvelle fonctionnalité dans un module Gradle distinct. Dans ce module, implémenter une couche réseau moderne dédiée à GraphQL (ex: Apollo Client) et une architecture propre (MVVM avec Repository et UseCases). Cette approche, inspirée du "Strangler Fig Pattern", permet de livrer la fonctionnalité rapidement avec des technologies modernes sans perturber le code legacy, tout en établissant un standard pour les futures migrations.

Noté sur 1,00 **Non répondue** Question 32

: Contexte

En tant que nouveau développeur mobile pour la police nationale, vous êtes affecté à une application native existante (Kotlin/Swift) utilisée pour la coordination tactique en temps réel. Les agents sur le terrain se plaignent d'une consommation excessive de la batterie, d'une utilisation élevée des données mobiles et de mises à jour lentes, en particulier dans les zones à faible couverture réseau (3G/4G). Votre analyse initiale révèle que l'application interroge (poll) un unique endpoint REST (`/api/v1/situation/all`) toutes les 5 secondes pour rafraîchir l'intégralité de la situation tactique (positions des unités, événements, points d'intérêt), ce qui résulte en la récupération d'une charge utile (payload) JSON volumineuse et complexe à chaque appel. L'équipe backend est de taille réduite et a une disponibilité limitée pour des changements majeurs à court terme. Conformément à votre fiche de poste, votre rôle inclut "analyser les besoins", "participer à la conception technique et à l'architecture", "collaborer avec les équipes" et "optimiser les performances".

Parmi les stratégies initiales suivantes, laquelle représente l'approche la plus rigoureuse et la plus professionnelle pour résoudre ce problème de manière efficace ?

من بين الاستراتيجيات الأولية التالية، أيها يمثل النهج الأكثر صرامة واحترافية لحل هذه المشكلة بفعالية؟

a. Utiliser des outils de profilage (ex: Android Profiler, Instruments) pour identifier les goulots d'étranglement spécifiques au parsing JSON et au rendu de l'interface utilisateur. Optimiser ces sections du code, par exemple en déplaçant le parsing sur un thread d'arrière-plan et en utilisant des mécanismes de mise à jour de l'UI plus efficaces comme `DiffUtil`.

b. Aucune des réponses proposées.

c. Refactoriser l'application mobile pour utiliser des WebSockets. Proposer à l'équipe backend un nouveau endpoint WebSocket qui enverra des mises à jour différentielles (delta updates) au client. C'est la norme moderne pour la communication temps réel et cela résoudra le problème de manière définitive.

d. Implémenter immédiatement une couche de cache côté client avec une durée de vie (TTL) de 30 secondes. Cela réduira le nombre de requêtes d'un facteur 6, adressant directement les problèmes de consommation de données et de batterie sans nécessiter de modification du backend.

e. Demander à l'équipe backend d'implémenter immédiatement GraphQL. Cela permettra au client mobile de ne demander que les champs de données dont il a besoin, ce qui est la manière la plus efficace de résoudre le problème de sur-récupération de données (over-fetching) inhérent à l'API REST actuelle.

: Analyse technique

La question évalue la capacité du candidat à analyser un problème de performance complexe et à proposer une solution pragmatique qui tient compte des contraintes techniques et organisationnelles, conformément aux responsabilités décrites dans la fiche de poste. Une approche experte ne consiste pas à appliquer aveuglément une solution "moderne" (comme WebSockets ou GraphQL) ou un correctif superficiel, mais à suivre une démarche méthodique.

: Justification

Aucune des options proposées n'est idéale.

- **Option A** est incorrecte car un cache TTL de 30 secondes est inacceptable pour une application de coordination "en temps réel". Cela introduit une latence qui peut avoir des conséquences critiques sur le terrain.
- **Option B** est prématurée. Proposer une refonte complète vers WebSockets sans une analyse approfondie et en ignorant la contrainte de la disponibilité de l'équipe backend n'est pas une stratégie professionnelle. C'est une solution lourde qui doit être planifiée.
- **Option C** est insuffisante. Optimiser le parsing JSON et le rendu UI est une bonne pratique, mais cela ne résout pas le problème fondamental : la consommation excessive de données et la charge réseau. C'est une micro-optimisation qui traite les symptômes, pas la cause.
- **Option D** est également prématurée et irréaliste à court terme, pour les mêmes raisons que l'option B. Exiger une migration vers GraphQL ignore les contraintes du projet.

La stratégie correcte, non listée, serait :

1. **Quantifier le problème :** Utiliser des outils de profilage réseau pour documenter précisément la taille des données, la fréquence et la durée des appels.
2. **Collaborer et proposer une solution incrémentale :** Présenter ces données à l'équipe backend et proposer la solution la plus simple et la plus standard pour ce cas : l'utilisation des en-têtes HTTP `ETag` et `If-None-Match`. Cela permettrait au serveur de répondre avec un statut `304 Not Modified` si les données n'ont pas changé, évitant ainsi le transfert de la charge utile complète. C'est une modification mineure côté backend avec un impact majeur côté client.
3. **Planifier à long terme :** Documenter cette solution comme un correctif à court terme et initier une discussion architecturale pour une solution à long terme (WebSockets, GraphQL, etc.) à intégrer dans la feuille de route technique.

La réponse correcte est : Aucune des réponses proposées.

Note de -0,25 sur 1,00 Incorrect Question 33

: Contexte

En tant que nouveau développeur mobile expert pour la police nationale, vous êtes affecté à une application native existante (Kotlin/Swift) qui présente des signes de lenteur et une consommation de batterie excessive lors d'opérations sur le terrain. Votre analyse initiale, conforme à l'activité "Corriger les bugs et optimiser les performances", révèle que l'application interroge (poll) un endpoint REST unique (`/api/situation/updates`) toutes les 3 secondes. Cet appel retourne systématiquement une charge utile JSON volumineuse contenant l'état complet de toutes les unités, même si une seule a changé de position. L'équipe backend, avec qui vous devez "Collaborer" selon la fiche de poste, est de taille réduite et a une bande passante très limitée pour des changements d'architecture majeurs à court terme. Votre mission, en tant qu'expert qui doit "Participer à la conception technique et à l'architecture", est de proposer un plan d'action initial.

: Question

Parmi les propositions suivantes, laquelle représente la démarche la plus rigoureuse et stratégiquement la plus saine pour initier la résolution de ce problème, en tenant compte de toutes les contraintes ?

a. Aucune des réponses proposées.

b. Modifier immédiatement le code client pour augmenter l'intervalle de polling à 15 secondes et ajouter un mécanisme de "pull-to-refresh" pour permettre aux agents de forcer la mise à jour manuellement.

Incorrect. C'est une solution de contournement qui ne résout pas le problème de fond (la taille de la charge utile et l'inefficacité des appels). Pire, elle dégrade l'aspect "temps-réel" de l'application, ce qui peut être dangereux dans un contexte policier.

c. Rédiger un document d'analyse technique démontrant que le polling est une anti-pratique et exiger de l'équipe backend qu'elle priorise la mise en place d'une communication temps-réel via WebSockets ou la migration vers GraphQL.

d. Implémenter un mécanisme de mise en cache agressif côté client (ex: cache en mémoire pour 30 secondes) pour éviter de faire des appels réseau si les données ont été récemment récupérées, sans en informer l'équipe backend.

e. Initier un projet de refonte complet de la couche réseau de l'application mobile en utilisant une bibliothèque plus moderne comme gRPC, qui est plus efficace que REST, et planifier la migration du backend en conséquence.

: Analyse technique

Cette question évalue la capacité du candidat à agir en tant qu'architecte et leader technique, et pas seulement en tant que simple développeur. Elle teste la capacité à équilibrer les contraintes techniques, organisationnelles et les objectifs de performance, ce qui est au cœur des missions "Analyser les besoins", "Participer à la conception technique" et "Collaborer avec les équipes".

: Justification

: Aucune des options proposées n'est une solution complète et rigoureuse. Un expert identifierait les défauts de chaque approche

- L'option A est une solution unilatérale qui ignore la collaboration et les bonnes pratiques de gestion de cache (qui nécessitent un support backend comme les ETags).
- L'option B ignore les contraintes organisationnelles (équipe backend limitée) et propose une solution à long terme pour un problème immédiat, manquant de pragmatisme.
- L'option C est une solution de contournement qui dégrade l'expérience utilisateur et ne résout pas le problème fondamental de l'inefficacité du réseau.
- L'option D est une solution extrême et prématurée qui ne s'appuie sur aucune donnée chiffrée et ignore des solutions intermédiaires moins coûteuses.

: La démarche correcte, non listée, serait une approche progressive et collaborative

1. **Quantifier le problème :** Mettre en place des outils de monitoring (profiling réseau, batterie) pour obtenir des métriques précises.
2. **Atelier de conception :** Organiser une réunion avec l'équipe backend pour présenter les données et discuter des solutions à faible coût pour eux (ex: ajout d'un header `If-Modified-Since` ou d'ETags, création rapide d'un endpoint qui ne retourne que les deltas depuis un timestamp donné).
3. **Feuille de route :** Proposer une feuille de route d'optimisation (court, moyen, long terme) qui commence par des améliorations simples et évolue vers des solutions plus complexes comme les WebSockets ou GraphQL, en alignement avec la stratégie globale de l'entreprise.

La réponse correcte est : Aucune des réponses proposées.

Note de -0,25 sur 1,00 Incorrect **Question 34**

: Contexte

Vous êtes le développeur mobile principal pour la police nationale. Une application critique, utilisée par les agents en patrouille, communique avec le backend via une API RESTful. Lors d'opérations de grande envergure (ex: sécurisation d'un événement majeur), l'application devient extrêmement lente et les données ne sont plus synchronisées en temps réel. L'équipe backend signale une surcharge massive due à un phénomène de "thundering herd" : chaque client de l'application interroge (poll) simultanément et à haute fréquence plusieurs endpoints (ex: `/api/officers/status`, `/api/map/updates`, `/api/messages/new`) pour maintenir les informations à jour. Votre mission, conformément à la fiche de poste, est de "corriger les bugs et optimiser les performances" et de "participer à la conception technique et à l'architecture".

: Question

Parmi les stratégies suivantes, laquelle, implémentée **uniquement côté client**, représente la solution **immédiate et unique** la plus rigoureuse pour résoudre **définitivement** le problème de surcharge du backend tout en garantissant la synchronisation des données en temps réel ?

a. Intégrer une base de données locale (ex: Room/Realm) avec une stratégie de cache agressive (TTL de 1 seconde) pour servir les données directement depuis l'appareil et ne faire un appel réseau que si le cache a expiré.

Incorrect. Un TTL (Time-To-Live) de 1 seconde est fonctionnellement équivalent à un polling toutes les secondes. Cela ne change rien au problème de "thundering herd" sur le backend et ne constitue pas une solution "définitive".

b. Mettre en place un système de "request batching" qui regroupe les trois appels API en un seul appel réseau toutes les 2 secondes, en utilisant les capacités de multiplexage de HTTP/2.

c. Aucune des réponses proposées.

d. Optimiser l'exécution des requêtes en les passant toutes sur des threads d'arrière-plan via des coroutines (Kotlin) ou GCD (Swift) et réduire l'intervalle de polling à 500ms pour augmenter la fraîcheur des données.

e. Intégrer un service tiers comme Firebase (Cloud Firestore) pour gérer uniquement la synchronisation des données temps réel, en laissant le reste des fonctionnalités sur l'API REST existante.

: Analyse technique

Le cœur du problème est l'architecture de communication basée sur le polling à grande échelle, qui est intrinsèquement non scalable pour des besoins temps réel. Aucune solution purement "côté client" ne peut "définitivement" résoudre un problème de surcharge serveur de cette nature. Les solutions proposées sont des palliatifs ou aggravent le problème. Un expert doit reconnaître que le problème est architectural et nécessite une refonte du protocole de communication (passer d'un modèle PULL à un modèle PUSH), ce qui implique une collaboration avec l'équipe backend et ne peut être une décision "unilatérale, immédiate et uniquement côté client".

: Justification

La seule réponse correcte est de reconnaître qu'aucune des options proposées ne satisfait simultanément toutes les contraintes de la question (uniquement côté client, immédiate, unique, et résolution définitive). La solution réelle (par exemple, l'adoption de WebSockets, gRPC streaming, ou GraphQL Subscriptions) n'est pas listée car elle n'est pas une stratégie "uniquement côté client". Cette question teste la capacité du candidat à analyser un problème complexe dans sa globalité, à comprendre les limites de son périmètre d'action et à ne pas choisir une solution techniquement plausible mais inadaptée au problème de fond.

La réponse correcte est : Aucune des réponses proposées.

Note de -0,25 sur 1,00 Incorrect Question 35

: Contexte

En tant que nouveau développeur mobile expert pour la police nationale, vous êtes affecté à une application native existante (Kotlin/Swift) qui présente des signes de lenteur et une consommation de batterie excessive lors d'opérations sur le terrain. Votre analyse initiale, conforme à l'activité "Corriger les bugs et optimiser les performances" de votre fiche de poste, révèle que l'application interroge (poll) un endpoint REST unique (`/api/v2/situation`) toutes les 2 secondes. Cet appel retourne systématiquement une charge utile JSON volumineuse contenant l'état complet de toutes les unités tactiques, même si une seule a changé de statut ou de position. L'équipe backend, avec qui vous devez "Collaborer" selon la fiche de poste, est de taille réduite et a une bande passante très limitée pour des changements d'architecture majeurs à court terme.

: Question

Conformément à vos missions ("Analyser les besoins", "Participer à la conception technique et à l'architecture"), quelle stratégie initiale parmi les suivantes représente la démarche la plus rigoureuse et la plus pragmatique pour adresser ce problème de fond ?

a. Commencer immédiatement le refactoring de la couche de données de l'application mobile vers une architecture modulaire (ex: Repository Pattern) pour la préparer à une future migration de l'API, tout en continuant d'utiliser l'endpoint actuel.

b. Augmenter unilatéralement l'intervalle de polling côté client à 15 secondes et ajouter un mécanisme de "jitter" (délai aléatoire) pour désynchroniser les requêtes des différents appareils.

c. Aucune des réponses proposées.

d. Implémenter une stratégie de cache client robuste en utilisant les en-têtes HTTP ETag et `If-None-Match`. L'application enverra l'ETag de la dernière réponse, et le serveur répondra `304 Not Modified` si les données n'ont pas changé, réduisant ainsi la consommation de données et le parsing côté client.

e. Proposer immédiatement une migration complète vers une communication temps-réel via WebSockets. Cela implique de réécrire la couche réseau de l'application et de demander au backend de développer un nouveau service dédié, car c'est la seule solution technique viable à long terme.

Cette approche, bien que techniquement valide à long terme, ignore la contrainte majeure de la faible disponibilité de l'équipe backend. Une proposition aussi disruptive en phase initiale, sans analyse collaborative préalable, est irréaliste et ne respecte pas le principe de collaboration.

: Analyse technique

Le problème est une surconsommation de ressources (réseau, batterie, CPU client, charge serveur) due à une stratégie de polling inefficace (charge utile complète au lieu de différentielle) pour un besoin temps-réel. Les solutions proposées sont soit irréalistes (A), incomplètes (B), fonctionnellement inacceptables (C) ou non pertinentes pour le problème immédiat (D).

: Justification

La démarche experte initiale, absente des propositions, consisterait à :

1. **Quantifier le problème**: Utiliser des outils de profilage (Android Profiler, Instruments) pour mesurer précisément l'impact sur le client et demander des métriques précises au backend.
2. **Collaborer et proposer une solution incrémentale**: Organiser une réunion avec l'équipe backend pour présenter les données quantifiées. Proposer la solution la plus simple et à plus fort impact pour eux, comme la création d'un nouvel endpoint `api/v2/situation/diffs?since=[timestamp]` qui ne renverrait que les changements (deltas).
3. **Établir une feuille de route**: Utiliser cette première optimisation comme base pour planifier une migration architecturale plus ambitieuse (WebSockets, GraphQL) à moyen ou long terme, en accord avec la disponibilité du backend.

Cette approche pragmatique, collaborative et basée sur les données est la marque d'un développeur expert.

La réponse correcte est : Aucune des réponses proposées.

Noté sur 1,00 Non répondue Question 36

: Contexte

Vous êtes recruté en tant que développeur mobile expert au sein de la police nationale. Vous héritez d'une application critique existante, avec des bases de code distinctes pour Android (Java historique, architecture monolithique) et iOS (Objective-C historique). L'application souffre de régressions fréquentes, de performances médiocres et d'un cycle de développement lent. Parallèlement, l'équipe backend annonce une migration stratégique de ses API REST vers GraphQL, qui sera finalisée d'ici 12 mois, afin de mieux répondre aux besoins complexes en données des agents sur le terrain. Votre mission principale, conformément à la fiche de poste, est de "participer à la conception technique et à l'architecture" et d'"assurer la maintenance et les mises à jour" tout en optimisant les performances.

: Question

Compte tenu de ce contexte, quelle(s) proposition(s) d'action démontre(nt) l'approche la plus rigoureuse et experte, alignant les choix techniques avec les contraintes du projet et les responsabilités du poste ?

a. Mener un audit complet des applications existantes pour identifier les goulots d'étranglement de performance et les zones critiques de dette technique. Proposer une stratégie hybride : pour les écrans non critiques, évaluer l'utilisation d'une solution multiplateforme (comme Flutter pour ses performances) afin d'accélérer le développement, tout en conservant les fonctionnalités critiques et dépendantes du matériel en modules natifs (Kotlin/Swift). Cette approche doit être validée par une Preuve de Concept (PoC) qui intègre également un client GraphQL préliminaire.

b. Aucune des réponses proposées.

c. Initier un refactoring progressif des applications natives. Pour Android, migrer le code de Java vers Kotlin et introduire une architecture moderne (MVVM). Pour iOS, migrer d'Objective-C vers Swift. Continuer à consommer l'API REST et traiter la migration GraphQL comme un projet distinct et ultérieur.

d. Lancer immédiatement une réécriture complète des deux applications en utilisant un framework multiplateforme comme React Native pour unifier la base de code et accélérer le développement. Utiliser l'API REST existante via une couche de compatibilité et prévoir de basculer vers GraphQL une fois qu'il sera prêt.

e. Définir une stratégie de modularisation pour les deux applications natives. Isoler les fonctionnalités existantes dans des modules indépendants et développer les nouvelles fonctionnalités dans des modules propres (en Kotlin/Swift). En parallèle, construire un nouveau module "données" qui abstrait la source de données, en le concevant pour être compatible avec GraphQL afin de faciliter une transition en douceur depuis l'API REST sans impacter le reste de l'application.

: Analyse technique

Cette question évalue la capacité du candidat à élaborer une stratégie technique à long terme face à une dette technique importante et une évolution majeure du backend. Un expert doit savoir équilibrer la maintenance de l'existant, l'amélioration progressive, la gestion des risques et l'alignement stratégique avec les autres équipes, ce qui correspond aux activités de la fiche de poste ("conception technique", "maintenance", "optimiser les performances", "collaborer avec les équipes backend").

: Justification

- **Options A et C (Correctes) :** Elles représentent deux approches matures et professionnelles. L'option A (modularisation) est une application du "Strangler Fig Pattern", une stratégie reconnue pour moderniser les systèmes hérités à faible risque. Elle prépare activement la migration vers GraphQL. L'option C (audit et approche hybride) est également excellente car elle est basée sur des données concrètes (l'audit) et explore un compromis intelligent entre la vitesse du multiplateforme et la robustesse du natif pour les parties critiques, tout en validant les hypothèses par un PoC. Ces deux stratégies démontrent une vision architecturale solide.
- **Option B (Incorrecte) :** La réécriture complète ("big bang rewrite") est notoirement risquée, surtout pour une application critique. Elle interrompt la livraison de valeur, introduit un risque massif d'échec et ignore la nécessité de maintenir l'application actuelle. C'est une approche souvent proposée par des profils moins expérimentés qui sous-estiment la complexité de la migration.
- **Option D (Incorrecte) :** Cette approche est trop conservatrice. Bien que le refactoring natif soit bénéfique, ignorer activement la migration imminente vers GraphQL est une erreur stratégique. Cela conduirait à un travail redondant lorsque le backend changera, et démontre un manque de collaboration et de vision à long terme.

Les réponses correctes sont : Définir une stratégie de modularisation pour les deux applications natives. Isoler les fonctionnalités existantes dans des modules indépendants et développer les nouvelles fonctionnalités dans des modules propres (en Kotlin/Swift). En parallèle, construire un nouveau module "données" qui abstrait la source de données, en le concevant pour être compatible avec GraphQL afin de faciliter une transition en douceur depuis l'API REST sans impacter le reste de l'application., Mener un audit complet des applications existantes pour identifier les goulots d'étranglement de performance et les zones critiques de dette technique. Proposer une stratégie hybride : pour les écrans non critiques, évaluer l'utilisation d'une solution multiplateforme (comme Flutter pour ses performances) afin d'accélérer le développement, tout en conservant les fonctionnalités critiques et dépendantes du matériel en modules natifs (Kotlin/Swift). Cette approche doit être validée par une Preuve de Concept (PoC) qui intègre également un client GraphQL préliminaire.

Note de 0,17 sur 1,00 Partiellement correct Question 37

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes chargé d'améliorer les processus de développement d'une application native (Kotlin/Swift) critique. L'équipe actuelle utilise une stratégie de branching Git chaotique : des branches de fonctionnalités (feature-branches) vivent plusieurs semaines et sont fusionnées directement dans la branche `main` lorsque la fonctionnalité est jugée "terminée". Cela entraîne des conflits de fusion complexes et fréquents, une branche `main` souvent instable et non déployable, et un processus de release entièrement manuel et risqué. La pipeline CI/CD existante se contente de lancer des tests unitaires sur chaque `push`, sans aucune automatisation des builds de release ou des déploiements. Conformément à vos missions de "Participer à la conception technique et à l'architecture" (activité 3) et de "Rédiger un code propre, maintenable" (activité 4), la direction vous demande de proposer une solution structurante.

: Question

Quelle(s) stratégie(s) parmi les suivantes représente(nt) une approche architecturale robuste et professionnelle pour résoudre ces problèmes de fond et garantir des livraisons plus rapides et fiables ?

a. Aucune des réponses proposées.

.b Adopter la stratégie GitFlow. Isoler le développement sur une branche `develop`, utiliser des branches `feature` pour les nouvelles fonctionnalités, des branches `release` pour préparer les versions de production, et des branches `hotfix` pour les corrections urgentes. Configurer la CI/CD pour construire et tester automatiquement les branches `develop` et `release`, et pour automatiser .le déploiement vers les environnements de test et de pré-production

Correct. GitFlow est une stratégie très structurée qui offre une excellente stabilité et un contrôle précis sur le processus de release, ce qui est particulièrement adapté aux applications critiques où la fiabilité de la branche de production .est primordiale

.c Conserver le modèle actuel mais imposer une politique de `rebase` systématique des branches de fonctionnalités sur `main` avant toute fusion, et rendre les .revues de code (pull requests) plus strictes avec au moins deux approbateurs

Incorrect. Bien que ces pratiques soient bénéfiques, elles ne constituent qu'une amélioration tactique et ne résolvent pas le problème architectural fondamental des branches de longue durée qui sont la source principale .`des conflits complexes et de l'instabilité de `main

.d Créer une nouvelle branche `integration` à partir de `main`. Les développeurs continuent de travailler sur leurs branches de fonctionnalités de longue durée, et les fusionnent dans `integration` une fois par semaine. La CI/CD est configurée pour tester cette branche `integration` .chaque nuit

.e Mettre en place une stratégie de Trunk-Based Development (TBD) avec des branches de fonctionnalités à très courte durée de vie (moins d'un jour) fusionnées quotidiennement dans le tronc (`main`). Conditionner l'intégration de toute nouvelle fonctionnalité dans l'interface utilisateur par l'utilisation de "feature flags" (via un service comme Firebase Remote Config) pour garantir que la branche `main` est .toujours dans un état déployable

: Analyse technique

Le problème principal est l'absence d'une stratégie de gestion de code source structurée, ce qui rend la branche principale (`main`) instable et le processus de release imprévisible. Une solution robuste doit à la fois organiser le flux de travail des développeurs et s'intégrer profondément avec .l'automatisation CI/CD pour garantir la qualité et la fiabilité

: Justification

- **GitFlow (A)** est une excellente solution pour les applications nécessitant un contrôle strict des versions et une grande stabilité. La séparation claire entre les branches de développement, de release et de production permet de sécuriser le processus et de garantir que `main` ne contient .que du code testé et validé. C'est un choix classique et très robuste pour un contexte critique
- **Trunk-Based Development (B)** est une approche plus moderne et agile qui favorise l'intégration continue et réduit drastiquement les conflits de fusion. En gardant les branches très courtes et en utilisant des feature flags, on s'assure que `main` est toujours prêt pour une release. Cette stratégie est très performante mais exige une maturité technique élevée, notamment une suite de tests automatisés très complète et rapide, ce .qui en fait une solution d'expert
- Les options C et D sont des "anti-patterns" ou des solutions incomplètes. L'option C (rebase et revues de code) améliore la qualité locale mais ne résout pas le problème systémique des branches de longue durée et du manque d'automatisation des releases. L'option D (branche d'intégration) ne fait que déplacer et concentrer le problème, créant un "enfer de la fusion" (merge hell) périodique et une branche d'intégration .chroniquement instable

Les réponses correctes sont : Adopter la stratégie GitFlow. Isoler le développement sur une branche `develop`, utiliser des branches `feature` pour les nouvelles fonctionnalités, des branches `release` pour préparer les versions de production, et des branches `hotfix` pour les corrections urgentes. Configurer la CI/CD pour construire et tester automatiquement les branches `develop` et `release`, et pour automatiser le déploiement vers les environnements de test et de pré-production., Mettre en place une stratégie de Trunk-Based Development (TBD) avec des branches de fonctionnalités à très courte durée de vie (moins d'un jour) fusionnées quotidiennement dans le tronc (`main`). Conditionner l'intégration de toute nouvelle fonctionnalité dans l'interface utilisateur par l'utilisation de "feature flags" (via un service comme Firebase Remote Config) pour garantir .que la branche `main` est toujours dans un état déployable

Noté sur 1,00 Non répondue Question 38

: Contexte

En tant que développeur mobile expérimenté pour la police nationale, vous êtes responsable d'une application Android native existante, écrite en Java historique. L'application présente une dette technique significative : une architecture monolithique (une seule `God Activity` gérant des dizaines de `Fragments`), une couche d'accès aux données fortement couplée à l'interface utilisateur, et une couverture de tests unitaires quasi inexistante. Cette situation rend toute évolution lente et risquée, impactant directement vos missions "Rédiger un code propre, maintenable" (activité 4) et "Corriger les bugs" (activité 6). Une nouvelle exigence à haute priorité vient d'être demandée : l'intégration d'un module complexe d'identification biométrique. Une analyse préliminaire montre que l'ajout direct de ce module dans .l'architecture actuelle augmenterait sa complexité et créerait des risques de régression importants

: Question

Conformément à vos responsabilités "Participer à la conception technique et à l'architecture" (activité 3) et "Assurer la maintenance et les mises à jour" (activité 7), quelle(s) stratégie(s) parmi les suivantes représente(nt) l'approche la plus rigoureuse et pragmatique pour gérer la dette technique existante tout en intégrant la nouvelle fonctionnalité ?

- a. Plaider pour une réécriture complète de l'application en Kotlin avec une architecture moderne (ex: MVI), en arguant que tout travail sur la base de code actuelle est une perte de temps et qu'il est préférable de repartir de zéro pour garantir la qualité.
- b. Intégrer le nouveau module directement dans la `God Activity` existante, mais en encapsulant sa logique dans un nouvel ensemble de classes "utilitaires" pour limiter le mélange de code. Ajouter quelques tests d'intégration pour couvrir le flux principal de la nouvelle fonctionnalité.
- c. Proposer une refonte incrémentale en utilisant le "Strangler Fig Pattern". Isoler la partie de l'application qui interagira avec le nouveau module dans un nouveau composant bien architecturé (ex: utilisant MVVM). Développer le module biométrique au sein de ce composant isolé avec une couverture de tests complète.
- d. Implémenter le nouveau module dans une "Android Library Module" (.aar) distincte. Définir une interface (API) claire et stable entre l'application legacy et ce nouveau module, ce qui permet de le développer indépendamment avec sa propre architecture et ses propres tests.
- e. Aucune de ces réponses.

> **: Analyse technique**
>
> Cette question évalue la capacité du candidat à prendre des décisions architecturales stratégiques face à une dette technique importante, une situation très courante dans la maintenance de logiciels critiques. Un expert doit savoir équilibrer les besoins de livraison de nouvelles fonctionnalités avec la nécessité de maîtriser la complexité et d'améliorer la qualité du code sur le long terme.
>
> **: Justification**
>
> Les approches correctes (A et B) favorisent une modernisation incrémentale et contrôlée. Elles permettent de développer la nouvelle fonctionnalité dans un environnement sain et isolé (un nouveau module ou une bibliothèque), avec des pratiques modernes (architecture propre, tests), tout en se connectant à l'ancien code via une interface claire. C'est la stratégie la plus professionnelle car elle est pragmatique, réduit les risques et apporte une valeur immédiate tout en initiant l'amélioration de la base de code.
>
> La réécriture complète (C) est souvent irréaliste et risquée ("big bang rewrite"), ignorant les contraintes de temps. L'intégration directe (D) est une solution de facilité qui aggrave la dette technique et la complexité, rendant la maintenance future encore plus difficile.

> Les réponses correctes sont : Proposer une refonte incrémentale en utilisant le "Strangler Fig Pattern". Isoler la partie de l'application qui interagira avec le nouveau module dans un nouveau composant bien architecturé (ex: utilisant MVVM). Développer le module biométrique au sein de ce composant isolé avec une couverture de tests complète., Implémenter le nouveau module dans une "Android Library Module" (.aar) distincte. Définir une interface (API) claire et stable entre l'application legacy et ce nouveau module, ce qui permet de le développer indépendamment avec sa propre architecture et ses propres tests.

Noté sur 1,00 Non répondue Question 39

: Contexte

En tant que développeur mobile expert pour la police nationale, vous intervenez sur une application iOS critique, écrite avec un mélange d'Objective-C historique et de Swift moderne. Un composant central, `APIManager`, est un singleton Objective-C responsable de toutes les communications réseau. Votre audit révèle les problèmes suivants :

1. L'instance du singleton est accédée directement depuis les `ViewControllers`) dans toute l'application (`[APIManager sharedManager]`).
2. Ses méthodes prennent souvent des instances de `UIViewController` en paramètre afin d'afficher directement des alertes (`UIAlertController`) en cas d'échec réseau.

3. La classe n'a aucune couverture de tests unitaires, et son couplage fort avec `UIKit` la rend non testable en dehors de tests d'interface utilisateur lents et peu fiables.
4. Une nouvelle fonctionnalité prioritaire exige l'ajout de plusieurs nouveaux appels API et la modification de la logique de gestion d'erreurs existante dans `APIManager`.

Question :

Conformément à vos missions ("Rédiger un code propre, maintenable", "Réaliser des tests", "Participer à la conception technique et à l'architecture"), quelle(s) stratégie(s) parmi les suivantes représente(nt) l'approche la plus rigoureuse et pragmatique pour introduire la nouvelle fonctionnalité tout en améliorant la testabilité et la maintenabilité de la couche réseau ?

- a. Isoler la gestion des erreurs de la couche réseau. Créer un mécanisme de gestion d'erreurs dédié (ex: un coordinateur ou un service spécifique) qui est découplé de la couche réseau. Le `NetworkService` retournera désormais des `Result<Success, Error>` au lieu d'afficher directement des alertes UI. Cela sépare les responsabilités et rend la logique réseau et la logique de présentation des erreurs testables indépendamment.
- b. Aucune de ces réponses.
- c. Ajouter les nouvelles méthodes directement dans le singleton `APIManager` existant et utiliser des directives de compilation (`#if DEBUG`) pour désactiver l'affichage des `UIAlertController` pendant les tests.
- d. Réécrire immédiatement et intégralement la classe `APIManager` en Swift en utilisant des bibliothèques modernes comme Alamofire.
- e. Appliquer une refactorisation progressive basée sur la Programmation Orientée Protocole. Définir un protocole Swift (ex: `NetworkService`) qui abstrait les opérations réseau. Créer une nouvelle classe Swift `DefaultNetworkService` qui implémente ce protocole et encapsule (wrap) l'ancien `APIManager`. Utiliser l'injection de dépendances pour fournir ce service aux `ViewControllers`), leur permettant ainsi de ne plus accéder directement au singleton. Cette approche permet la création d'un `MockNetworkService` pour les tests unitaires.

> **Analyse technique :**
>
> Le problème principal est le couplage fort entre la couche réseau (`APIManager`) et la couche de présentation (`UIViewController`, `UIKit`), ainsi que l'utilisation du patron de conception Singleton qui rend l'injection de dépendances et les tests difficiles. Une approche experte vise à briser ces couplages de manière progressive et sécurisée.
>
> **Justification :**
>
> - **Réponses A et B (Correctes) :** Ces deux stratégies sont complémentaires et représentent la meilleure pratique. L'utilisation d'un protocole (A) crée une abstraction (un "seam") qui permet de découpler le code client de l'implémentation concrète du singleton Objective-C. Cela permet d'injecter une fausse implémentation (mock) pour les tests unitaires. Isoler la gestion des erreurs (B) est une application directe du principe de responsabilité unique (Single Responsibility Principle). La couche réseau ne doit s'occuper que du réseau, pas de l'affichage des alertes. Le retour d'un type `Result` permet au code appelant de décider comment gérer les succès et les erreurs, rendant les deux composants testables indépendamment.
> - **Réponse C (Incorrecte) :** La réécriture "big bang" est une erreur classique. Elle est risquée, prend du temps et bloque la livraison de valeur (la nouvelle fonctionnalité). Une refactorisation progressive est toujours préférable pour les systèmes critiques en production.
> - **Réponse D (Incorrecte) :** C'est une solution de contournement qui ne fait qu'aggraver la dette technique. Elle ne résout pas le problème fondamental du couplage et rend le code plus complexe et plus difficile à maintenir à long terme.

> Les réponses correctes sont : Appliquer une refactorisation progressive basée sur la Programmation Orientée Protocole. Définir un protocole Swift (ex: `NetworkService`) qui abstrait les opérations réseau. Créer une nouvelle classe Swift `DefaultNetworkService` qui implémente ce protocole et encapsule (wrap) l'ancien `APIManager`. Utiliser l'injection de dépendances pour fournir ce service aux `ViewControllers`), leur permettant ainsi de ne plus accéder directement au singleton. Cette approche permet la création d'un `MockNetworkService` pour les tests unitaires., Isoler la gestion des erreurs de la couche réseau. Créer un mécanisme de gestion d'erreurs dédié (ex: un coordinateur ou un service spécifique) qui est découplé de la couche réseau. Le `NetworkService` retournera désormais des `Result<Success, Error>` au lieu d'afficher directement des alertes UI. Cela sépare les responsabilités et rend la logique réseau et la logique de présentation des erreurs testables indépendamment.

Note de -0,25 sur 1,00 Incorrect **Question 40**

Contexte : Vous êtes chargé de sécuriser les communications entre une nouvelle application mobile de la police (développée en Kotlin/Swift) et une API REST centrale qui manipule des données d'enquête hautement sensibles. Une exigence fondamentale est d'empêcher les attaques de type "Man-in-the-Middle" (MitM), même dans le scénario où le magasin de confiance (trust store) du système

d'exploitation du terminal est compromis (par exemple, par l'installation d'un certificat d'autorité de certification - CA - malveillant par un .logiciel espion)

Question : Parmi les implémentations suivantes, laquelle représente une approche technique à la fois moderne, robuste et ? opérationnellement viable pour répondre à cette exigence de sécurité côté client

- .a Aucune des réponses proposées.
- .b Mettre en place le "Certificate Pinning" en stockant dans l'application un hash du certificat feuille (leaf certificate) exact du serveur. Lors .de la poignée de main TLS, toute connexion est rejetée si le certificat présenté par le serveur ne correspond pas au hash stocké
- .c Mettre en place le "Public Key Pinning" en "pinnant" la clé publique d'une autorité de certification racine (Root CA) publique et réputée (par .exemple, DigiCert ou Let's Encrypt). Cela garantit que seuls les certificats émis par cette autorité sont acceptés
- .d Mettre en œuvre le protocole HTTPS/TLS standard, en se fiant à la validation par défaut du système d'exploitation de la chaîne de .certificats du serveur par rapport à son magasin de CA de confiance
- .e Utiliser les en-têtes HTTP Public Key Pinning (HPKP). Configurer le serveur pour qu'il envoie l'en-tête `Public-Key-Pins` lors de la première connexion, demandant au client de mémoriser (pin) les clés publiques du serveur pour une durée .déterminée

Incorrect. HPKP est une technologie dépréciée et considérée comme dangereuse. Elle a été retirée de la plupart des clients modernes car une mauvaise configuration pouvait très facilement rendre un service .complètement inaccessible pour tous les utilisateurs

Analyse technique : Cette question évalue la compréhension approfondie des mécanismes de sécurisation des canaux de communication au-delà du simple TLS, en particulier dans un contexte de haute sécurité où le client ne peut pas être entièrement fiable. Le candidat doit non seulement connaître .les techniques comme le "pinning", mais aussi comprendre leurs implications opérationnelles et les meilleures pratiques actuelles

.**Justification :** Aucune des options proposées n'est idéale

- .L'option A (TLS standard) est insuffisante car elle ne protège pas contre un trust store compromis
- L'option B (pinning du certificat feuille) est techniquement valide mais opérationnellement très fragile. Le renouvellement du certificat du serveur .(même prévu) obligerait à une mise à jour forcée de tous les clients, ce qui est irréalisable pour une flotte critique
- L'option C (pinning d'un CA racine public) est une erreur de conception. Un attaquant pourrait obtenir un certificat valide pour un autre domaine auprès du même CA, et cette technique ne l'empêcherait pas. Elle ne garantit pas que l'on parle au bon serveur, seulement à un serveur .approuvé par ce CA
- L'option D (HPKP) est une technologie obsolète et dangereuse, retirée des navigateurs et clients modernes en raison du risque élevé de ."suicide" par mauvaise configuration

La meilleure approche moderne, non listée, serait de **"pinner" la clé publique d'un certificat intermédiaire (Intermediate CA) contrôlé par l'organisation**. Cela offre un excellent compromis : les certificats feuilles du serveur peuvent être renouvelés fréquemment par cet intermédiaire sans nécessiter de mise à jour de l'application, tout en garantissant que seuls les certificats émis par l'autorité de confiance de l'organisation sont acceptés, .protégeant ainsi contre les CA malveillants

.La réponse correcte est : Aucune des réponses proposées

Note de -0,25 sur 1,00 Incorrect Question 41

: Contexte

En tant que développeur mobile expert pour la police nationale, vous êtes chargé de définir l'architecture de sécurité pour une nouvelle application native (Kotlin/Swift). Une exigence non-négociable est de protéger une clé d'API critique, utilisée pour accéder à des données sensibles, contre toute tentative d'extraction. Le modèle de menace principal stipule qu'un attaquant pourrait obtenir un accès physique à l'appareil d'un agent et utiliser des techniques avancées (root/jailbreak, analyse de mémoire, décompilation) pour trouver cette clé. .L'implémentation actuelle, jugée insuffisante, consiste à stocker la clé en clair dans les fichiers de configuration du projet

: Question

Conformément à vos missions de "Participer à la conception technique et à l'architecture" (activité 3) et d'"Effectuer une veille technologique régulière" (activité 9), laquelle des stratégies suivantes, implémentée entièrement côté client, offre la protection la plus ? robuste et la plus moderne contre l'extraction de la clé d'API sur un appareil compromis

- .a Utiliser des bibliothèques de préférences chiffrées (EncryptedSharedPreferences sur Android, ou une solution basée sur le .Keychain sur iOS) pour stocker la clé sur l'appareil

  Incorrect. Sur un appareil rooté/jailbreaké, les protections du système d'exploitation, y compris celles du Keychain, peuvent être contournées. La clé de chiffrement utilisée par EncryptedSharedPreferences est elle-même gérée par le Keystore d'Android, qui peut être compromis. Ce n'est pas une défense .suffisante contre un attaquant ayant des privilèges élevés

- .b .Aucune des réponses proposées
- .c Intégrer la clé directement dans le code source en tant que constante, puis utiliser des outils d'obfuscation avancés (comme R8 en mode complet sur Android, ou des outils tiers sur iOS) pour rendre la décompilation et la lecture de la chaîne de caractères extrêmement .difficiles
- .d .Stocker la clé dans le code natif (C/C++) via le NDK sur Android et une bibliothèque C sur iOS, puis y accéder via JNI/Interop

: Analyse technique

Cette question est conçue pour évaluer la compréhension approfondie de la sécurité côté client au-delà des techniques de surface. Toutes les options proposées représentent des couches de défense valides mais imparfaites, souvent utilisées mais qui ne constituent pas une solution "robuste" face à un attaquant déterminé sur un appareil compromis. Le principe fondamental de la sécurité moderne est de ne jamais considérer le client comme une .entité de confiance et, par conséquent, de ne jamais y stocker de secrets statiques à longue durée de vie

: Justification

La seule stratégie véritablement robuste consiste à ne pas stocker la clé d'API statique dans l'application. L'approche correcte, non listée, est une : architecture dynamique

.L'application authentifie l'utilisateur (par exemple, via OAuth 2.0) .1

L'application utilise une API d'attestation de l'OS (SafetyNet/Play Integrity sur Android, DeviceCheck sur iOS) pour prouver son intégrité et l'état de .2 .l'appareil au backend

.Le backend valide l'identité de l'utilisateur ET le rapport d'attestation .3

Si les deux sont valides, le backend génère un jeton de session à courte durée de vie (par exemple, un JWT) que l'application utilise pour ses appels .4 .API

Ce modèle garantit que même si un attaquant vole les identifiants de l'utilisateur, il ne peut pas utiliser un client non autorisé (car il échouerait à l'attestation) pour accéder à l'API. La clé statique n'existant pas côté client, elle ne peut pas être volée. Puisque cette architecture n'est pas proposée, ."Aucune des réponses proposées" est la seule réponse correcte pour un expert

.La réponse correcte est : Aucune des réponses proposées

Note de 0,00 sur 1,00 Incorrect **Question 42**

La police nationale souhaite pouvoir activer ou désactiver dynamiquement des fonctionnalités dans son application mobile en fonction du contexte opérationnel, via un service de configuration à distance (type Firebase Remote Config). Par exemple, activer un mode "Maintien de l'Ordre" qui débloque des cartes tactiques et des canaux de communication spécifiques lors d'une manifestation. Le risque principal est qu'un attaquant compromette ce service pour désactiver des sécurités, injecter une configuration malveillante (ex: rediriger les appels API vers un serveur pirate), ou empêcher les agents de recevoir la bonne configuration. Décrivez l'architecture de sécurité de bout en bout que vous mettriez en place pour cette fonctionnalité. Votre réponse doit détailler les mécanismes pour garantir : 1. L'authenticité et l'intégrité de la configuration reçue par le client. 2. Le contrôle d'accès et le principe de moindre privilège sur la plateforme de configuration. 3. La .résilience de l'application face à une configuration corrompue ou indisponible

Réponse : Question 42

Utilisé silent fcm notification l'App écoute c'est type de messages si il reçu bloqué ou débloquer

: La réponse correcte est

Note de 0,00 sur 1,00 Incorrect **Question 43**

La police nationale mauritanienne souhaite déployer une nouvelle application mobile pour ses unités de patrouille. Une fonctionnalité clé est la géolocalisation en temps réel des officiers pour améliorer la coordination et assurer leur sécurité. Cependant, cette fonctionnalité soulève d'importantes préoccupations en matière de sécurité et de vie privée : un adversaire pourrait tenter d'intercepter ces données pour suivre les mouvements de la police, ou le système pourrait être utilisé à mauvais escient pour surveiller les officiers en dehors de leurs .heures de service

En tant que développeur mobile principal, vous êtes chargé de concevoir l'architecture technique de bout en bout pour cette fonctionnalité : de géolocalisation sécurisée. Décrivez votre solution en vous concentrant sur les aspects critiques suivants

.1 **Confidentialité et Intégrité des Données :** Comment sécuriseriez-vous la transmission des coordonnées GPS du mobile vers les serveurs centraux, même sur des réseaux publics non fiables (3G/4G) ? Allez au-delà de la simple mention "utiliser HTTPS/TLS" et .détaillez les mécanismes spécifiques que vous mettriez en œuvre (ex: certificate pinning, chiffrement de bout en bout de la charge utile)

.2 **Mesures Anti-Spoofing :** Quelles mesures techniques implémenteriez-vous sur l'application mobile pour détecter et atténuer les ? tentatives de spoofing GPS, où un attaquant tenterait d'injecter de fausses données de localisation

.3 **Privacy by Design :** Comment architectureriez-vous le système (côté client et serveur) pour garantir que le suivi de la localisation est strictement limité aux heures de service ? Décrivez le mécanisme d'activation/désactivation et comment vous garantiriez la non- .persistance des données de localisation en dehors des besoins opérationnels

.4 **Considérations sur l'Architecture Backend :** Esquissez brièvement les composants clés du backend nécessaires pour supporter cette .fonctionnalité de manière sécurisée et à grande échelle (ex: choix de l'API, stockage des données, logique de contrôle d'accès)

Réponse : Question 43

té mobile par un paire keynle les données déchiffrer par le'serveur je veux utiliser mongodb et reste api nextjs

: La réponse correcte est

Note de 0,03 sur 1,00 Partiellement correct **Question 44**

La police nationale doit mettre en place un système d'alertes critiques (ex: "officier en difficulté", "localisation de suspect") envoyées via des notifications push aux applications mobiles des agents. Les mécanismes standards (APNs/FCM) ne garantissent ni la confidentialité du .contenu transitant par des serveurs tiers (Apple/Google), ni l'intégrité du message, ni une confirmation de lecture fiable

En tant qu'architecte de la solution, décrivez une architecture technique de bout en bout pour ce système de notifications. Votre réponse : doit impérativement adresser les trois points suivants

.1 **Confidentialité :** Comment protéger le contenu de la notification contre toute interception entre le serveur de la police et l'appareil de ? l'agent

.2 **Intégrité et Authenticité :** Comment garantir que la notification n'a pas été altérée et provient bien d'une source légitime ?

.3 **Fiabilité :** Comment obtenir une confirmation certaine que l'agent a bien reçu et visualisé l'alerte, malgré la nature "best-effort" des ? services de push

Réponse : Question 44

ı téléphone on utilise fcm comme source de resevalblite cré u api qui fait bulk des notifications a tou les agent

La réponse correcte est : Une architecture robuste utiliserait une approche multi-couches : 1. Confidentialité (Chiffrement de bout-en-bout - E2EE) : Le contenu sensible de l'alerte ne serait jamais envoyé en clair dans la notification push. Le backend chiffrerait la charge utile de l'alerte avec une clé symétrique (ex: AES-256) connue uniquement du client légitime. La notification push elle-même ne contiendrait que cette donnée chiffrée. L'application mobile, après avoir authentifié l'utilisateur, utiliserait la clé stockée de manière sécurisée (ex: Android Keystore / iOS Keychain) pour déchiffrer le message. 2. Intégrité/Authenticité (Signature de la charge utile) : Avant le chiffrement, le backend signerait la charge utile (ex: avec HMAC-SHA256 en utilisant un secret partagé, ou une signature RSA). L'application cliente, après déchiffrement, vérifierait cette signature pour s'assurer que le message est authentique et n'a pas été modifié en transit. 3. Fiabilité (Notification Silencieuse et Callback API) : Pour garantir la réception, on utiliserait une notification "silencieuse" (data-only). Cette notification ne contient pas de message visible mais réveille l'application en arrière-plan. Dès sa réception, l'application effectuerait immédiatement un appel API sécurisé vers le backend pour : a) Confirmer la réception (ACK), et b) Récupérer les détails complets de l'alerte. L'application afficherait alors une notification locale hautement prioritaire. Un second appel API serait effectué lorsque l'utilisateur interagit avec la notification pour confirmer sa visualisation. ستستخدم الهيكلية القوية نهجًا متعدد الطبقات: 1. السرية (التشفير من طرف إلى طرف - E2EE): لن يتم إرسال المحتوى" الحساس للتنبيه كنص واضح في الإشعار الفوري أبدًا. سيقوم الخادم الخلفي بتشفير حمولة التنبيه باستخدام مفتاح متماثل (مثل AES-256) معروف فقط للعميل الشرعي. سيحتوي الإشعار الفوري نفسه فقط على هذه البيانات المشفرة. سيقوم تطبيق الجوال، بعد مصادقة المستخدم، باستخدام المفتاح المخزن بشكل آمن (مثل Android Keystore / iOS

Keychain) لفك تشفير الرسالة. 2. السلامة/الأصالة (توقيع الحمولة): قبل التشفير، سيقوم الخادم الخلفي بتوقيع الحمولة (على سبيل المثال، باستخدام HMAC-SHA256 مع سر مشترك، أو توقيع RSA). سيقوم تطبيق العميل، بعد فك التشفير، بالتحقق من هذا التوقيع للتأكد من أن الرسالة أصلية ولم يتم تعديلها أثناء النقل. 3. الموثوقية (الإشعار الصامت واستدعاء API): لضمان الاستلام، سنستخدم إشعارًا "صامتًا" (data-only). هذا الإشعار لا يحتوي على رسالة مرئية ولكنه يوقظ التطبيق في الخلفية. عند استلامه، سيقوم التطبيق فورًا بإجراء استدعاء API آمن إلى الخادم الخلفي من أجل: أ) تأكيد الاستلام (ACK)، و ب) جلب التفاصيل الكاملة للتنبيه. سيقوم التطبيق بعد ذلك بعرض إشعار محلي ذي أولوية عالية. سيتم إجراء استدعاء API ثانٍ عندما يتفاعل المستخدم مع الإشعار لتأكيد الاطلاع عليه.

Note de 0,01 sur 1,00 Partiellement correct Question 45

Dans le cadre du développement d'une application mobile pour la police nationale, destinée à être utilisée en patrouille dans des zones à connectivité réseau intermittente ou nulle, vous devez permettre la saisie et le stockage local de rapports d'incidents contenant des données personnelles sensibles et des preuves photographiques. Décrivez l'architecture technique et les mesures de sécurité spécifiques que vous mettriez en œuvre pour garantir la confidentialité, l'intégrité et la résilience des données stockées sur l'appareil mobile avant leur synchronisation sécurisée avec les serveurs centraux. Votre réponse doit aborder le choix du stockage, les mécanismes de chiffrement, la gestion des clés et la stratégie de synchronisation.

Réponse : Question 45

cryptographie pour les données et gestion des appareils si l'un des téléphone perdu pour détruire a distances

Cette question évalue la capacité du candidat à concevoir une solution mobile sécurisée pour un cas d'usage critique et réaliste. L'évaluation se concentre sur la profondeur de la connaissance des mécanismes de sécurité spécifiques au mobile (stockage, chiffrement, gestion des clés) et sur la robustesse de l'architecture proposée pour gérer les contraintes de connectivité.

La réponse correcte est : Chiffrement BDD (SQLCipher) + stockage clés (Keystore/Keychain) + intégrité (HMAC) + synchronisation robuste. تشفير قاعدة البيانات (SQLCipher) + تخزين المفاتيح (Keystore/Keychain) + السلامة (HMAC) + مزامنة قوية.

Note de 0,03 sur 1,00 Partiellement correct Question 46

Vous êtes chargé de concevoir le système d'authentification et d'autorisation pour une nouvelle application mobile de la police nationale. Cette application doit communiquer via une API REST avec une base de données centrale contenant des informations d'enquête hautement confidentielles. Le modèle de menace principal est qu'un attaquant, ayant volé les identifiants d'un officier, tente d'accéder à l'API en utilisant un client non autorisé (ex: un script personnalisé, Postman). Décrivez en détail l'architecture de sécurité que vous mettriez en place pour contrer cette menace, en vous assurant que seules les instances légitimes et non-altérées de l'application officielle peuvent communiquer avec le backend. Votre réponse doit aller au-delà d'une simple authentification par identifiants et tokens JWT.

Réponse : Question 46

ions open source qu est déduit a la secretydu endpoint envoyé jwt dans le token si vient de mobile ou session

Cette question évalue la capacité à concevoir une architecture de sécurité de bout en bout pour une application mobile critique, en se concentrant sur la validation de l'intégrité du client et la protection contre l'usurpation d'identité au niveau de l'API. Un candidat expert doit démontrer une compréhension des mécanismes d'attestation de plateforme (Play Integrity/App Attest) en plus des pratiques standards de gestion de session et d'authentification multi-facteurs.

La réponse correcte est : Une solution experte inclurait : 1. Authentification Multi-Facteurs (MFA) : Combinaison des identifiants de l'officier avec un second facteur, comme la biométrie (via les API natives) ou un code OTP, pour sécuriser la connexion initiale. 2. Gestion de Tokens Sécurisée : Utilisation d'un couple de tokens (Access Token court, Refresh Token long) stockés de manière sécurisée dans le Keystore (Android) ou Keychain (iOS). 3. Attestation de l'Application et de l'Appareil : C'est le point crucial. Implémentation de l'API Play Integrity (Android) ou App Attest (iOS). Au login, l'application génère un nonce côté serveur, le transmet au client qui l'utilise pour obtenir un jeton d'attestation signé par Google/Apple. Ce jeton, prouvant l'intégrité de l'app et de l'appareil, est envoyé au backend pour validation avant d'émettre les tokens de session. 4. Signature des Requêtes API : Chaque requête API subséquente pourrait être signée avec une clé privée stockée dans le Keystore/Keychain, avec la clé publique correspondante connue du backend, pour se prémunir contre le rejeu d'attaques et garantir l'origine de la requête.الحل الخبير يجب أن يتضمن: 1. المصادقة متعددة العوامل (MFA) : دمج بيانات اعتماد الضابط مع عامل ثانٍ مثل القياسات الحيوية (عبر واجهات برمجة التطبيقات الأصلية) أو رمز OTP، لتأمين تسجيل الدخول الأولي. 2. إدارة آمنة للرموز (Tokens) : استخدام زوج من الرموز (رمز وصول قصير العمر، رمز تحديث طويل العمر) يتم تخزينهما بشكل آمن في Keystore (Android) أو Keychain (iOS). 3. توثيق التطبيق والجهاز (Attestation) : هذه هي النقطة الحاسمة. تنفيذ Play Integrity API (Android) أو App Attest (iOS). عند تسجيل الدخول، يقوم الخادم بإنشاء قيمة فريدة (nonce) وإرسالها إلى العميل، الذي يستخدمها للحصول على رمز توثيق موقع من Google/Apple. هذا الرمز، الذي يثبت سلامة التطبيق والجهاز، يتم إرساله إلى الخادم الخلفي للتحقق منه قبل إصدار رموز الجلسة. 4. توقيع طلبات API : يمكن توقيع كل طلب API لاحق بمفتاح خاص مخزن في Keystore/Keychain، مع معرفة الخادم الخلفي للمفتاح العام المقابل، للحماية من هجمات إعادة الإرسال وضمان مصدر الطلب.

Note de 0,00 sur 1,00 Incorrect Question 47

Une application mobile critique, utilisée par les unités d'intervention de la police pour la coordination en temps réel (partage de position, photos, messages), subit de graves latences et des déconnexions lors d'opérations à grande échelle. Le backend est une API RESTful. L'équipe backend rapporte une surcharge de requêtes API, notamment sur les endpoints de mise à jour de statut et de récupération de .données multimédias, et suspecte une implémentation inefficace côté client

En tant que développeur mobile expert, vous êtes chargé de diagnostiquer le problème et de proposer un plan d'action technique détaillé. : Votre réponse doit aborder les trois points suivants

.1 Votre méthodologie de diagnostic pour identifier les goulots d'étranglement (bottlenecks) au sein de l'application mobile, en précisant .les outils et métriques

.2 Au moins trois optimisations concrètes côté client pour réduire la charge sur l'API, en expliquant le compromis (trade-off) de chaque .optimisation

.3 Les mesures de sécurité spécifiques pour protéger les données sensibles dans le cadre de vos nouvelles stratégies de communication .(ex: mise en cache, traitement par lots)

Réponse : Question 47

: La réponse correcte est

Note de 0,00 sur 1,00 Incorrect Question 48

Vous êtes chargé de la conception de la sécurité d'une nouvelle application mobile destinée aux agents de la police nationale opérant en civil. Le modèle de menace principal suppose que l'appareil d'un agent puisse être saisi par une organisation criminelle techniquement compétente, qui tentera activement de procéder à la rétro-ingénierie de l'application. L'objectif de l'attaquant est de découvrir les protocoles de communication, d'extraire les clés d'API, et de comprendre la logique interne pour potentiellement intercepter des données ou usurper l'identité d'un agent. Décrivez une stratégie de défense en profondeur (defense-in-depth) pour protéger l'application au : moment de son exécution (runtime) contre les menaces suivantes

.1 **Analyse Statique :** tentatives de décompilation du paquet de l'application (APK/IPA) pour analyser le code

.2 **Analyse Dynamique :** utilisation d'outils comme Frida ou de débogueurs pour inspecter la mémoire et intercepter le trafic réseau (MitM)

.3 **Altération (Tampering) :** modification du code de l'application pour contourner les sécurités

Pour chaque point, détaillez les contre-mesures techniques spécifiques que vous mettriez en œuvre (ex: obfuscation, détection de .root/jailbreak, certificate pinning, anti-debugging, contrôles d'intégrité) et expliquez leur synergie

Réponse : Question 48

Cette question évalue la capacité à concevoir une architecture de sécurité mobile robuste, allant au-delà des pratiques standards. Elle vise à identifier les candidats qui comprennent les menaces avancées telles que la rétro-ingénierie et le "tampering", et qui peuvent articuler une stratégie de défense multicouche. Les points clés à évaluer sont la connaissance des techniques d'obfuscation, de détection d'environnement compromis (root/jailbreak), .de sécurisation des communications (certificate pinning), et de protection de l'intégrité de l'application au moment de l'exécution (runtime)

: La réponse correcte est

Note de 0,01 sur 1,00 Partiellement correct Question 49

Un officier de police sur le terrain signale la perte de son terminal mobile de service, qui contient une application critique donnant accès à des données d'enquête en temps réel et à des informations sur des informateurs (données mises en cache pour un fonctionnement hors ligne). Le terminal est présumé compromis mais est encore connecté au réseau. En tant qu'architecte de la solution, décrivez de manière hiérarchisée et détaillée le protocole technique de réponse à incident que vous déclencheriez immédiatement. Votre réponse doit non seulement lister les actions (ex: effacement à distance), mais aussi justifier leur ordre de priorité et spécifier les prérequis architecturaux .(côté application et backend) qui auraient dû être implémentés au préalable pour permettre une telle réponse sécurisée et efficace

Réponse : Question 49

rester device id comme impossible de contacter sur le système utilisé des sonneries fort si on est très proche

La réponse correcte est : La réponse doit être structurée en deux parties : le plan d'action immédiat et les prérequis architecturaux. Plan d'action immédiat (par ordre de priorité) : 1. Révocation des accès (Backend) : La toute première action est de révoquer immédiatement tous les jetons d'accès (JWT, OAuth) et les sessions associés à l'utilisateur et à l'appareil depuis le backend. Cela empêche toute nouvelle communication authentifiée de l'application vers les serveurs, coupant l'accès aux données fraîches. 2. Verrouillage à distance (via MDM/Push) : Déclencher un verrouillage à distance de l'appareil via une solution de Mobile Device Management (MDM) ou une commande push sécurisée. L'écran est verrouillé et demande un code de déverrouillage spécifique, rendant l'appareil inutilisable. 3. Activation du mode "leurre" ou "forensique" (Application) : Si implémenté, envoyer une commande push silencieuse qui met l'application dans un état de "leurre" (affichant de fausses données) ou qui déclenche la collecte et l'exfiltration discrète de données forensiques : dernières coordonnées GPS, adresses IP, tentatives de déverrouillage, et éventuellement une photo via la caméra frontale. 4. Effacement sélectif des données (Application) : Déclencher l'effacement du conteneur de données chiffrées de l'application. Ceci est préférable à un effacement complet immédiat car cela préserve le fonctionnement du reste de l'appareil, ce qui peut aider à sa localisation. 5. Effacement complet de l'appareil (MDM) : En dernier recours, si l'appareil ne peut être récupéré ou si les données sont d'une sensibilité extrême, déclencher un effacement complet (factory reset) du terminal. Prérequis Architecturaux : Côté Application : Chiffrement fort des données au repos (Data-at-Rest) dans un conteneur sécurisé (ex: Keychain sur iOS, Keystore sur Android) avec des clés de chiffrement non exportables. Détection de root/jailbreak et mécanismes d'anti-tampering (obfuscation du code, RASP - Runtime Application Self-Protection). Implémentation d'un "kill switch" ou d'une logique d'effacement de données déclenchable à distance via une notification push silencieuse et sécurisée. Certificate Pinning pour se prémunir des attaques Man-in-the-Middle. Côté Backend/Infrastructure : Intégration avec une solution MDM pour la gestion centralisée de la flotte mobile. Un système d'authentification robuste avec gestion des jetons à courte durée de vie et possibilité de révocation instantanée. Une API sécurisée permettant d'envoyer des commandes spécifiques aux appareils (verrouillage, effacement, etc.). Journalisation (logging) centralisée et détaillée de toutes les actions et accès pour l'analyse post-incident.يجب أن تكون الإجابة منظمة في جزأين: خطة العمل الفورية والمتطلبات الهيكلية المسبقة. خطة العمل الفورية (حسب الأولوية): 1. إلغاء صلاحيات الوصول (من جانب الخادم): الإجراء الأول على الإطلاق هو إلغاء جميع رموز الوصول (JWT, OAuth) والجلسات المرتبطة بالمستخدم والجهاز فورًا من الخادم. هذا يمنع أي اتصال مصادق عليه جديد من التطبيق إلى الخوادم، مما يقطع الوصول إلى البيانات الجديدة. 2. القفل عن بعد (عبر MDM/Push): تفعيل قفل الجهاز عن بعد عبر حل إدارة الأجهزة المحمولة (MDM) أو أمر دفع آمن. يتم قفل الشاشة وتطلب رمز فتح محدد، مما يجعل الجهاز غير قابل للاستخدام. 3. تفعيل وضع "التمويه" أو "التحقيق الجنائي" (التطبيق): إذا كان مطبقًا، إرسال أمر دفع صامت يضع التطبيق في حالة "تمويه" (يعرض بيانات مزيفة) أو يبدأ في جمع وإخراج بيانات التحقيق الجنائي الرقمي بسرية: آخر إحداثيات GPS، عناوين IP، محاولات فتح القفل، وربما صورة عبر الكاميرا الأمامية. 4. المسح الانتقائي للبيانات (التطبيق): تفعيل مسح حاوية البيانات المشفرة الخاصة بالتطبيق. هذا أفضل من المسح الكامل الفوري لأنه يحافظ على عمل بقية الجهاز، مما قد يساعد في تحديد موقعه. 5. المسح الكامل للجهاز (MDM): كملاذ أخير، إذا لم يكن من الممكن استعادة الجهاز أو إذا كانت البيانات ذات حساسية قصوى، يتم تفعيل المسح الكامل (إعادة ضبط المصنع) للجهاز. المتطلبات الهيكلية المسبقة: على مستوى التطبيق: تشفير قوي للبيانات في حالة السكون (Data-at-Rest) في حاوية آمنة (مثل Keychain على iOS و Keystore على Android) باستخدام مفاتيح تشفير غير قابلة للتصدير. كشف الـ root/jailbreak وآليات مكافحة التلاعب (إخفاء الكود، الحماية الذاتية للتطبيق وقت التشغيل - RASP). تنفيذ "مفتاح إيقاف الطوارئ" (kill switch) أو منطق لمسح البيانات يمكن تفعيله عن بعد عبر إشعار دفع صامت وآمن. تثبيت الشهادة (Certificate Pinning) للحماية من هجمات الرجل في المنتصف. على مستوى الخادم/البنية التحتية: التكامل مع حل MDM للإدارة المركزية لأسطول الأجهزة المحمولة. نظام مصادقة قوي مع إدارة رموز وصول قصيرة العمر وإمكانية الإلغاء الفوري. واجهة برمجة تطبيقات (API) آمنة تسمح بإرسال أوامر محددة للأجهزة (قفل، مسح، إلخ). تسجيل مركزي ومفصل لجميع الإجراءات والوصول لتحليل ما بعد الحادث.

Note de 0,00 sur 1,00 Partiellement correct Question 50

Pour une nouvelle fonctionnalité de l'application mobile de la police, l'équipe backend propose d'utiliser une API GraphQL pour permettre aux agents sur le terrain de requêter des informations complexes et interconnectées sur des suspects (identités, véhicules, relations, incidents passés). En tant que développeur mobile principal, vous devez collaborer avec l'équipe backend pour définir une architecture sécurisée. Décrivez votre stratégie technique pour atténuer les trois risques majeurs suivants, en précisant les contrôles à mettre en place côté client et ceux à exiger côté serveur :

1. **Déni de Service (DoS) par Requêtes Abusives :** Comment empêcher qu'une requête malveillante (trop profonde, trop complexe ou circulaire) n'épuise les ressources du serveur ?
2. **Fuite d'Informations via l'Introspection :** Quelles mesures prendre pour empêcher un attaquant d'utiliser les capacités d'introspection de GraphQL pour cartographier la totalité du schéma de données confidentielles ?
3. **Contournement des Autorisations :** Comment garantir qu'un agent ne puisse pas "traverser le graphe" pour accéder à des données pour lesquelles il n'a pas les droits (par exemple, consulter un dossier hors de sa juridiction) ?

Réponse : Question 50

Utilisé rate limite pour l'endpoint login

> L'évaluation portera sur la capacité du candidat à identifier des solutions spécifiques à GraphQL, au-delà des mesures de sécurité API génériques. La réponse doit démontrer une compréhension de la responsabilité partagée entre le client mobile et le backend, et proposer des contrôles techniques précis pour chaque risque identifié.

La réponse correcte est : Une excellente réponse doit proposer une stratégie de défense en profondeur : 1. Contre le DoS : Côté Serveur (à exiger) : Mettre en place une limitation de la profondeur (query depth limiting), une limitation de la complexité (query complexity analysis) en assignant un "coût" à chaque champ, et un timeout sur les requêtes. Côté Client (à implémenter) : Utiliser des requêtes persistantes (persisted queries) ou des listes blanches de requêtes (query whitelisting). L'application n'envoie qu'un identifiant de requête pré-approuvée, empêchant toute requête arbitraire et malveillante. 2. Contre la Fuite d'Informations : Côté Serveur (à exiger) : Désactiver complètement l'introspection GraphQL en environnement de production. L'équipe de développement mobile peut travailler avec un schéma exporté localement. Côté Client (à implémenter) : S'assurer que les outils de développement ou de débogage qui pourraient tenter d'utiliser l'introspection ne sont pas inclus dans les builds de production. 3. Contre le Contournement des Autorisations : Côté Serveur (à exiger) : L'autorisation doit être implémentée au niveau de chaque "resolver" GraphQL. Chaque champ doit vérifier les droits de l'utilisateur (via son token) avant de retourner une valeur. Côté Client (à implémenter) : L'interface utilisateur doit être conçue pour ne générer que des requêtes correspondant aux droits de l'utilisateur, ce qui constitue une première ligne de défense.يجب أن تقترح الإجابة الممتازة استراتيجية دفاع

متعمقة: ١. ضد هجمات الحرمان من الخدمة (DoS): جانب الخادم (يجب طلبه): تنفيذ تحديد لعمق الاستعلام (query depth limiting)، وتحديد لتعقيد الاستعلام (query complexity analysis) عن طريق تعيين "تكلفة" لكل حقل، ومهلة زمنية (timeout) للاستعلامات. جانب العميل (يجب تنفيذه): استخدام الاستعلامات المستمرة (persisted queries) أو قوائم الاستعلامات المسموح بها (query whitelisting). يرسل التطبيق فقط معرفًا لاستعلام تمت الموافقة عليه مسبقًا، مما يمنع أي استعلامات عشوائية وضارة. ٢. ضد تسريب المعلومات: جانب الخادم (يجب طلبه): تعطيل استبطان GraphQL بالكامل في بيئة الإنتاج. يمكن لفريق تطوير تطبيقات المحمول العمل باستخدام مخطط مُصدَّر محليًا. جانب العميل (يجب تنفيذه): التأكد من عدم تضمين أدوات التطوير أو تصحيح الأخطاء التي قد تحاول استخدام الاستبطان في إصدارات الإنتاج. ٣. ضد تجاوز الصلاحيات: جانب الخادم (يجب طلبه): يجب تنفيذ التفويض على مستوى كل "مُحلِّل" (resolver) في GraphQL. يجب على كل حقل التحقق من صلاحيات المستخدم (عبر الرمز الخاص به) قبل إرجاع قيمة. جانب العميل (يجب تنفيذه): يجب تصميم واجهة المستخدم لإنشاء استعلامات تتوافق فقط مع صلاحيات المستخدم، مما يشكل خط دفاع أول.

Note de 0,02 sur 1,00 **Partiellement correct** **Question 51**

Dans le cadre de la mise en place d'une nouvelle pipeline d'Intégration Continue et de Déploiement Continu (CI/CD) pour les applications mobiles critiques de la police nationale (développées nativement en Kotlin pour Android et Swift pour iOS), vous êtes chargé d'y intégrer une stratégie de sécurité robuste dès la conception ('Shift-Left Security'). Décrivez de manière détaillée les étapes de sécurité, les types d'outils et les contrôles que vous implémenteriez à travers cette pipeline. Précisez pour chaque étape (ex: commit, build, test) son objectif principal et le type de vulnérabilités qu'elle permet de prévenir, en justifiant vos choix technologiques.

Réponse : Question 51

Après chaque push sur main branche on fait des scan sur l'App s 'il contient vulnérables et utilisé

Une réponse excellente identifiera des étapes précises (scan de secrets, SAST, SCA, DAST), suggérera des outils spécifiques pour chaque plateforme (ex: MobSF, owasp-dependency-check, SonarQube), et expliquera leur positionnement stratégique dans la pipeline (ex: pre-commit, post-build) pour une détection précoce des vulnérabilités.

La réponse correcte est : Une réponse complète doit structurer la pipeline en plusieurs étapes clés : 1. Phase de Pré-Commit/Commit : Intégration de hooks Git pour le scan de secrets (ex: `git-secrets`, `TruffleHog`) afin d'empêcher la fuite d'identifiants (clés d'API, mots de passe) dans le code source. 2. Phase de Build : Analyse de Composition Logicielle (SCA) : Utilisation d'outils comme `OWASP Dependency-Check` ou `Snyk` pour scanner les dépendances (bibliothèques tierces) et identifier les vulnérabilités connues (CVEs). C'est crucial car une grande partie du code n'est pas écrite en interne. Analyse Statique de la Sécurité des Applications (SAST) : Intégration d'un outil comme `SonarQube`, `Checkmarx` ou l'analyseur statique de `MobSF` pour analyser le code source Kotlin/Swift à la recherche de failles de sécurité (ex: stockage non sécurisé, injections, mauvaise gestion des certificats). Cette étape est automatisée après chaque build. 3. Phase de Test : Analyse Dynamique de la Sécurité des Applications (DAST) : Déploiement de l'application sur un émulateur ou un appareil réel dans un environnement de test isolé. Utilisation d'outils comme `MobSF (Dynamic Analyzer)` ou `OWASP ZAP` pour intercepter le trafic réseau et tester l'application en cours d'exécution afin de détecter des vulnérabilités dynamiques (ex: validation de certificat SSL/TLS, fuites de données via le réseau, failles dans les API).يجب أن تنظم الإجابة الكاملة خط الأنابيب في عدة مراحل رئيسية: 1. مرحلة ما قبل الإيداع/الإيداع (Pre-Commit/Commit): دمج خطافات Git (Git hooks) لمسح الأسرار (مثل `TruffleHog`، `git-secrets`) لمنع تسريب بيانات الاعتماد (مفاتيح API، كلمات المرور) في الكود المصدري. 2. مرحلة البناء (Build): تحليل مكونات البرمجيات (SCA): استخدام أدوات مثل `OWASP Dependency-Check` أو `Snyk` لمسح الاعتماديات (المكتبات الخارجية) وتحديد الثغرات المعروفة (CVEs). هذا أمر بالغ الأهمية لأن جزءًا كبيرًا من الكود ليس مكتوبًا داخليًا. التحليل الثابت لأمان التطبيقات (SAST): دمج أداة مثل `Checkmarx`، `SonarQube` أو المحلل الثابت لـ `MobSF` لتحليل الكود المصدري لـ Kotlin/Swift بحثًا عن العيوب الأمنية (مثل: التخزين غير الآمن، حقن الأوامر، الإدارة السيئة للشهادات). تتم هذه الخطوة تلقائيًا بعد كل عملية بناء. 3. مرحلة الاختبار (Test): التحليل الديناميكي لأمان التطبيقات (DAST): نشر التطبيق على محاكي أو جهاز حقيقي في بيئة اختبار معزولة. استخدام أدوات مثل `MobSF (Dynamic Analyzer)` أو `OWASP ZAP` لاعتراض حركة مرور الشبكة واختبار التطبيق أثناء تشغيله للكشف عن الثغرات الديناميكية (مثل: التحقق من صحة شهادة SSL/TLS، تسريب البيانات عبر الشبكة، الثغرات في واجهات برمجة التطبيقات).